fdtdKit: taking openEMS towards the memory ceiling of an Apple M5

A bit-identical speed-up of the EC-FDTD engine, and two run-length levers on the modelling side

Angel Martinez
University of Texas at Austin

Work of 2026-10-28 to 2026-11-13; draft of 2026-11-13

Abstract

openEMS is a free electromagnetic field solver that uses the equivalent-circuit finite-difference time-domain (EC-FDTD) method. On an Apple M5 laptop its multithreaded CPU engine ran the four models of up to about a million cells studied here (two openEMS tutorials and two printed-circuit boards) at 268–379 million cell updates per second (MC/s; its best of 2, 4 and 8 threads), well below the roughly 2 000 MC/s that the machine's 153 GB/s of unified memory allows at the 74 bytes per cell and time step of our traffic model. We report changes to the CPU engine that bring the same models to 1 808–1 961 MC/s, 4.73–6.89× the speed of stock v0.37.0 built with Apple clang, and two refined tutorial models of 5–8 million cells to 1.95–2.72× (snapshot cg29 of 2026-11-05, paired with stock in the same rounds; on the patch tutorial the fork's base alone, with upstream's background dump writer, ran 1.13× as fast as stock in separate runs of 2026-11-03), with results identical bit for bit to stock for the same number of time steps; changes to the operator setup and the far-field post-processing that cut their times (4.1 s to 0.9 s and 17.9 s to 1.0 s on a 7.9-million-cell model; from the project log); an opt-in Metal engine that advances tiles of the grid several time steps at a time (the diamond tile schedule of luikore's openEMS pull request #228, without its code), identical bit for bit to the CPU engine when the CPU, like the GPU, flushes subnormal numbers; and two changes on the modelling side (a run-end criterion based on the port probes, and a mesh-line merging rule with a minimum cell size) that shorten the runs themselves. In the same paired runs the Metal engine ran the two large models at 2 879 and 3 193 MC/s, 1.47 and 1.46× the CPU engine at its best, and one board, a grounded-coplanar (GCPW) line, 1.17×; it was level on the other, a vertical SMA launch (1.00×, 1.07 million cells), and slower on the two tutorials (0.97 and 0.93×); a later build (cg31, 2026-11-11) raised the Metal engine's rates by 1.07–1.53× on the six models. In a sweep of three models over 0.1–7.9 million cells it was ahead in every round from 1.7–2.9 million cells on, depending on the model, by 1.25–1.52× above 3 million, and within 0.89–1.23× of the CPU engine below that. Unblocked, as a fused sweep, it reaches only 67–75 % of that sweep's traffic bound on the large models: on this machine, whose GPU shares the bus with the cores, the blocking carries the gain. Together, the CPU engine of cg18 (patches 0001–0006, the author's installed build until cg29 replaced it on 2026-11-06) and the toolkit's defaults since 2026-11-04 cut the solver time of a complete simulation of the SMA launch from 562.7 s with stock openEMS to 37.4 s (15.0×) and that of a 50 mm GCPW line from 76.5 s to 5.2 s (14.7×; 5.8× and 8.9× with the toolkit's defaults until 2026-11-04, a 0.02 mm floor and a hold of 1); with cg29 the Metal engine ran these default runs 1.22 and 1.37× faster than the CPU engine (paired per round, on a loaded machine). All these times are measured. We describe the measurement harness and the verification policy that made this safe, the mechanisms behind each gain (barriers and the neighbour flags that replaced them, the inner loop, work distribution, a fused E/H sweep that cuts the modelled DRAM traffic from 74 to 49 bytes per cell, an allocation-placement valley, the boundary and dispersive-material paths, coefficient tables, the GPU's diamond tiles), and what did not work, and repeat the main measurements, as a set of their own, on a second machine, an M5 Pro Mac mini, where the CPU engine of cg31 runs 2.7 to 14.1× as fast as stock at stock's best thread count, and time three later patches (0041 to 0043) there, with a paired comparison against upstream master's new CPU engine (ahead at 1 million cells, level to 14 % ahead at 8 million on empty and PML grids (0.93× on the PML grid at 5 threads), behind on an 8-million-cell dispersive case). Because on the M5 the memory bus of one machine bounds the cell rate on large grids (on the M5 Pro it held back only the largest grid measured, 16.8 million cells), we also outline, without measurements, how a domain-decomposed engine (openEMS has none today) could spread one model over a Thunderbolt-connected cluster of Mac mini or Mac Studio machines, each adding its own unified memory and bus. The engine patches, the benchmark harness, the result tables, the scripts and a command line that runs the harness and the checks (`emskit fdtd`) are published at `https://github.com/anmartinez1256/emsKit`.

1 Introduction

openEMS [1] is a widely used free finite-difference time-domain (FDTD) [2, 3] solver. The author runs it from a Python toolkit for printed-circuit-board RF work: 50 Ω grounded-coplanar waveguide (GCPW) lines with lossy conducting sheets, vertical and edge-mount SMA (SubMiniature version A) connector launches with a coaxial port and a perfectly matched layer (PML), coupled lines and cavities. The machine is an Apple M5 laptop (4 performance and 6 efficiency cores, 16 GB of unified memory on a single bus of about 150 GB/s [7], shared with the GPU). A survey of the toolkit's runs on it found 0.1–2.4 million cells, 41 000–165 000 time steps and 10–50 minutes per run at 117–460 million cell updates per second (MC/s; from the project log, not in `results/`). The two boards benchmarked here take 76.5 s and 562.7 s per full run with stock openEMS (measured, Table 11). A parameter study of a launch footprint is a day.

The question this work set out to answer was simply "how fast can openEMS go on this machine, without changing its answers?" The constraint matters: the solver is used as a design gate, its results are compared to measurements and to earlier runs, and a fast engine that moves the answer by a tenth of a decibel is a liability, not an asset. Every change to the solver reported here therefore had to reproduce the stock engine's port voltages and currents, S-parameters and field dumps to the bit for the same number of time steps (the two exceptions, the GPU engines, as Apple's GPU flushes subnormal numbers to zero, and a float accumulation order in the far-field transform, are called out where they appear; the fork's own Metal engine is held to the bit against the CPU engine run with the same flush). These changes make up a fork of openEMS, which together with the benchmark harness and the verification method used to build it is called fdtdKit (emsKit until 2026-11-05, the name its repository keeps; Section 14). fdtdKit is the FDTD kit of EMSKit, a family of electromagnetic solvers for Apple silicon; the family's other kits (femKit, momKit and sbrKit) are not part of this report.

Contributions.

- A benchmark harness that runs the stock and the modified engines in-process on the same models with a fixed number of time steps, waits for a quiet machine, and compares port probes and S-parameters bit for bit; HDF5 field dumps are compared dataset by dataset with `scripts/checks/h5cmp.py` (Section 3).
- A sequence of engine changes, each measured and verified on six models (Section 4): thread barriers and a per-extension hook plan, a rewritten inner loop, work stealing and an automatic thread count, vector paths for the Mur and uniaxial-PML (UPML) boundaries, a fused E/H sweep that lowers the modelled DRAM traffic from 74 to 49 bytes per cell, the SSE engines' use of upstream's precomputed dump gather, staggered allocations that avoid an allocation-placement valley, a scalar-SSE path and compressed tables for PML slabs, line hooks for the dispersive-material extension, packed coefficient tables, and thread quality-of-service on macOS. Together (snapshot cg14, against stock v0.37.0 built with Apple clang): 3.2–6.9× on the four models of up to about a million cells, 1.7–2.5× on the two of 5–8 million cells, where stock already ran at up to half the bus (Section 2). On the patch tutorial about 1.13× of the 6.9× comes from the fork's base, upstream's `gpu-engine` branch, whose HDF5 writer writes that model's time-domain dumps in the background; on the notch tutorial, the GCPW line and patch x5 the base runs within 0.98–1.03× of stock (Section 8). Snapshot cg18 (patches 0001–0006, the build installed on the author's Mac from 2026-11-04 to 2026-11-06) gives 3.1–6.8× on the same four models against the stock rows of its check tables. Snapshot cg29 (patches 0001–0027, the installed build from 2026-11-06 to 2026-11-11) adds the CPU items of a second review and ideas from other contributors' work (Section 4.14): an operator index per z-vector, neighbour flags in place of the two barriers per time step, a y-row path for UPML boxes, merged ADE updates for conducting sheets, geometry coefficient tables and line hooks for the steady-state detection and Debye materials. Paired with stock in the same rounds (cg29 at 8 threads, its best setting; stock at its best of 2, 4 and 8 threads), it runs the four models 4.73–6.89× and the two large ones 1.95× and 2.72× as fast as stock (Table 10); paired with cg18 in rounds of its own (5 rounds, other processes on 0.3–0.6 cores; the SMA launch on its 0.871 M-cell mesh), 1.03–1.23× as fast at 4 threads and 1.13–1.44× at 8 over the six models. An opt-in tiled sweep (patch 0040 of 2026-11-11, not in the installed build) runs several time steps on one cache-sized tile that all threads share; at 4 threads it is ahead of the fused sweep at its best on two large grids (1.08 and 1.23×) and behind on four other models (0.71–0.92×; Section 9.6). Three further patches (0041 to 0043, written on 2026-11-12 and 2026-11-13, the fork's head, 13350ca, not installed) start the automatic thread search at every core outside the efficiency class and pack the state of Debye, Lorentz and Drude cells for the line hooks: on the M5 Pro the search then runs at 0.97–1.00 of a fixed 15 threads where cg31's ran at 0.81–0.96, and upstream's dispersive case ran 1.9× as fast at 1 M cells (Section 10.1).
- Setup and post-processing: a threaded operator construction (4.1 s to 0.9 s on the 7.9-million-cell patch model), a far-field transform written as complex matrix products, one per plane and block of directions (17.9 s to 1.0 s on the same model; this path needs Apple's Accelerate, other builds keep the original loop),

and two small CSXCAD changes (times from the project log, not in `results/`).

- An analysis of the ceiling (Section 5). On a 201^3 box the CPU engine (snapshot cg9, 8 threads), upstream's Metal engine from its pre-merge `gpu-engine` branch, and a standalone Swift/Metal kernel that runs the exported openEMS operator bit-identically reached 1 562, 1 638 and 1 670 MC/s (from the project log, not in `results/`). By the traffic model the two Metal kernels move 74 B per cell (separate E and H passes) and the fused CPU sweep 49 B, so the GPU drew more of the bus for about the same cell rate; no byte count was measured with hardware counters. On all six test models the CPU engine of cg14 and of cg18 is faster than that Metal engine (Section 5.4). Upstream master's merged Metal engine and luikore's diamond-blocked one were built and run here on 2026-11-04 (Section 11), and on 2026-11-11 next to cg31 on luikore's benchmark model (Table 19).
- A diamond-blocked Metal engine (`--engine=metal-diamond`, opt-in, Apple only; patches 0007–0016, Section 6) that advances tiles of the grid several time steps at a time, with the tile schedule of luikore's openEMS pull request #228 [20] (none of its code is used). All six models run on the GPU, identical bit for bit to the multithreaded CPU engine run with `OPENEMS_FTZ=1`; against stock, which keeps subnormal numbers, its port probes differ by 1.7×10^{-7} to 9.4×10^{-7} relative. Paired with the CPU engine of cg29 at its best (Table 10) it runs patch x5 and notch x3 1.47× and 1.46× as fast and the GCPW line 1.17×, is level on the SMA launch (1.00×, 1.071 M cells) and slower on the notch and patch tutorials (0.97× and 0.93×). Across grid sizes (Figure 3) it leads in every round from 1.7–2.9 million cells on, depending on the model, and by 1.25–1.52× above 3 million; on smaller grids it waits for the CPU between short calls (the patch tutorial) or runs at a lower rate of its own (the notch tutorial), and the CPU engine is at its fastest between 0.6 and 1.9 million cells. Unblocked, as a fused sweep ($K = 1$), the Metal engine runs the two large models at 1 730–1 840 MC/s, only 67–75 % of that sweep's traffic bound (cg27, medians of paired rounds with 2.5–4 cores of other load; `results/fused_sweep_k1_2026-11-04.txt`): the blocking carries the gain. A later build, cg31 (patches 0028–0039 of 2026-11-11, the installed build since that day), raises the Metal engine's rates on the six models by 1.07–1.53×, which puts it ahead of the CPU engine on five of them and level on the patch tutorial (Section 9).
- Two modelling-side levers (Section 7): a run-end criterion on the port probes (Table 6; the energy criterion stalls on the static charge left behind by openEMS's Gaussian pulse, which has a DC component, so the runs always went to their maximum time; upstream has since added an opt-in DC-free pulse, commit d686f1d, which is not in this work's base), and a mesh-line merging rule with a minimum cell size whose sweep on the SMA launch gives another 2.0× for 0.12 dB of S_{11} at −23 dB (Section 7.3). Since 2026-11-04 the toolkit's defaults use the shorter settings of both (a hold of 0.5 instead of 1; on the SMA launch a 0.05 mm floor instead of 0.02 mm); together with the CPU engine of cg18 they cut a complete simulation of the SMA launch from 562.7 s with stock to 37.4 s and one of the GCPW line from 76.5 s to 5.2 s (Table 11), and with cg29 the Metal engine ran these runs 1.22× and 1.37× as fast as the CPU engine (Table 12, paired per round, on a loaded machine).
- A list of things that did not work, with numbers (Section 12), the installed build and the tiled sweep measured again on a second machine, an M5 Pro Mac mini, as a set of their own, where the CPU engine of cg31 runs 2.7 to 14.1× as fast as stock at stock's best thread count, together with the build of patches 0041 to 0043 and a paired comparison with upstream master's `cpu` engine (Section 10), and the artefact set (Section 14).

Scope. All numbers are from one machine, an Apple M5 MacBook with 16 GB, macOS 27.0.1, Apple clang 21, except two validation-suite runs made on a Linux virtual machine (Section 14) and Section 10, which measures the same library files on a second machine, a Mac mini with an M5 Pro (5 + 10 cores, 24 GB, macOS 27.2.1); its numbers are a set of their own and are not compared with the M5's. The table of Section 9.6 was measured on the M5 under macOS 27.2.1. They were measured while the machine was also used for other work (the harness reports the load of other processes with every number: sampled in the second before each run and, since the cg27 tables of 2026-11-04, their CPU and GPU use during it, Section 3). Stock openEMS and the fork were both built with Apple clang, the default on macOS; upstream's FAQ [19] advises building openEMS with GCC on macOS because clang generates inefficient code for the SSE engine. That was measured for the inner loop only, in the standalone kernel, which holds a copy of the stock loop and of its rewrite (one thread, three grids outside the valley; `results/kernel_gcc_vs_clang_2026-11-03.txt`): GCC 16 runs the stock loop 1.16–1.39× faster than Apple clang, and the rewrite of snapshot AB (Section 4) at about the same speed with both, so against a GCC-built stock the rewrite's one-thread gain in the kernel is 1.26–1.47× instead of 1.74–1.80×. In the same kernel the rewrite gives the stock loop's results bit for bit under clang but not under GCC, which contracts it differently (Section 4.11). A GCC build of openEMS itself was not made, so the factors of this paper are against stock built with Apple clang, the default compiler on macOS, and the bit-identity checks used clang builds. The synchronisation and hook changes do not depend on the compiler. Six details are platform-specific and guarded: thread QoS (macOS), the starting

point of the automatic thread search (since patch 0041 the number of cores outside the efficiency class on macOS, before it the performance-core count; half the available hardware threads elsewhere; since cg29 the flag waits on macOS spin only while the thread count is at most that count, and other CPUs keep the barrier spin time), the separable far-field path (built only with Apple's Accelerate; other builds keep the original loop), the opt-in flush-to-zero of subnormals (any arm64 CPU), the Metal diamond engine (Apple only, opt-in, built with `ENABLE_METAL`) and the cache size that the opt-in tiled sweep's `auto` rule reads (the performance cores' L2 cache on macOS, elsewhere the last-level cache where the C library reports one). The fused sweep's mechanism, fewer passes over the fields, is not specific to this CPU, but it was measured only on Apple silicon: on this M5 and, as a set of its own, on the M5 Pro of Section 10. The allocation-placement valley depends on where the allocator puts the two field arrays and was seen only with macOS's allocator: under 27.0.1 on the M5 (Section 4.6) and, on the GCPW line, under 27.2.1 on the M5 Pro (Section 10); the stagger that avoids it is applied on every platform and costs nothing measurable outside the valley.

2 Background

2.1 The EC-FDTD update and its memory traffic

openEMS stores voltages v (the E-field integrated along a cell edge) and currents i (the H-field integrated along a dual edge), three components each, as 32-bit floats, and advances them with the usual leap-frog:

$$v_n \leftarrow vv_n\, v_n + vi_n\,(\text{curl of } i \text{ around the edge}), \tag{1}$$

$$i_n \leftarrow ii_n\, i_n + iv_n\,(\text{curl of } v \text{ around the dual edge}), \tag{2}$$

with four coefficients per component and cell. Stock openEMS already keeps the coefficients in a *compressed* operator (`Operator_SSE_Compressed` in v0.37.0): `f4vector`s of four cells (Section 2.2) whose coefficients agree lane by lane share one entry of a table, and each `f4vector` carries a 32-bit index into it, 1 B per cell. The index array itself is allocated with one 32-bit entry per cell, of which one per vector is used; on meshes with few z lines the unused entries share cache lines with the used ones, so the engine streams up to the full 4 B per cell (in stock and in the fork up to cg28; cg29 allocates one entry per z-vector, Section 4.14). The table is small (2 500–69 500 entries for the models here, under 14 MB: 1.5 MB for the tutorials, 13 MB for the SMA launch; from the project log) and largely cache-resident; the per-cell traffic that remains is the fields themselves. With separate E and H passes, each pass reads and writes its own field (12 B each way) and reads the other field (12 B) from DRAM; with the index that is 37 B per update and roughly 74 bytes per cell and time step. This is a traffic model, not a counter measurement, as are all bytes-per-cell figures in this paper; it counts the compact 1 B of index, and Section 4 also gives the figures with the index as allocated. At 150 GB/s, a sweep that moves these 74 bytes per cell from DRAM in every time step runs at 2 000 MC/s at best [6] (an engine that keeps a tile's fields on chip over several time steps moves fewer, Sections 6 and 9.6); the stock engine's 1 041 MC/s on a 7.9-million-cell model is 77 GB/s, half the bus. Smaller models fit in part in a last-level cache of a few tens of MB and could in principle run faster; stock ran them *slower* (277–318 MC/s), which is the first hint that something other than bandwidth dominated there.

2.2 Engines, operators, extensions

openEMS separates the operator (coefficients and mesh, built once) from the engine (the time loop). The SSE engines work on `f4vector`s, four floats in one 16-byte vector. With $n_v = \lceil N_z/4 \rceil$ z-vectors per z-line, one vector holds the cells z, $z + n_v$, $z + 2n_v$ and $z + 3n_v$; the arrays are stored component-fastest (C order `[x][y][z_v][component]`, the I-J-K-N layout of Section 4), and the z extent is padded to $4n_v$. Boundary conditions, excitations, lumped elements, dispersive materials and the cylindrical multigrid are *extensions*, each with up to six hooks per time step (pre-/apply-/post- for voltages and for currents). The multithreaded engine splits the x range into one slab per thread and runs each hook of each extension between barriers.

2.3 The machine

Apple M5 (2025): 4 performance (P) and 6 efficiency (E) cores, 16 GB LPDDR5X unified memory, 153 GB/s nominal [7]. The GPU has no memory of its own; it competes with the cores for the same bus. We measured the ceiling with two standalone kernels (a 200^3 box with perfect-electric-conductor (PEC) walls, no PML; from the project log, not in `results/`): a material-table CPU loop reaches 1 491 MC/s at 72 bytes per cell (107 GB/s) and the same kernel in Metal 1 763 MC/s (127 GB/s). The GPU gets 15–20 % more of the bus than the cores, but no bandwidth of its own. Everything that follows is therefore about moving fewer bytes, fewer times, with fewer stalls.

3 Method: how speed was measured and correctness enforced

3.1 Models

Six models span the sizes and features that matter (Table 1). Four are the openEMS Python tutorials, two of them with the mesh refined ("x5", "x3") to a size that is clearly bandwidth-bound on the M5 (on the M5 Pro they still scale to 15 threads, Section 10); two are the author's real boards, built by the author's toolkit. Every feature in daily use is covered: Mur and PML boundaries (full-z slabs and a z-only slab), lumped, microstrip-line (MSL) and coaxial ports, conducting sheets (the dispersive-material extension), near-field-to-far-field (nf2ff) dumps, frequency-domain field dumps. The cylindrical multigrid was checked separately on openEMS's CylindricalWave_CC tutorial.

model	origin	cells	features exercised
patch tutorial	Simple_Patch_Antenna.py	0.104 M	lumped port, 6× MUR, nf2ff box (12 time-domain dumps)
notch tutorial	MSL_NotchFilter.py	0.270 M	MSL ports, PML_8 at both x ends, PEC at $z_{\min}$, MUR elsewhere
patch x5	patch, mesh ×5	7.86 M	bandwidth-bound size, MUR
notch x3	notch, mesh ×3	5.25 M	bandwidth-bound size, PML slabs
GCPW line (kgcpw)	rfsim gcpw_line	0.242 M	conducting sheets (dispersive extension), MSL ports, MUR, 17 z lines
SMA launch (ksma)	rfsim sma_launch	1.071 / 0.871 M	coaxial port, z-only PML_8, 69 500 / 64 410 distinct operators, FD dumps

Table 1: Benchmark models. "M" = million Yee cells. The SMA launch has two meshes: 1.071 M cells with the toolkit's former 0.02 mm mesh floor, the mesh of the benchmark tables up to snapshot cg18, and 0.871 M with the 0.05 mm floor that is the toolkit's default since 2026-11-04 (toolkit commit 4a7e20e, Section 7.3), on which the benchmark model has been built since then: the GPU engine's measurements (Section 6), the other engines' runs (Section 11), the checks of cg29 and its comparison with cg18 (`results/cpu_items_cg29_2026-11-05.txt`) use it. Tables that use the 1.071 M mesh say so. Rates on the two meshes do not compare directly. Operator counts: 69 500 on the 1.071 M mesh (project log, not in `results/`), 64 410 on the 0.871 M mesh (`results/diamond_engine_cg24_cg27_2026-11-04.txt`).

3.2 The harness

`bench.py MODEL --configs CFG...` builds the model in a fresh Python process per run (`runner.py`) and calls `FDTD.Run()` with a fixed number of time steps (`EndCriteria=0`), so every engine performs identical work. A configuration names the library to load (`SNAP@`, a saved set of `libopenEMS`, `libCSXCAD`, `libnf2ff` dylibs picked up through `DYLD_LIBRARY_PATH`), the engine, the thread count, and environment variables for individual knobs (`#NAME=VAL`). The first configuration is the reference; for every other run the harness reports the maximum relative difference of the port probe files and the largest difference of S_{11}/S_{21} (linear and dB, over the frequencies where $|S_{\mathrm{ref}}| > -40\,\mathrm{dB}$), the engine speed in MC/s parsed from openEMS's own output, and the CPU load of *other* processes in the second before the run (`others NN%`, from `top`). A probe file whose reference is all zero, and an S-parameter that stays below $-40\,\mathrm{dB}$, cannot be compared that way, and the published tables leave them out without a note: the notch x3 check ends before the pulse reaches port 2, so its tables compare neither the port-2 probes nor S_{21}, and the GCPW tables do not compare S_{11}. `--quiet 60` waits until other processes use less than 60 % of a core, for at most `--quiet-max` seconds; `--repeat 2` runs each configuration twice and reports the faster run (both speeds are listed), and in the tables in `results/` only each configuration's last run was compared. The harness now compares every probe file and S-parameter exactly, over all frequencies, and names those without a figure in its output; it also compares every run with the reference configuration's first run and flags the reported run when its quiet wait timed out. Since 2026-11-04 the two GPU engines (`gpu`, `metal-diamond`) and any configuration with `OPENEMS_FTZ` set count as tolerant: their differences from a stock reference are printed but do not fail the run, since neither is bit-identical to a run that keeps subnormal floats (the GPU engines' reference is the CPU engine run with `OPENEMS_FTZ=1`; Section 3.3). From the cg27 tables of the same day on, each line also gives the CPU and GPU use of the other processes during the run (`bench/usage.py`), following the author's rule of that day that every benchmark line carries both. A knob's value cannot contain a comma, because the harness splits its settings on commas; a tile setting `M,V` of the GPU engine was once silently dropped that way (the engine now also reads `M:V`). From cg24 (2026-11-04) most speed comparisons are paired and interleaved (`bench/abtest.py`): the configurations run in turn, round after round, and a ratio is the median over the rounds of the per-round ratio of two configurations, given with its range, so that a slow phase of the shared machine hits both configurations of a round alike. The unpaired runs of 2026-11-04 and 2026-11-05 (the faster of two runs, or one run each: the tile sweep of cg23, the other engines, the Swift diamond mode, part B of the $K = 1$ file, the GPU end-to-end runs of cg23 and the check tables) say so where they are quoted. Runs go through `FDTD.Run()` rather than `Write2XML` plus the binary because in v0.37.0 (with CSXCAD v0.7.0), and so in this fork, `Write2XML` writes primitive coordinates

with 7 significant digits but mesh lines with 15, so zero-thickness boxes (an MSL port's excitation plane) fall off their mesh line and are dropped, reported only as unused primitives. Upstream master has since added a drawing tolerance taken from the mesh (commit db68e62, 2026-10-21) that covers this case.

`check.sh TAG SNAP` is the acceptance test: the six models at 1, 4, 8 and 10 threads and with the automatic thread count, against the stock snapshot, two repeats, results in the top-level `results/` folder as `MODEL_sS_nN_TAG.txt`. A snapshot was accepted only when every comparison printed `probes rel=0.0e+00` and `max|dS|=0.0e+00`; `bench.py` and `check.sh` now also exit with an error status when a comparison differs or a file is missing. cg18 (patches 0001–0006, the build installed on the author's Mac from 2026-11-04 to 2026-11-06) passed the full check on a quiet machine on 2026-11-03 (`results/*_cg18.txt`, Section 8). Its successor, cg29 (patches 0001–0027, the installed build from 2026-11-06 to 2026-11-11), passed it on 2026-11-05 with three more configurations, the geometry coefficient tables forced at 4 threads, the packed ones at 8 and the barriers instead of the neighbour flags at 8 (`results/*_cg29.txt`; the speeds in those tables are the faster of two unpaired runs under the check's load, not comparisons). Its other checks (`results/cpu_items_cg29_2026-11-05.txt`, section 3): `scripts/checks/debye_check.sh`; the validation suite, 15 of 15 with the multithreaded engine at 4 threads and with the fastest engine; the cylindrical multigrid, 34 datasets equal to stock's; box models with conducting sheets, Drude/Lorentz blocks, UPML, Mur faces and the steady-state detection against the basic or sse-compressed engine of the same build, whose extension code is stock's; and a matrix of 45 small models in up to 11 run modes, 461 comparisons identical and 10 that differ only in the stop step of one model, for the reason given under "Energy end criterion" below. The Metal engine of Section 6 has its own acceptance test, against the multithreaded engine run with `OPENEMS_FTZ=1` instead of stock (Section 3.3): `scripts/checks/diamond_check.sh` compares every exported field value of box models (PEC, Mur and PML faces, delayed and soft excitations, lumped RLC elements, conducting sheets, Drude/Lorentz and Debye blocks) at $K = 1$, 2, 3, 4 and 8 time steps per tile and with an odd tile, threadgroup and batch size, 72 of 72 comparisons identical on cg28, on cg29 and on every snapshot of cg31, and `bench.py` compares the port probes and S-parameters of the six models, identical on cg29 (`results/*_cg29gpu.txt`) as on the earlier GPU snapshots. The cg29 runs set `OPENEMS_FTZ=1` for the whole call (`results/cpu_items_cg29_2026-11-05.txt`, section 3), so the reference is tolerant too, and a tolerant reference makes every configuration tolerant: the files end with `NOTHING REQUIRED TO MATCH`, and the identity is in each line's `probes rel=0.0e+00` and `max|dS|=0.0e+00`. When cg29 was installed on 2026-11-06 (its openEMS library and `openems.h` replaced cg18's; the install's CSXCAD and nf2ff libraries already had cg29's bytes; the Python modules stayed the v0.37.0 binding, which affects SAR but not FDTD runs or nf2ff, `BUILD.md` section 6), it was checked from the install with the command line of Section 14: the reduced check (`check.sh` with `LITE=1`: 4 and 8 threads and the automatic count, one run each) identical to stock on all six models, `debye_check.sh` passed (with the same fields as the cg29 snapshot), `diamond_check.sh` 72 of 72, the validation suite 15 of 15 with the fastest engine and with the multithreaded engine at 4 threads, and the GCPW end-to-end run with the toolkit's defaults stopped at step 34 300, as on cg18 (Table 11). Other processes used 4.6–8.5 cores during these runs, so their speeds are not quoted, and their tables are not in `results/`; `Docs/fdtdKit/PLAN.md`, section 16, logs the runs. On 2026-11-07 the 27 patches published then (0001–0027) were built from a fresh clone of upstream's superproject into an empty prefix (`BUILD.md`, sections 1, 4 and 5; snapshot pub29), and that build passed `check.sh` (every run identical to stock on all six models, `results/*_pub29.txt`), `debye_check.sh`, `diamond_check.sh` 72 of 72, the six models on the Metal engine, the validation suite 15 of 15 with the multithreaded engine at 4 threads and with the fastest engine, and the cylindrical multigrid (`results/fresh_build_pub29_2026-11-07.txt`); other programs used up to 8.9 cores during `check.sh`, so here too only the verdicts count. The installed build, cg31 (patches 0001–0039, installed in cg29's place on 2026-11-11), passed the reduced check (identical to stock on all six models, 18 of 18 rows), `debye_check.sh`, `diamond_check.sh` (72 of 72 on each of its snapshots and on the installed library), the validation suite on the Metal engine (15 of 15, on snapshot cg30l, patches 0001–0038) and both boards end to end with the toolkit's defaults, which stopped at the steps of cg29 (34 300 and 50 320; Section 9, `results/metal_engine_cg31_2026-11-11.txt`). Patch 0040, the opt-in tiled sweep (snapshot cg32d, not installed), was checked both ways: with its switch unset the reduced check is identical to stock on all six models (18 of 18 rows), and with it set the probes of six models and the raw fields of a closed cylinder and of a steady-state run equal those of the fused sweep (Section 9.6, `results/tiled_sweep_cg32_2026-11-11.txt`). Patches 0041 to 0043, the fork's head (snapshots cg33a to cg33d and cg33, not installed), were checked on the M5 Pro on 2026-11-13: the reduced check, the Debye check, the diamond check, the validation suite, six models in eight configurations and the cylindrical multigrid example are identical to stock as before, and a new script, `scripts/ade_check/ade_check2.py`, compares the dispersive cases (the packed hooks of patch 0042 against stock, cg31, the unpacked paths and the other engines, with a guard that stops a row whose probes are all zero) in 66 of 66 rows at 37 mesh lines and in the rows with a source at 101 lines (Section 10.1; `results/cpu_items_cg33_2026-11-13.txt`).

3.3 Verification policy

- **Bit identity, not tolerance.** Floating-point FDTD is sensitive enough to rounding that a reordered sum shows up at 10^{-7} after a few thousand steps and grows. Every CPU change keeps the *same* float operations in the *same* order per cell; only who executes them, when, and from which copy of the coefficients changes. Hence the comparison is exact, and it caught a real bug: a hook plan built before the cylindrical multigrid extension registered, which made two snapshots wrong on that example only. The Metal engine of Section 6 is held to the same rule where the arithmetic can match: its kernels give every cell the CPU engines' operations in the same order, with `fma()` exactly where clang fuses on arm64 and Metal's fast math off, so its fields must equal those of the multithreaded engine run with `OPENEMS_FTZ=1`, which flushes subnormal floats to zero as Apple's GPU does. Where the arithmetic cannot match (a GPU engine against a CPU run that keeps subnormals, stock included, or another implementation), the harness reports the largest relative difference of the port probes and the largest S-parameter difference in dB, and these are measurements, not acceptance criteria ("Named exceptions" below).
- **Field dumps too.** HDF5 time- and frequency-domain dumps are compared dataset by dataset with `scripts/checks/h5cmp.py`, run by hand rather than by the harness. On the cylindrical example the earlier checks compared 18 datasets (project log); the check of cg18, a run of 1 500 steps with a frequency- and a time-domain dump, compared 34, all equal to stock's (`results/correctness_cg18_2026-11-03.txt`), and so did that of cg29 (`results/cpu_items_cg29_2026-11-05.txt`, section 3).
- **Named exceptions.** No GPU engine measured here is bit-identical to stock. Upstream's GPU engine in the fork's base (`--engine=gpu`, the pre-merge `gpu-engine` branch, commit 7bb294f) differs from the CPU engines by up to 2.3×10^{-4} dB in S_{11} (notch tutorial, `results/notch_s1_n16000_cg2.txt`). The cause is not the order of operations, which the Metal kernels follow operation by operation, but that Apple's GPU flushes subnormal floats to zero while the CPU cores keep them (Section 5): in a 3 000-step check on the notch tutorial (snapshot cg17, run in the final review of 2026-11-03, `results/notch_s1_n3000_ftzgpu.txt`), the multithreaded engine at 2 threads and the sse-compressed engine, both with `OPENEMS_FTZ=1`, gave port probes and S_{11} identical to the GPU engine's, while the multithreaded engine without it differed by 3.2×10^{-7} relative in the probes and 9.3×10^{-6} dB in S_{11}; field dumps were not compared in that check. On 2026-11-04 the same held on all six models: with `OPENEMS_FTZ=1` on the CPU, the base's GPU engine and the Metal diamond engine both gave port probes and S-parameters identical to the multithreaded engine's (`results/*_cg22gpu.txt`, `results/diamond_engine_cg22_cg23_2026-11-04.txt`). Against stock without the flush the diamond engine differs by 1.7×10^{-7} to 9.4×10^{-7} relative in the port probes, by at most 2.3×10^{-4} dB in S_{11} (notch tutorial; at most 2.3×10^{-5} dB on the other models) and by at most 9.4×10^{-5} dB in S_{21} (`results/*_cg22.txt`; its results have not changed since). Its nf2ff time-domain dumps were compared too: identical on the patch tutorial (984 datasets) and patch x5 (120) when both runs set `OPENEMS_FTZ=1`, because the dumps are computed on the creating thread, which the switch also flushes; with the flush in the CPU run only, 27 of the tutorial's 984 datasets differed while the fields were identical (`results/diamond_engine_cg20_2026-11-04.txt`). Upstream master's reworked GPU engine (merged as c3a956b on 2026-10-17) was measured on 2026-11-04 in a build of master 5fecbb5 (`results/others_2026-11-04/`, Section 11): on five of the six models its differences from stock are, to the printed digits, those of the diamond engine above; on the SMA launch master's CPU engine differs from stock as well (1.5×10^{-4} relative in the probes, 3.4×10^{-3} dB in S_{11}), a change in master since v0.37.0 that its GPU engine shows too. It was not compared with a CPU run with `OPENEMS_FTZ=1`. The new nf2ff path accumulates in BLAS order and differs from the legacy loop by up to 7×10^{-7} of the pattern maximum in the project log, and by 4.5–6.8×10^{-7} on the patch tutorial with cg18 (`results/correctness_cg18_2026-11-03.txt`); a weak cross-polar component can differ more relative to its own peak. The legacy loop is kept behind `OPENEMS_NF2FF_LEGACY=1` and is itself bit-identical (also checked in that file).
- **Energy end criterion.** The fork's base checks the energy end criterion every Nyquist period during the first minute of a run (upstream commit eb1c5ee, in the `gpu-engine` base, not in v0.37.0); v0.37.0 checks it, unless run with `--exact-endcriteria`, only at its progress report every 4 s of wall time, so its stop step depends on the machine's speed. A run that ends on the energy criterion can therefore stop at a different step than stock v0.37.0. For the same number of time steps the results are identical, which is why every comparison here uses a fixed step count. In the run-mode matrix of the cg29 check this was the only difference: on a 0.7 s model that ends on the energy criterion, stock ran to its step limit of 1 496 steps and every cg29 configuration stopped at step 255, with its 64 probe lines equal to the first 64 of stock's 375 (`results/cpu_items_cg29_2026-11-05.txt`, section 3). Up to cg29 the base's schedule also ran in a run without an end criteria, one thread summing both fields every Nyquist period while the engine waited; since cg31 such a run makes no energy estimates, as in v0.37.0 (patch 0028, Section 9).
- **Every number has a load figure next to it.** The machine was shared with other sessions; a speed

quoted without its `others` percentage was not trusted, and comparisons across snapshots were re-run when loads differed. Up to the cg23 tables of 2026-11-04 the figure was a one-second sample taken just before each run, not during it, and it failed once: the snapshot-A row at 8 threads in `results/patch_s5_n1200_AB.txt` reads `others -15493379%`, so the load during that run is unknown. From the cg27 tables of 2026-11-04 on (`bench/usage.py`) the lines also give the other processes' CPU and GPU use during the run, and from cg24 on most speed comparisons are paired and interleaved (the harness, above); the unpaired ones say so where they are quoted.

- **Nothing is accepted on speed alone.** Several items can be switched off by an environment variable that restores the previous path (listed in `BUILD.md`): `OPENEMS_FUSED=0` (fused sweep), `OPENEMS_BALANCE=0` (slab balancing), `OPENEMS_QOS=0` (thread QoS), `OPENEMS_NO_STAGGER` (allocator), `OPENEMS_NO_PML_TABLE` (PML tables), `OPENEMS_NF2FF_LEGACY` (far-field product) and `OPENEMS_NO_PACKED_COEFF`, which turns off three items at once (Section 4 says which). cg29 adds `OPENEMS_OP_INDEX_PER_CELL` (the former operator index per cell), `OPENEMS_ADE_MERGED=0` (the ADE orders of a conducting sheet one after the other), `OPENEMS_PML_YVEC=0` (the scalar UPML rows one at a time), `OPENEMS_COEF` (`packed` or `geom`: the coefficient layout), `OPENEMS_FLAGS=0` (the two barriers instead of the neighbour flags) and `OPENEMS_FLAGS_SPIN_US` (the flag spin at any thread count; Section 4.14). The `OPENEMS_NO_*`, `OPENEMS_NF2FF_LEGACY`, `OPENEMS_OP_INDEX_PER_CELL` and `OPENEMS_FTZ` switches act when set to any value other than empty or 0; `OPENEMS_FTZ` and the probe end criterion are off unless set. The other items (the spin barriers and the hook plan, work stealing and the thread search, the Mur and UPML fast paths, the dispersive-material line hooks, the dump gather, the setup changes and the CSXCAD changes, from cg29 the threaded UPML coefficient setup and the line hooks of the steady-state detection and the Debye materials, and from cg31 those of the cylinder extension) have no switch: they rest on the bit-identity checks and are bisected through the snapshots and commits. The Metal engine is opt-in (`--engine=metal-diamond`); `OPENEMS_DIAMOND_TUNE=0` turns off its run-time choice of the tiling, and `OPENEMS_DIAMOND_K`, `OPENEMS_DIAMOND_TILE` and `OPENEMS_DIAMOND_COEF` fix the tiling and the coefficient layout (Section 6); its changes of cg31 have eight switches of their own (Section 9). The tiled sweep of patch 0040 is opt-in too: it runs only when `OPENEMS_TBLOCK` is set (Section 9.6). Patches 0041 and 0042 add two switches: `OPENEMS_TUNE_START=p` restores the start of the thread search of cg31, and `OPENEMS_ADE_PACKED=0` turns the packed dispersive hooks off (Section 10.1); with 43 switches in all in the command line's list.
- **What "bit-identical" rests on.** Three rewrites depend on the compiler fusing a multiply-add the same way in the old and the new expression; this was verified by comparison on every model with Apple clang builds, not forced in the source (Section 4.11). In the standalone kernel the rewritten inner loop matches the stock loop bit for bit under Apple clang but not under GCC 16 (`results/kernel_gcc_vs_clang_2026-11-03.txt`). The Metal kernels write `fma()` where Apple clang fuses the CPU expressions, so the GPU engine's identity rests on the same compiler's choices.
- **The policy has a known hole:** a bit-identical test only covers the features the test models exercise; the Debye material of Section 4.13 was not in them and its bug surfaced only in the pre-publication review.

3.4 Baseline: stock openEMS v0.37.0

Table 2 is the first measurement. Class and engine names in this paper are those of v0.37.0 and of the fork's base, upstream's `gpu-engine` branch at 7bb294f. Upstream master has since reorganised them (commit c0816a2, 2026-11-02): `Operator_SSE_Compressed` became `Coeffs_SSE_Compressed`, `Engine_SSE_Compressed` and `Engine_Multithread` gave way to `Engine_sse` running on an `Engine_ThreadPool`, and `--engine=sse`, `sse-compressed` and `multithreaded` now all select that engine. Three observations shaped the work:

1. The multithreaded engine's best thread count is 8 on the big models but 2–4 on the small ones, and the small ones get *slower* with every thread past that. The per-time-step synchronisation is the reason. Stock runs $2 + 6N$ barriers per step for N extensions: 50 on the patch model (one excitation, six Mur faces, and a lumped-RLC extension without active cells that the port's feed resistor creates) and 38 on the notch. The initial review (below; it counted 44 for the patch, missing that extension) measured 42 such barriers at 0.21 ms per step on 4 threads and 0.66 ms on 8 on the tutorial grid, where a step is 0.09 ms of work, and at 0.36 and 1.0 ms on the 7.9-million-cell grid.
2. The default `numThreads=0` starts with one thread and adds one every ∼4 s of run time, so a 10–20 s run spends most of its life under-threaded. The "fastest" column is 41 % and 37 % slower than the best fixed count on the two large models (15.3 s and 19.6 s runs), 22 % on the notch tutorial (17.5 s) and 8 % on the patch tutorial (4.3 s).
3. PML slabs at both x ends load the first and last thread's slab; the notch x3 model is flat at 671–693 MC/s for 3–6 threads and only reaches 845 at 8.

model	basic	sse	sse-compressed	multithreaded best (thr)	multithreaded 4	default (“fastest”)
patch x5 (7.86 M)	157	265	394	1041 (8)	1006	615
notch x3 (5.25 M)	147	204	290	845 (8)	693	534
patch tutorial (0.10 M)	162	195	255	277 (2)	205	254
notch tutorial (0.27 M)	51	117	133	318 (4)	318	247

Table 2: Stock v0.37.0, MC/s. Every engine's port probes and S-parameters were bit-identical to the basic engine's (patch x5: S_{11} only; notch x3: port 1 only, since the pulse has not reached port 2 by the end of the run).

An initial read-only review of the source (kept as `results/fable_optimization_review.md`) ranked the candidates by expected gain per effort, using two micro-benchmarks it wrote for the purpose (`barrier_bench.cpp` and `kernel_bench.cpp`, named there; the later standalone kernels are `scripts/kernel/kb_idx.cpp`, `kb_tb.cpp` and `kb_tile.cpp`): a barrier diet, the inner loop, dynamic work distribution, the boundary paths, and, further out, a fused E/H sweep. That order was followed, and the estimates held up within their stated error except that the fused sweep mattered more and an unfused GPU engine less than predicted. The review's remark that a Metal engine would pay only with fp16 storage or with temporal blocking was borne out later by the diamond-blocked engine of Section 6, which runs the two large models faster than the CPU engine.

4 The engine, change by change

Each item names what stock did, what the fork does, the knob that restores the old path where there is one, and the measured effect at the time it was added (the models and thread counts differ because the suite grew; Figure 6 has the common view; the SMA launch is on its 1.071 M-cell mesh up to cg18 and on its 0.871 M-cell mesh in the items of cg29, Table 1). All items are bit-identical to stock on the six models, the cylindrical multigrid example and the special cases of Section 3.3 unless stated, for clang builds (Section 4.11). The on/off knobs named below act when set to any value other than empty or `0` (patch 0006; `OPENEMS_FUSED=0`, `OPENEMS_BALANCE=0` and `OPENEMS_QOS` keep their meaning, the switches that cg29 adds with `=0` act, like `OPENEMS_FUSED=0`, on any non-empty value that `atoi` reads as 0 (so `off` too), and an empty value of any of the fork's variables counts as unset). Class and file names are those of v0.37.0 and of the gpu-engine base 7bb294f that the patches apply to (Section 3.4); upstream master reorganised these classes on 2026-11-02, and the patches do not apply to it as they are. A companion document in the repository (Section 14), `paper/code_notes.md`, written from the diffs by an AI review agent, gives file names, line counts and code fragments for every item of patches 0001–0006 (cg18) and, in sections added on 2026-11-11 and 2026-11-13, for patches 0040 to 0043; it does not cover patches 0007–0039, whose commit messages describe them: the Metal diamond engine of patches 0007–0016 is in Section 6, the CPU items of patches 0017–0027 (cg29) in Section 4.14, patches 0028–0039 (cg31) in Section 9, which also has the opt-in tiled sweep of patch 0040 (Section 9.6); patches 0041–0043 (the thread-search start and the packed dispersive hooks, the fork's head) are in Section 10.1. This section keeps to the mechanisms.

Memory layout, for reference. The SSE engines store each field as a 4-D array of `f4vector`s in I-J-K-N order (component stride 1, z-vector stride 3), so one (x, y, z_v) holds the three components of four cells; cell z sits in lane z/n_v of z-vector z mod n_v with $n_v = \lceil N_z/4 \rceil$, and the z extent is padded to $4n_v$. A cell costs 12 B per field. The compressed operator index needs one 32-bit entry per z-vector, 1 B per cell. Up to cg28 it was allocated with one entry per cell ($N_x N_y N_z$), of which only the first n_v of each z-row were used, so with 128-B cache lines the engine streamed up to 4 B of index per cell, all of it when $N_z \leq 42$; cg29 allocates it per z-vector (Section 4.14). The byte counts below give the compact 1 B and, where it mattered up to cg28, the index as allocated. The Metal diamond engine keeps the same `f4vector` fields in N-I-J-K order (component outermost) in memory shared with the GPU, so probes, dumps and the CPU extensions read the same arrays (Section 6).

4.1 Barriers and the hook plan (snapshot A)

Stock `Engine_Multithread` separates the pieces of a time step with a `boost::barrier` (mutex and condition variable): voltage update, barrier, then *for every extension* each of the six hooks followed by a barrier, whether or not the extension implements it; $2 + 6N$ barriers per step for N extensions, 50 on the patch model (one excitation, six Mur faces, and the lumped-RLC extension of the port's feed resistor, which stock keeps with no active cell because its `BuildExtension` returns true), each a kernel round trip. Thread 0 also incremented the time-step counter after the last hook with no barrier before the other threads' next step.

The fork replaces the barrier with a spin barrier (`tools/spin_barrier.h`): an atomic arrival counter and a generation number; the last arrival bumps the generation (and, when asked, the time-step counter, which removes the race), waiters spin on the generation with a CPU pause for up to 50 μs (`OPENEMS_BARRIER_SPIN_US`) and only then park on a condition variable. Every extension now declares which hooks it uses (`GetUsedHooks()`, a bit mask), and a *hook plan* is built per engine: extensions that do not declare a hook group are not called at all; pre- and post-hooks of all declaring extensions share one barrier at the end of their group (they only read engine fields, by contract; the UPML, whose pre/post hooks swap field and flux, declares `PREPOST_WRITES_FIELDS` and gets its barriers back); the apply hooks keep one barrier per extension because they write fields in priority order. Extensions that declare nothing keep the stock behaviour. Unfused, the count becomes 2 + [any pre-V] + [any post-V] + #apply-V + [any pre-I] + [any post-I] + #apply-I: 12 on the patch model with the current declarations (one apply barrier each for the lumped-RLC extension, the six Mur faces and the excitation, the shared pre- and post-voltage barriers and the two update barriers); 18 at snapshot A, where the lumped-RLC extension still declared nothing, and 13 until patch 0005, where the excitation also declared the apply-current group. With the fused sweep below it is exactly 2, and with the neighbour flags of cg29, when every extension allows them, none: the threads then meet only at the start and the end of a run of steps (Section 4.14). The plan is rebuilt whenever the extension list changes; the first version was built once, before the cylindrical multigrid registered its child engines, and was wrong on that example only, which is why the check suite runs it. Patch tutorial 277 to 502 MC/s, notch tutorial 318 to 573, patch x5 1 041 to 1 151 (the load sampler failed on that 8-thread run; the 10-thread run, at 56 % other load, gave 1 121), notch x3 845 to 902.

4.2 The inner loop (AB)

`Engine_SSE_Compressed::UpdateVoltages` located every operand through the array library's subscript operators and, because Apple clang gives every store through the `f4vector` union the alias type of `char` (the TBAA type 'omnipotent char', which may alias any object), reloaded strides and coefficient pointers after every store: 94 instructions per vector update (the three components of one z-vector), 44 of them loads, of which 21 load vectors (95 and 44 for the currents). `UpdateVoltages_IJKN` / `UpdateCurrents_IJKN` walk the I-J-K-N layout with all invariants in local variables and `__restrict` pointers (the three components of a z-vector at `vz[0..2]`, the next z-vector at `+3`, the x and y neighbours at constant offsets, the z wrap-around with a lane shift) and store each component once: 44 instructions at snapshot AB (from the project log); 41 with 16 loads in cg18, which also reads the packed tables of cg13 (42 and 15 for the currents). The stock and cg18 counts, taken in the copy of each loop that runs, are in `results/kernel_loop_instructions_2026-11-03.txt`. Per component stock computes $v \leftarrow v \cdot vv$ then $v \leftarrow v + vi \cdot t$ with $t = (c_1 - c_2) - c_3 + c_4$; the new code computes $a = v \cdot k_0$, $v = a + k_1 t$ with the identical t, and with clang's default contraction both forms become a rounded product followed by one fused multiply-add, so the bit patterns agree. This identity was verified by comparison (and against the Metal mirror of the same loop), not forced in the source; GCC contracts the two forms differently (Section 4.11). Traffic is unchanged by this item (37 B per cell per update with a compact index, 40 B with the index as allocated). Patch tutorial 502 to 579, notch tutorial 573 to 668, patch x5 1 151 to 1 258. The stock baseline is the default clang build. Upstream's FAQ [19] asks macOS users to build openEMS with GCC because clang's code for the SSE engine is slow. In the standalone kernel (`scripts/kernel/kb_idx.cpp`) GCC 16 does not reload the strides and pointers: its stock loop has 80–82 instructions and 29 loads, 7 of them reloads of spilled registers (same instruction-count file). At one thread, on three grids outside the valley of Section 4.6 (`results/kernel_gcc_vs_clang_2026-11-03.txt`), it runs the stock loop 1.16–1.39× faster than Apple clang, while the rewritten loop runs at about the same speed with both: the rewrite's one-thread gain in the kernel is 1.26–1.47× with both loops built by GCC and 1.74–1.80× with both built by clang. A GCC build of openEMS itself was not made, so whole runs with GCC were not measured, and the factors of this paper are against stock built with Apple clang, the default compiler on macOS.

4.3 Work distribution and the automatic thread count (C, refined up to cg8)

Stock gives every thread a fixed x-slab of equal width, and `numThreads=0` (the default) starts with one thread and adds one at every progress report, about every 4 s, until the reported speed drops: a 1→N ramp. In Table 2 the default is 37–41 % slower than the best fixed count on the two large models (15–20 s runs), 22 % on the 17.5 s notch tutorial run and 8 % on the 4 s patch tutorial. On a 4P+6E machine equal slabs are wrong twice over: an E core takes longer, and the first and last slabs hold the PML.

The fork does three things. In the unfused path, threads take x-lines from per-thread atomic counters and steal from the other threads' ranges when their own is done (the order of lines within one update is irrelevant: the voltage update reads only currents and vice versa, and each line is written by exactly one thread). In the fused path (next item) threads own contiguous slabs and `BalanceSlabs()` moves the slab boundaries after

every `IterateTS` call once 20 ms of sweeps have accumulated, to equal cumulative cost per thread from the measured busy time per line (`OPENEMS_BALANCE=0` turns it off). And the automatic thread count is chosen by measurement on the running simulation: starting, up to cg31 and with `OPENEMS_TUNE_START=p`, at the P-core count (`hw.perflevel0.physicalcpu` on Apple, half the available hardware threads elsewhere; since patch 0041 on Apple at every core whose performance level is not named "Efficiency", going down by 2 while each step is the fastest, Section 10.1), `IterateTS` runs in pieces of an eighth of a 150 ms window (`OPENEMS_TUNE_WINDOW_MS`) with balancing between pieces; each candidate count is timed for one window after a settling time of 0.4 windows; the search goes P, P+2, P+4, … (stopping once a count falls below 85 % of the best), then P−2, then ±1 around the best, and prints the result. Every candidate is clamped to the number of x-lines (patch 0005 clamped only the larger ones; patch 0006 also clamps P−2 and the start). The tuner and its 150 ms windows date from C; slab balancing followed right after ABCDEG and is in every cg snapshot; the window/8 pieces and the search order came with cg7 and cg8. Speed is not monotonic in the thread count (notch x3 at cg8: 4:1791, 5:1564, 6:1800, 7:1919, 8:2022, 9:1901 MC/s; from the project log, not in `results/`), hence the coarse step and the refinement. The search needs about 1.5 s and costs short benches 15–25 % (both from the project log); at cg14 it picked 4 for models up to a few hundred thousand cells and 7–8 above a million (Table 9). With the neighbour flags (a development build of cg29) it picked 9–10 threads in most runs on the GCPW line and the SMA launch at their real run lengths. A cache of the chosen count per model (second review, item 3) and a 50 ms window were measured for cg29 and gained nothing at those run lengths (Section 12). The cylindrical multigrid engine keeps the stock ramp, with its inner engine pinned to one thread.

4.4 Mur and UPML on vectors (D; Mur line hooks rebuilt in cg4)

The Mur absorbing boundary [4] and the uniaxial PML [5] extensions were written against the scalar field interface: per face cell, `GetVolt`/`SetVolt` computes a 4-D subscript with a division and a modulo for every access. The Mur fast path precomputes the float index of every face cell once and checks that the offsets to the two components and to the shifted line are the same constants for all cells (otherwise the template path stays); its three hooks are then the template's three expressions on the same operands. The UPML vector path, for PML boxes that span the whole z range (only then does every `f4vector` belong entirely to the PML), copies the six per-cell coefficient arrays and the two flux arrays into vector order and runs the update per (x, y) row of $3n_v$ vectors with a lane mask for the padding, splitting rows rather than x-lines over threads (a slab of 8 lines would otherwise occupy at most 8 threads). The vector form $p = -(c_{\mathrm{fo}} \cdot F)$, $f = c \cdot E + p$ pins the contraction to what clang emitted for the scalar line (GCC fuses the other product, Section 4.11). Mur's *line* hooks for the fused sweep (E) were then rebuilt in cg4: the face cells of each x-line in memory order, the four lanes of one z-vector of an x- or y-face (cells $z, z + n_v, z + 2n_v, z + 3n_v$) as one vector with lane-wise coefficients, the rest scalar. Patch tutorial 1 519 to 1 810, patch x5 1 454 to 1 602 (cg2 to cg6, 4 threads).

4.5 The fused E/H sweep (E)

A time step is one pass over the voltages followed by one pass over the currents; on a model larger than the cache every field array is streamed from DRAM twice per step, 74 B per cell (12 B read and 12 B written for the updated field, 12 B read for the other, 1 B of compact index, per update; 80 B with the index as allocated). The fork fuses the two passes. $V(x)$ reads $I(x)$ and $I(x-1)$; $I(x)$ reads $V(x)$ and $V(x+1)$. Once the voltages and voltage hooks of x-line block $[x_0, x_1)$ are final, the currents of lines $x_0 - 1 \ldots x_1 - 2$ have all their operands final and their old values are needed by no voltage of this step, so each thread runs, per block of its slab:

```
RunLineHooks(PRE_VOLT, x0, x1);  UpdateVoltages(x0, x1-x0);
RunLineHooks(POST_VOLT, x0, x1); RunLineHooks(APPLY_VOLT, x0, x1);
FusedCurrents(x0==start ? x0 : x0-1, x1-1);   // pre-I hooks, I, post-I, apply-I of the line behind
...                                            // after the slab:
barrier();  if (start>0) FusedCurrents(start-1, start);   // the line before the slab, now that the neighbour
    's V is final
barrier(numTS++);
```

The currents of a line are computed while its voltages and the neighbouring currents are still in L1/L2, so both fields stream once: 49 B per cell (52 B with the index as allocated up to cg28), and two barriers per step (neighbour flags since cg29, Section 4.14). For this to be legal every extension that touches fields during the step needs per-line hooks (`*Lines(x0,x1)`) besides the whole-grid ones, touching only its own lines (an x-face Mur line that reads the next line declares it as *coupled*, and the two are always in one block); the excitation and the lumped RLC sort their entries by x-line once and run their unchanged bodies on the entries of a block (up to cg28 the RLC's per-step container rotation was done by whichever thread arrived first, under a time-step stamp; cg29 takes the three slots from the time step). Mur, UPML, excitation, RLC, since cg12

the Drude/Lorentz extension, which also carries the conducting sheets, and since cg29 the Debye extension and the steady-state detection have line hooks, and since cg31 the cylinder extension of cylindrical meshes closed in alpha (Section 9.5). Any extension without them (Debye materials from patch 0005 to cg28, the cylinder extension up to cg29, the cylindrical multigrid, TFSF, the steady-state detection up to cg28, the `Absorbing_BC` extension) puts the whole run on the unfused plan, and a verbose message names it (once per level engine of a cylindrical multigrid since patch 0006). The fused sweep also needs the packed coefficient tables of cg13 (or, since cg29, the geometry tables of Section 4.14); when their allocation fails, the engine says that the fused sweep is off because there are no packed tables (patch 0006). The fused sweep never calls the current-line hooks for the last x-line (documented in `engine_extension.h`; no engine updates currents there). `OPENEMS_FUSED=0` restores two passes. Notch x3 1 153 to 1 916 MC/s in the same build, patch x5 1 348 to 1 559 (from the project log; not in `results/`); tutorials unchanged (they fit in the cache). A standalone kernel of the same shape, but with a compact 1-B index (`scripts/kernel/kb_idx.cpp`), reaches 1 932 MC/s on 4 threads (95 GB/s at 49 B per cell, read:write 1:1), and one independent sub-grid per thread (`kb_tb.cpp`) 2 273 on 6: those are the ceilings (from the project log; the kernel output is not in `results/`) of a sweep that reads the fields from memory once per time step. The opt-in tiled sweep of patch 0040 reads a tile once for several steps, so they do not bound it: on 2026-11-11, under macOS 27.2.1 and not paired with these kernel runs, it ran patch x5 at 2 305 MC/s at 4 threads, and it was behind the fused sweep's best setting on four of six models (Section 9.6). The fused sweep's remaining gap on patch x5 at 8 threads (20–25 %, project log) is P/E imbalance at the two barriers while macOS migrates threads and other sessions load the machine; cg29 measured the wait (at 8 threads 24–27 % of the time at the two barriers, `OPENEMS_FUSED_STAT=1`) and replaces the barriers by neighbour flags (Section 4.14). Wider blocks (`OPENEMS_FUSED_BLOCK`) lose the L2 reuse and were measured slower (Section 12).

4.6 The allocation-placement valley and staggered allocations (cg10)

At 4 threads the two real boards had gained only 1.4–1.7× from everything above (SMA 320 to 436 MC/s, GCPW 302 to 505 with the stagger below switched off), while the tutorials had gained 4.5–6.5× at their best thread counts. A one-thread sweep of a PEC box with a dielectric slab over its size (Figure 1, `results/cache_valley_2026-11-01.txt`) shows why: without the stagger, boxes of 200 000 to 338 000 cells ran at 246.8–249.5 MC/s, about 3.6× slower than those of 162 000 and 392 000 cells (891.7 and 890.7). The valley holds the models whose field arrays are 2–4 MiB each, about 0.175–0.35 million cells at 12 B per cell and array (N_z a multiple of 4; 4.2–8.4 MB of fields): on cg18 without the stagger, 176 720 to 338 000 cells are slow, 172 980 and 348 480 fast (`results/cache_valley_placement_2026-11-03.txt`, section 3). Stock v0.37.0 has the same valley, 2.1–2.2× deep, at about half the fork's speed outside it (`results/cache_valley_stock_2026-11-03.txt`).

The trigger is where the allocator puts the two arrays. macOS 27.0.1's libmalloc places two consecutive `posix_memalign` blocks of equal size 4 MiB + 16 KiB apart at every tested size from just above 2 MiB to 3 000 000 B, exactly 4 MiB apart at 4 181 760 B and 4 MiB, 8 MiB + 16 KiB apart from just above 4 MiB to 8 000 000 B, and the second about 6 MiB below the first from 1.94 to 2 MiB (section 1 of the placement file). The slow sizes fall in the range placed 4 MiB + 16 KiB apart (the test has no size between 3 000 000 and 4 181 760 B, where the arrays of the 338 000-cell box lie); 348 480 cells (4 181 760 B per array, exactly 4 MiB apart) are already fast. All these distances are multiples of 16 KiB, the fast ones too, so 16 KiB set aliasing alone does not explain the valley; the hardware mechanism was not identified. In the standalone kernel at $110 \times 110 \times 20$ (section 2), moving the current array by 512 B or more, 16 and 32 KiB included, removes the slowdown (stock loop 210–211 to 455–464 MC/s, rewritten loop 233–234 to 779–819), and 256 B removes part of it. The project log of 2026-10-29 had 16 and 32 KB offsets failing, which was then read as L1 set aliasing; at this shape that does not reproduce.

The fix is in the array allocator (`tools/arraylib/impl/allocator.h`, not platform-specific): every allocation of 256 KiB or more with an alignment of at most 256 B is made $256 + (2560\,k \bmod 16128)$ bytes larger and the array starts that far into it, $k = 0, 1, \ldots$ counting such allocations in the process (63 distinct multiples of 256 B, so any alignment up to 256 B is kept); the base pointer is stored in the word before the array, where the free path finds it. `OPENEMS_NO_STAGGER=1` turns the stagger off. Outside the valley it changes nothing measurable (cg18, finer sweep: 867.7–899.7 MC/s with it at every size, 864.9–890.5 without it outside the valley). It is applied on every platform, but the valley was seen only with macOS's allocator: under 27.0.1 on the M5 and, on the GCPW line, under 27.2.1 on the M5 Pro of Section 10, where the stagger removes it too; other allocators place large blocks differently, so the size window, and whether there is one, depends on the platform. The allocator is a header, so this is an ABI change for code compiled against the old one (Section 4.13). GCPW at 4 threads 505 (stagger off) to 881 MC/s in one session, 868 in another. The same snapshot fixed a crash of `--disable-dumps` at the end of runs with frequency-domain dumps (post-processing of fields that were never allocated).

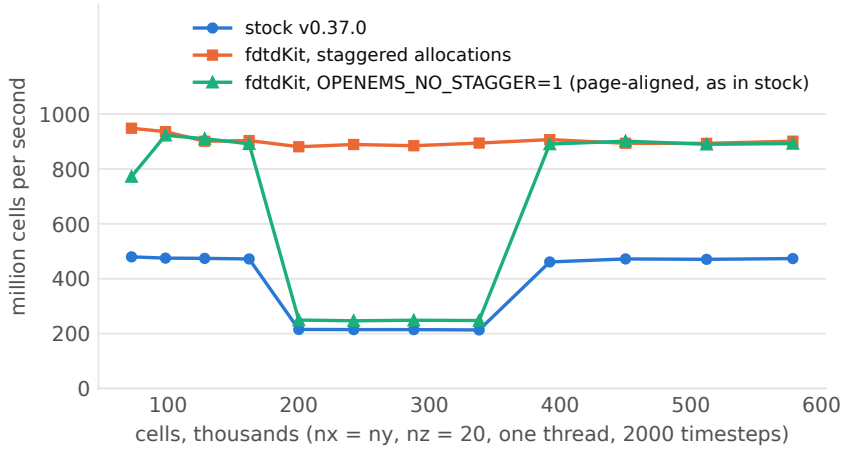


Figure 1: The allocation-placement valley on one thread: PEC box with $N_z = 20$, 2 000 steps. fdtdKit: the installed cg16 build on 2026-11-01, with staggered allocations and with `OPENEMS_NO_STAGGER=1`, which leaves the field arrays page-aligned as in stock (`results/cache_valley_2026-11-01.txt`; the low no-stagger point at 72 000 cells is one run that was not re-measured). Stock v0.37.0: measured on 2026-11-03 (`results/cache_valley_stock_2026-11-03.txt`).

4.7 PML slabs that do not span z: a scalar-SSE path and compressed tables (cg11, cg14)

The SMA launch did not gain from the stagger. Its PML is a z-only slab at the top (13 % of the cells) and the UPML vector path requires a full-z box, so those cells ran the original template: 202 MC/s single thread, 654 with the PML swapped for Mur. The fork adds a scalar-SSE path for any PML box that is not a full-z slab: per local z the float offset inside an (x, y) row of vectors is tabulated once ($12(z \bmod n_v) + z/n_v$), the hooks iterate rows, components and z with contiguous flux and coefficient rows, and the expression is literally the template's ($f = vv \cdot V - vv_{\text{fo}} \cdot \text{flux}$, then the swap), so the compiler contracts it the same way; each cell-component touches only its own V and flux, so the loop order does not change values. With line hooks the fused sweep now covers the slab. SMA single thread 202 to 405, 4 threads 436 to 737, 8 threads 951 (cg29 runs this path four y-rows at a time, Section 4.14). Of the test models only the SMA launch, with its top z slab, takes this path. A notch tutorial run with `PML_8` requested on all six faces (`results/notch_s1_n8000_pml6.txt`) does not: on its 14 z lines openEMS resets the z PML to PEC, and the four x and y slabs span the whole z range. The 69 500 distinct operators of the SMA model cost only about 11 % (standalone kernel with 70 000 operators; from the project log). cg14 then lets the hooks read a table of distinct coefficient sets (24 B scalar or 96 B vector entries, a few hundred of them: the PML profile times the materials) through a 4-B index, instead of the six per-cell coefficient arrays (72 B per cell). The index is per cell-component on the scalar-SSE path and per z-vector component (four cells) on the vector path. The operator extension keeps its arrays, as in stock, and the vector path frees its own vector copies of them, so the gain is in traffic, not in memory: with the index (and, on the vector path, the vector copies of the flux) a PML cell holds more memory than in stock. `OPENEMS_NO_PML_TABLE=1` keeps the hooks on the per-cell arrays. Bandwidth-side only: SMA 4 threads 1 137 to 1 241, notch x3 8 threads 2 022 to 2 099; single thread unchanged.

4.8 Line hooks for the dispersive extension (cg12)

The optimisation review of the branch found the real GCPW problem: every lossy board uses `AddConductingSheet`, whose engine extension (`Engine_Ext_LorentzMaterial` on `Engine_Ext_Dispersive`) implemented only the non-threaded hooks, so the multithreaded engine ran them on thread 0 through `GetVolt`/`SetVolt`, declared nothing (19 barriers per step on that model at cg11) and kept the whole run on the unfused plan. The fork gives the dispersive extensions line hooks: the auxiliary-differential-equation positions of every order are stable-sorted by x, the float index of each component is precomputed, the per-position arithmetic is the template's (apply: $v \leftarrow v - \text{ade}$; Lorentz pre-hooks: the template's three statements), and the extension declares the pre/apply voltage and current groups only for the orders that have such terms. Positions on different x cannot share a cell, and positions on the same x keep their original order, so even a duplicated cell would be updated in the stock order. Verified on the GCPW line and on a Drude/Lorentz box exercising both branches (plasma only, plasma plus pole; template versus line hooks; raw fields; 1 and 4

threads). GCPW 4 threads 868 to 1 435–1 661 MC/s; cg29 runs the two orders of a conducting sheet back to back, four positions at a time (Section 4.14). The threaded whole-grid hooks of these extensions are still not parallelised; they matter only when something else forces the unfused plan. The same snapshot hardened two things the correctness review flagged: the per-thread material-lookup cache in the operator setup keys on the first and last primitive as well as the coordinate and the vector address, and `SetupCSXGrid` clears the tabulated mesh deltas.

4.9 Packed coefficient tables (cg13)

The compressed operator keeps its coefficients in twelve structure-of-arrays tables (four coefficients × three components, 16 B entries); a voltage update needs six of them for one operator index, up to six cache lines, and with the 69 500 operators of the SMA model on its 1.071 M-cell mesh (13.3 MB of tables; operator count from the project log) the access is sparse. `Engine_SSE_Compressed::Init` copies them into two per-operator arrays of six vectors in the order the update reads them (`struct PackedCoeff {f4vector c[6];}`, 96 B), and the IJKN loops read those: one or two lines per operator, +13 MB for the SMA model. Plain copies, same floats. `OPENEMS_NO_PACKED_COEFF=1` skips the build, and because the IJKN loops require the packed table, that knob also reverts to the generic loops and the unfused sweep: it is an "old engine" switch, not an isolated A/B. The same happens when the tables cannot be allocated: both are freed and the engine runs the generic loops, unfused. GCPW 1 thread 507 to 532; SMA 1 017 to 1 137 at 4 threads. At 4 threads the GCPW runs of cg12 to cg14 (1 608–1 745) lie within the run-to-run spread, so no gain is credited there. Since cg29 a model whose packed tables would exceed 4 MB gets smaller geometry tables instead (of the six, only the SMA launch; Section 4.14).

4.10 Thread quality of service (cg15)

On macOS the engine threads now run at `QOS_CLASS_USER_INITIATED` (`OPENEMS_QOS=0` off, 2 = user-interactive, same speed), set right after the subnormal setup in each worker. Stock workers ran at the default class, which lets the scheduler park them on E cores or behind foreground applications. Scheduling only; probes bit-identical. Measured in a separate session (from the project log; not in `results/`): SMA 8 threads 1 304–1 385 to 1 409, notch x3 8 threads 2 039 to 2 102, 10 threads 1 657 to 1 856; 4 threads unchanged. The cg14 check runs in `results/`, without QoS, already reach 1 385 (SMA, 8 threads) and 2 099 / 2 038 (notch x3, 8 / 10 threads), so the gain is within the run-to-run spread of a shared machine. The cg17 runs in `results/` were made on a loaded machine and are not comparable. A per-thread split of the classes between P and E cores (second review, item 4) was measured for cg29 and dropped; every thread keeps the user-initiated class (Section 12).

4.11 On the bit-identity argument

Some items rely on the compiler contracting the rewritten expression the way it contracted the stock expression: the IJKN loop and the UPML vector rows (a rounded product plus one fused multiply-add; the y-row groups of cg29's scalar UPML path use the vector rows' form), the Mur fast path (one fused multiply-subtract or multiply-add per update) and cg29's merged sheet ADE (the template's statements on four lanes). This was verified by comparison with stock on every model and, for the cg29 items, on boxes against the template hooks; the IJKN loop was also checked against the Swift/Metal prototype, which calls `fma()` explicitly there and has no Mur or UPML. The Metal diamond engine (Section 6) runs Mur, UPML, lumped RLC and the dispersive extensions on the GPU with `fma()` where clang fuses and Metal's fast math off, and it is bit-identical to the multithreaded engine run with `OPENEMS_FTZ=1` on the six models and in the 72 comparisons of `scripts/checks/diamond_check.sh` (cg28 and cg29), so these paths now also agree with an independent implementation. The contraction is not forced in the C++ source (no `fma()` calls, no `#pragma STDC FP_CONTRACT`), and it holds for clang, whose default contracts within one expression. GCC contracts across statements by default (`-ffp-contract=fast`): on aarch64, and on x86-64 when FMA is enabled, it fuses the other product than stock in the IJKN loop and the UPML vector rows. For the inner loop the effect was measured in the standalone kernel (`results/kernel_gcc_vs_clang_2026-11-03.txt`, three grids, five rounds): the stock loop gives the same bits under Apple clang and GCC 16, and the clang-built rewrite equals it bit for bit, while the GCC-built rewrite differs from the GCC-built stock loop on every grid and round, because the two forms then round a different product. A GCC build of openEMS was not made, so the effect on whole runs was not measured; such a build would be correct but not bit-identical to stock. A bit comparison under GCC needs both builds made with `-ffp-contract=off` (upstream's Debug build sets it). A reader porting the fork should run `check.sh` once; it is macOS-only, since it selects the libraries through `DYLD_LIBRARY_PATH`. Making the contraction explicit in both the stock-equivalent and the new paths is a reasonable follow-up.

4.12 What a cell costs now

On a model larger than the cache a time step streams the six field arrays once (24 B read, 24 B written), reads the operator index (1 B per cell: one 32-bit entry per z-vector since cg29, up to 4 B as allocated before), and takes the coefficient tables, the PML tables and the neighbours' lines from L2: 49 B per cell of DRAM traffic (up to cg28, 52 B when $N_z \leq 42$). The exception is a graded mesh with many distinct operators: the SMA launch on its 0.871 M-cell mesh has 64 410 of them and 12.4 MB of packed tables, which stream from memory next to the fields; cg29's geometry tables take 100 instead of 192 B per operator (Section 4.14). On models that fit the cache the per-step cost is the synchronisation (two barriers up to cg28, the neighbour flags since), the hooks and the dumps, which is why the tutorials top out at about 1 900 MC/s rather than higher.

4.13 Review fixes (patches 0005 and 0006)

A further AI-assisted read-only correctness review of the branch (Section 15), run before publication, found one wrong-result bug; patch 0005 (snapshot cg17) fixes it and six smaller things; the four earlier patches are unchanged. A final review led to patch 0006 (below). `Engine_Ext_DebyeMaterial` implements the pre- and post-voltage hooks but inherited `GetUsedHooks()` from `Engine_Ext_Dispersive`, which declares only the apply group and line hooks, so the hook plan never called them and the fused sweep accepted the extension: the multithreaded engine ran a Debye cell as its base material while the single-threaded engines, which call every hook, were right. No model of the suite has a Debye material (its dispersive checks used Drude/Lorentz and conducting sheets), and bit identity covers only what the models exercise. The fix declares the two hooks (no line hooks then: the fused sweep was off with Debye materials until cg29 gave them line hooks, Section 4.14), and a structural guard, `OPENEMS_HOOKS_DECLARED_BY` at the top of every `GetUsedHooks()` override, hands a subclass that does not override it the stock every-hook set, so a declaration cannot be inherited again. `scripts/checks/debye_check.sh` ($60 \times 60 \times 20$ Debye box, 600 steps, final fields compared bit for bit): before the fix the largest voltage difference was 37 % of the peak voltage and 187 108 of 216 000 voltage values differed; after it the fields are bit-identical and the single-threaded result is unchanged (rerun with 2 threads during the final review; the output is not in `results/`).

The smaller fixes of patch 0005: the probe end hold can start only once every probe has a non-zero peak (a far port the pulse had not reached counted as settled). This catches only a probe that is still exactly zero. The numerical precursor of the pulse gives a far probe tiny non-zero values some time before the pulse arrives, the longer before the farther the probe; measured against 10^{-6} of the largest peak of its kind, they count as settled, so the hold must still be longer than the longest one-way delay. The GCPW stop point is unchanged: 45 570 steps before the fix (cg16, from the project log) and on cg18 (row `G_60` of `results/rfsim_e2e_2026-11-03.txt`, its port files a byte-identical prefix of the full run's). Further: the setup's material-lookup cache also keys on a setup generation counter (no stale hits in a later setup of the same process); the excitation declares the apply-current hook only when it has current excitations (one barrier per step less, unfused); the thread tuner clamps its larger candidates to the number of x-lines; the separable nf2ff path is used only with Apple's Accelerate (other builds keep the threaded legacy loop); the packed coefficient tables are skipped, with a message, when they would exceed twice the field memory or fail to allocate (the size test is removed again in patch 0006). cg17 passed a reduced check: the six models bit-identical to stock at 4, 8 and automatic threads, one run each without the quiet wait, on a loaded machine, so the speeds in `results/*_cg17.txt` are not comparable; the cylindrical multigrid example's 18 HDF5 datasets equal; the GCPW probe end at the same step with a bit-identical port-file prefix (these two from the project log).

Patch 0006 (snapshot cg18) holds the fixes of the final review. The packed-table size test of patch 0005 is removed again: it could fire only through the z padding, and then it also turned off the IJKN loops and the fused sweep, so a model with a graded material (nearly one operator per z-vector) and a z line count that is not a multiple of 4 ran on the generic loops, unfused. The tables are always built, as in cg16; only a failed allocation skips them (both tables freed), and the multithreaded engine then says that the fused sweep is off because there are no packed tables. The on/off knobs act only when set to a value other than empty or `0`. The probe end criterion counts the excitation as running until its signal length plus the largest excitation delay (the voltage and current entries of every grid, including the multigrid child grids, and the TFSF propagation delay); a mode-match probe counts with its mode amplitude only, not with its purity; a probe that has seen no signal still holds the run, but once every other probe has settled the run prints a one-time notice naming it; an invalid `OPENEMS_PROBE_END_DB` or `OPENEMS_PROBE_END_HOLD` prints a warning. The tuner clamps every candidate, including $P-2$ and the start. With `OPENEMS_FTZ` the creating thread gets its previous flush-to-zero bit back when the engine is destroyed on it. Each level engine of a cylindrical multigrid prints its fused-sweep line once (each printed it twice before); the per-block work buffer of the separable nf2ff path is bounded; the Accelerate include builds with GCC on macOS, and the spin barrier's yield is guarded for 32-bit ARM; a mesh whose node count overflows the 32-bit array sizes (above $2^{32}/3 \approx 1.43 \times 10^9$

nodes when arrays with three components over the whole mesh are needed, as with field dumps or an nf2ff box; $2^{32} \approx 4.29 \times 10^9$ otherwise) is refused with a clear error; the raw `OPENEMS_EXPORT_*` export reports failed writes; code comments are corrected; and the build reports its real git revision (the cg16 and cg17 libraries both call themselves v0.37.0-19-g0893f50).

cg18 went through the full check on a quiet machine on 2026-11-03: `check.sh` ran the six models at 1, 4, 8 and 10 threads and with the automatic choice, two runs each, and every run was bit-identical to stock in probes and S-parameters (`results/*_cg18.txt`). The other checks of that day are in `results/correctness_cg18_2026-11-03.txt`: the Debye box of `debye_check.sh` gave the same fields in the basic, sse-compressed and multithreaded (4 threads) engines, and the same as cg17; the cylindrical multigrid example (1.37 M cells, 1 500 steps, 4 threads, one frequency- and one time-domain dump) showed 0 differences against stock in 34 HDF5 datasets (the 18-dataset checks above used a different run length); the patch tutorial's far field was bit-identical to cg17's and, with `OPENEMS_NF2FF_LEGACY=1`, to stock's, and the separable path stayed within 6.8×10^{-7} of stock's pattern maximum. The validation suite (Section 14) passed 15 of 15 checks with the multithreaded engine at 4 threads and with the default engine and thread count (`results/validation_cg18_mt4.md`, `validation_cg18_fastest.md`). The probe-end runs of Table 6 were made with this build (`results/rfsim_e2e_2026-11-03.txt`); the GCPW line stops after 45 570 steps, as before, with a byte-identical port-file prefix.

Documented in `BUILD.md`, not changed: the operator setup uses every core whatever `numThreads` says; `OPENEMS_FTZ` has no effect on `--engine=basic`, which never calls the subnormal setup; with GCC's default contraction the rewritten loops round a different product than stock, so a GCC build would not be bit-identical (measured in the standalone kernel only, Section 4.11); a run that ends on the energy criterion can stop at a different step than v0.37.0, because the base checks that criterion on a different schedule (upstream eb1c5ee, Section 3.3), while bit identity holds for the same number of steps; upstream fixes made after the base are not in the fork (among them 367a63f, flush to zero only during the run and for every engine; 54a4c60, setup and run errors caught instead of terminating Python; d1fef7c, no excitation before its delay, which matters for delayed step and custom excitations; 0212258c3, correct sizes and counts for large meshes); the libraries are not ABI-compatible with v0.37.0 builds (the base's ArrayLib `m_owner` member and the staggered allocator), so after any install over another build both Python modules must be rebuilt for SAR (FDTD runs and nf2ff are not affected); the bodies of the five earlier commit messages are kept as written, and the message of 0006 lists their corrections.

4.14 The second review's CPU items and other contributors' ideas (cg29, patches 0017–0027)

Snapshot cg29 (2026-11-05; eleven commits on cg28, one per topic) holds the CPU items of the second optimisation review (Table 32) and the ideas of other contributors' openEMS work that the fork did not have yet (Section 11); cg19–cg28 had added the Metal diamond engine without changing the CPU engines (Section 6). An item was kept only when a paired interleaved A/B (`bench/abtest.py`: the configurations run in turn, round after round, and the ratio is taken within each round, so a slow phase of the shared machine hits both alike) gained clearly outside the spread of the rounds on the models it is for and cost nothing outside the noise on the others. One exception: 0021 cost upstream's ideal case 20 % at 8 threads (0.80×) and was kept because the steady-state detection on the flags (0026) recovers most of it (1.18× at 8 threads). Most A/Bs ran knob against knob in one build; 0020 and 0026, which have no knob, and 0027 were compared with the build before them, and 0021 and 0022 with `OPENEMS_FUSED=0`. The ratios are medians of the per-round ratios, with the other processes' load in cores; the SMA launch is the 0.871 M-cell model of the toolkit's default mesh. The numbers are from `results/cpu_items_cg29_2026-11-05.txt` (section 1 for the kept items, section 2 for the dropped ones, which Section 12 lists) unless a commit message is named. Table 3 has the eleven commits.

Operator index per z-vector (0017). The compressed operator allocated its index with one 32-bit entry per cell, although the operator and the engines read only the entries of the z-vectors (memory layout above); on meshes of up to 42 z lines the engines therefore streamed 4 B of index per cell instead of 1 B. The index is now allocated per z-vector: the same values in a smaller array. `OPENEMS_OP_INDEX_PER_CELL=1` keeps the old allocation. Patch x5 1.119× at 4 threads and 1.101× at 8, notch x3 1.042× and 1.052×, the other four models 1.005–1.021× (5 rounds, 0.3–1.4 cores; the commit message gives 0.4–1.7); about 3 B per cell less memory (15.8 MB on notch x3, 23.6 MB on patch x5, commit message).

Conducting-sheet orders merged (0018; review item 2). A conducting sheet is a two-order ADE on one set of positions, already in x order, and the line hooks of cg12 ran each order over all positions, one float at a time. When every order with voltage (current) terms has the same positions in x order, the pre-update

(`Engine_Ext_LorentzMaterial::PreMerged`) and the apply (`Engine_Ext_Dispersive::ApplyMerged`) now run the orders back to back per position, four positions at a time as `f4vector`s, with the field values gathered and scattered lane by lane. Every float sees the same operations in the same order: the states of different orders are independent, and the apply still subtracts order 0 before order 1. Because the apply scatters four positions at once, a position listed twice in one group of four would lose an update, so the merge also requires every position to appear once (the builders list each cell once, so the guard never fails today). The guard follows the strict-order check of upstream PR #245 by Roel Budé (unsettledroell404) [24]; the rest of that PR, the unfused hooks split over the threads, is not needed, since the line hooks already run on every thread. `OPENEMS_ADE_MERGED=0` runs one order after the other. GCPW line (6 624 sheet positions) 1.041× at 4 threads and 1.031× at 8 (8 rounds, 0.3–0.5 cores), against the review's estimate of +8–12 %; no effect on the models without sheets.

UPML scalar path, four y-rows at a time (0019; review item 1). The scalar-SSE path of cg11 (above) took 13 instructions per cell and component in its inner loop: a lane offset, a table index, two coefficients, the flux and the field value. With the coefficient table of cg14, four consecutive y-rows of an x-line now form a group whose fluxes and coefficient sets are stored per group (lane = row), so a group takes one vector operation per step of the update while the field values are gathered and scattered lane by lane, about 7 instructions per cell. A box that is not a multiple of four rows wide runs the last group of an x-line lane by lane on the same arrays, and the threads get whole groups. The vector statements have the form of the UPML vector rows ($p = -(c_{\mathrm{fo}} \cdot F)$, $f = c \cdot E + p$), the same operations per cell as the scalar path. `OPENEMS_PML_YVEC=0` keeps the scalar rows. SMA launch 1.183× at 1 thread, 1.125× at 4 and 1.129× at 8 (6 rounds, 0.4–0.6 cores; 421 to 498, 1 256 to 1 403 and 1 303 to 1 471 MC/s in the commit message, which gives 0.4–0.7 cores), against the review's estimate of +15–25 %.

UPML coefficients on the setup threads (0020; PR #244). The UPML operator extension computed the PML coefficients of every cell on one thread, evaluating the grading function through the function parser for every cell and component. From upstream PR #244 by Noriaki Takashima (nori-proto) [25] the fork takes the UPML part and its `RunJobsInThreads()` helper: the x-lines are split over the operator's setup threads (`GetNumSetupThreads()`, so the single-threaded engines keep a one-thread setup), each thread evaluates the grading on its own copy of the parser, which is not reentrant, and an exception in a thread is rethrown on the calling thread. The PR's Debye part is not taken (Section 12). Same values in every cell; there is no knob. UPML build time (`OPENEMS_SETUP_TIMING=1`, 6 rounds; 0.3–0.5 cores of other load, commit message): SMA launch 0.086 to 0.022 s (operator setup 0.32 to 0.24 s), notch x3 0.090 to 0.035 s (1.46 to 1.38 s), notch tutorial 0.013 to 0.005 s.

Steady-state detection and Debye materials on the fused sweep (0021, 0022). A sinusoidal excitation adds openEMS's steady-state detection, and that extension had no line hooks, so the multithreaded engine ran every such simulation, all cases of upstream master's `benchmark` tool among them, on the unfused plan. Its `Apply2VoltagesLines()` now records the E-field probes of its x-lines and, on an evaluation step, copies the voltages and currents of its lines (24 B per cell, allocated at the first evaluation step of a fused run); the thread that copies the last lines evaluates the period from the copies with the same energy sum. The unfused hook sees the voltages after their update and the currents before theirs, and so do the copies, line by line, so the values, the operations and the stop step are unchanged. The Debye extension's own pre- and post-voltage hooks get line versions that run the same per-position operations in x order; every position has its own state and reads only its own cell. Neither commit adds a knob; the A/B ran against `OPENEMS_FUSED=0` in the same build, on upstream's cases (1 M cells, 3 000 steps, 5 rounds, about half a core of other load according to the commit messages): pml 1.36× at 4 threads (at 8 within the noise), inhomogeneous 1.29× at 4 and 1.34× at 8, ideal 0.98× and 0.80× (a homogeneous box, where the unfused plan's work stealing evens out the threads within a step; the neighbour flags below recover most of it), and the dispersive case, a Lorentz and a Debye block, 40.8 to 263.9 MC/s at 4 threads (6.5×) and 43.8 to 439.9 at 8 (10.0×), since the unfused plan ran the Debye hooks on one thread.

Geometry coefficient tables (0023; review item 9 in another form). The packed tables of cg13 hold all six coefficient vectors of an operator for each update, 192 B per operator. A graded mesh has many distinct operators (the SMA launch 64 410, 12.4 MB of packed tables), and the tables then stream from memory next to the fields. Most of these operators differ only in their edge lengths: *vi* (*iv*) carries the geometry, *vv* (*ii*) the material. The geometry layout keeps the three *vi* (*iv*) vectors per operator, 48 B in operator order, and a 16-bit index into a small table of the distinct *vv* (*ii*) triplets: 100 B per operator for both updates. The IJKN loops take the six coefficients from either layout, the same values into the same operations. The geometry

layout is chosen when the packed tables would exceed 4 MB, and the packed one when there are more than 65 535 distinct triplets; `OPENEMS_COEF=packed` or `geom` forces one, and the fused sweep runs on either. SMA launch 1.067× [1.021–1.116] at 8 threads, 1.033× with the automatic count, 1.025× at 4 (noisy) and 0.992× at 1 (8 rounds, 0.3–0.4 cores). The other five models have at most 1.5 MB of packed tables and stay packed (geometry tables lose 2–5 % there at 4 threads, commit message). The review's form of the item, the distinct vectors in one table and four 16-bit ids per operator, gave nothing on the four models measured (0.933–1.023×; Section 12).

Neighbour flags (0024, 0025; review item 5). `OPENEMS_FUSED_STAT=1` (0024, a diagnostic) times every thread of the fused sweep: at 8 threads the threads waited 24–27 % of the time at the two barriers. A thread needs only its neighbours: the voltages of its last x-line need the currents of the line after its slab, which the next thread updates as its line before the slab, and its line before the slab needs the final voltages of the previous thread's last line. `FusedTimestepFlags()` (0025) replaces the two barriers by two step counters per thread; a wait spins like the spin barrier and then sleeps on a condition variable, with sequentially consistent stores and loads so that no wake-up is lost. Before its last block a thread waits until the next thread has done its line before the slab in the previous step, then publishes its sweep; before its line before the slab it waits for the previous thread's sweep of this step, then publishes that line. Every cell sees the same operands as with the barriers. Neighbours are then at most one step apart, threads t and $t+k$ at most k, so the extensions' line hooks must not depend on a common step: an extension declares `TS_LOCAL` when its line hooks keep no state shared by x-lines that changes from one step to the next and read the step through `GetNumberOfTimesteps()`, which returns the calling thread's own count while the flags run. Mur, UPML, the excitation, the dispersive family (Drude/Lorentz, conducting sheets, Debye) and the lumped RLC declare it. The lumped RLC used to move its $[n]$, $[n-1]$, $[n-2]$ containers on by a pointer rotation once per step, done by the first thread to arrive; it now takes them from the step number (slot k at step n is array $(k-n-1) \bmod 3$), the same arrays as the rotation: a slot indexed by the time step, as MrHighVoltage proposed in upstream Discussion #492 [23]. With an extension that does not declare `TS_LOCAL` the barriers stay and a verbose message names it; `OPENEMS_FLAGS=0` keeps them too. A stress run in which every wait that is not satisfied at once sleeps (`OPENEMS_BARRIER_SPIN_US=0`, at 3, 8 and 10 threads and the automatic count, under a watchdog) did not hang. Flags against barriers (5 rounds, 0.3–0.6 cores): 1.015–1.055× at 4 threads and 1.073–1.286× at 8 on the six models (Table 3); with them the threads wait 10–22 % of the time at 8 threads (patch x5, SMA, GCPW; commit message). The review had estimated 0–3 % on the GCPW line, +3–8 % on the SMA launch and +4–10 % on the large models at 8 threads; the GCPW line gained most, 1 337 to 1 719 MC/s (1.286×).

The steady-state detection on the flags (0026). The detection kept the barriers: it recorded its probes in a ring of two periods, and the evaluation of a period assumed every thread at the same step. With the flags a thread can be up to the thread count of steps ahead of another, so the ring now keeps three periods (the record of step n at $n \bmod 3p$ for a period of p steps), and an evaluation reads the last two periods by step. While a period is longer than twice the thread count, no thread reaches the next evaluation step or overwrites a record of the last two periods before the evaluation of this one is done, and the extension then declares `TS_LOCAL`. Same records, same sums in the same order. No knob of its own; against the build before (5 rounds): upstream's ideal case (1 M cells, 3 000 steps) 1.18× at 8 threads and 1.13× with the automatic count, ideal M (8 M cells, 600 steps) 1.07× and 1.05×, inhomogeneous 1.41× and pml 1.05× at 8 threads (these two under 1.5–3 cores of other load, commit message).

No flag spin above the P-core count (0027; review item 4 in part). With more threads than performance cores some threads run on the efficiency cores, the waits for them are long, and a spinning thread takes power and a core from the others. The flag waits now spin for the barriers' spin time only up to the P-core count (`hw.perflevel0.physicalcpu`); above it a wait makes its first 128 checks and then sleeps, while the barriers keep their spin at every thread count. On other CPUs the barrier spin time is kept (not measured there). `OPENEMS_FLAGS_SPIN_US` sets the spin for any thread count. Scheduling only. Against the build before (4 rounds, 0.3–0.5 cores): GCPW line 1.088× [1.086–1.133] at 8 threads and 1.123× with the automatic count, notch tutorial 1.094× and 1.051×, 4 threads unchanged (0.999×, 0.998×). Sleeping at once also at 4 threads, all on P cores, cost notch x3 2 % in every round, so the spin stays there.

Identity. The checks of cg29 ran on 2026-11-05 from 02:01 to 02:41 (section 3 of the results file). `check.sh` ran stock as the reference, at its thread count per model (8 threads for patch x5 and notch x3, 2 for the patch tutorial, 4 for the notch tutorial and the two boards), against cg29 at 1, 4, 8 and 10 threads, the automatic count, the geometry tables forced at 4 threads, the packed tables forced at 8 and the barriers instead of the flags at 8: every run of the eight cg29 configurations equals the reference on all six models in probes and S-parameters

(`results/*_cg29.txt`; their speeds are the faster of two unpaired runs under the check's load, not comparisons). The Debye box of `debye_check.sh` gives the same fields in the basic, sse-compressed and multithreaded engines, and the same as cg28. The validation suite passed 15 of 15 checks with the multithreaded engine at 4 threads and with the fastest engine (`results/validation_cg29_mt4.md`, `validation_cg29_fastest.md`); the Metal diamond engine had passed it at cg23 and cg27 (`validation_cg23_metal.md`, `validation_cg27_metal.md`). The cylindrical multigrid example showed 0 differences against stock in 34 datasets. Boxes against the template hooks of the same build: conducting sheets and Drude/Lorentz 16 of 16 identical, UPML 25 of 25, Mur 35 of 35, the steady-state detection 42 comparisons and 49 more on the flags. A run-mode matrix of 45 small models in 11 modes against stock's basic engine (its multithreaded engine at 1 thread on cylindrical meshes): cg29's basic, sse and sse-compressed engines, its multithreaded engine at 1, 2, 4 and 8 threads, with the barriers, unfused and with the geometry tables, and stock's multithreaded engine at 2 threads. It gave 461 identical comparisons; the other 10 are one model on the energy end criterion, which every cg29 mode stops at step 255 while stock runs to its limit of 1 496 steps, with the 64 probe lines up to the stop equal to stock's. The cause is the base, which checks that criterion earlier in short runs (upstream eb1c5ee, Section 3.3), not cg29. `diamond_check.sh` gave 72 of 72, and the Metal diamond engine equals the multithreaded engine run with `OPENEMS_FTZ=1` on the six models (`results/*_cg29gpu.txt`).

Against cg18. Paired against cg18 (5 rounds, 0.3–0.6 cores, each model on its default mesh; notch x3 at the automatic count from four quiet rounds measured afterwards; section 4 of the results file), cg29 runs 1.03–1.23× at 4 threads, 1.13–1.44× at 8 and 1.13–1.42× with the automatic count; the largest gains are at 8 threads on the two boards and the notch tutorial (1.39–1.44×), where the flags count most. On the 1.071 M-cell mesh of Table 10 the SMA launch gains 1.37× at 8 threads and 1.38× with the automatic count (4 rounds, 0.6–0.7 cores). On upstream's benchmark cases, which cg18 runs on the unfused plan (paired, 4 rounds that waited for a quiet Mac, the other processes on a median of 0.4–1.4 cores; every run identical across the builds; section 5 of the results file), cg29 runs 1.11–1.36× as fast as cg18 on the homogeneous ideal box, 1.74–2.18× on the pml and inhomogeneous cases and 4.25–7.40× on the dispersive case (the ideal box of 8 M cells at 8 threads and with the automatic count ran under load and is left out of these ranges). The absolute rates move between sessions, so the paired ratio is the comparison; Table 10 has stock, cg29 and the Metal diamond engine side by side.

5 Setup, post-processing, and the ceiling

5.1 Operator setup

Creating the operator took 4.1 s for both large models, longer than many short runs (this and the other setup times here are from the project log; not in `results/`). Upstream already spreads the material averaging (`Calc_EC`, which asks CSXCAD for the material at several positions around every cell) over the multithreaded operator's thread pool. The time-step search (`CalcTimestep_Var3`, a sweep over every cell) and the coefficient loop (`Calc_ECOperatorPos`, which turns each cell's equivalent-circuit values into its update coefficients) ran on one thread through the shared address helper. Both are now split over `std::thread`s with direct index arithmetic, and the multithreaded operator's setup pool uses every core whatever thread count the engine is given (stock sized it by the requested count). Each lookup in the averaging also got cheaper: the primitive list is passed by const reference instead of being copied per call, and the material lookup remembers its last result per thread, because the averaging asks for two material types at the same coordinate in a row (permittivity and electric conductivity, then permeability and magnetic conductivity). The discretisation deltas are tabulated once; the excitation extension skips y-z lines that contain no excitation primitive. In CSXCAD a new `FindPropertyByCoordPriority` takes the primitive list by const reference and the by-value `GetPropertyByCoordPriority` forwards to it (keeping the ABI); `CSPropMaterial::GetWeight` returns the plain value when no weighting function is set instead of building a parameter set. Result: patch x5 4.1 s to 0.9 s, notch x3 4.1 s to 1.3 s, with 4 engine threads (the fork's setup pool on all ten cores, stock's on four); the real boards set up in 0.1 s. `OPENEMS_SETUP_TIMING` (set to anything but empty or 0) prints the phase times. What remains is flat: material averaging over box primitives. The Metal engine of Section 6 set up its operator on one thread until cg24 (notch x3 5.55 s, against 2.74 s since and 2.61 s for the multithreaded engine; loaded Mac, `results/diamond_engine_cg24_cg27_2026-11-04.txt`); it now uses the multithreaded engine's setup.

5.2 Far field (nf2ff)

The near-to-far-field transform summed, for every observation direction, over every face cell with its own complex exponential: 17.9 s for a 91 × 181 pattern at two frequencies on the patch x5 box (the nf2ff figures

patch	change knob	paired A/B, median ratio; rounds (others' load, cores)
0017	operator index per z-vector, not per cell `OPENEMS_OP_INDEX_PER_CELL=1`: old	patch x5 1.119× (4 thr), 1.101× (8); notch x3 1.042×, 1.052×; the other four 1.005–1.021×; 5 (0.3–1.4)
0018	sheet ADE orders back to back, four positions per `f4vector` (item 2) `OPENEMS_ADE_MERGED=0`: old	GCPW 1.041× (4 thr), 1.031× (8); models without sheets unaffected; 8 (0.3–0.5)
0019	UPML scalar-SSE path, four y-rows per group (item 1) `OPENEMS_PML_YVEC=0`: old	SMA 1.183× (1 thr), 1.125× (4), 1.129× (8); 6 (0.4–0.6)
0020	UPML operator coefficients on the setup threads (PR #244) no knob	UPML build SMA 0.086 to 0.022 s, notch x3 0.090 to 0.035 s, notch tutorial 0.013 to 0.005 s; 6 (0.3–0.5)[a]
0021	steady-state detection: line hooks no knob (against `OPENEMS_FUSED=0`)	upstream cases: pml 1.36× (4 thr), inhomogeneous 1.29× (4), 1.34× (8), ideal 0.98× (4), 0.80× (8); 5 (about 0.5)[a]
0022	Debye material: line hooks no knob (against `OPENEMS_FUSED=0`)	upstream dispersive case 40.8 to 263.9 MC/s (4 thr, 6.5×), 43.8 to 439.9 (8 thr, 10.0×); 5 (about 0.5)[a]
0023	geometry coefficient tables, 100 instead of 192 B per operator, when the packed ones exceed 4 MB (item 9) `OPENEMS_COEF=packed` or `geom`	SMA 1.067× (8 thr), 1.033× (auto), 1.025× (4), 0.992× (1); the other five stay packed; 8 (0.3–0.4)
0024	per-thread time split of the fused sweep `OPENEMS_FUSED_STAT=1`: on	diagnostic: at 8 thr 24–27 % of the time waiting at the barriers
0025	neighbour flags instead of the two barriers; RLC slots by time step (item 5; Discussion #492) `OPENEMS_FLAGS=0`: barriers	six models 1.015–1.055× (4 thr), 1.073–1.286× (8; GCPW 1.286×, SMA 1.109×); 5 (0.3–0.6)
0026	steady-state detection declares `TS_LOCAL` (runs on the flags) no knob of its own	upstream ideal 1.18× (8 thr), 1.13× (auto); ideal M 1.07×, 1.05×; inhomogeneous 1.41× and pml 1.05× (8); 5 (1.5–3 for the last two)[a]
0027	no flag spin above the P-core count (item 4 in part) `OPENEMS_FLAGS_SPIN_US`	GCPW 1.088× (8 thr), 1.123× (auto); notch tutorial 1.094×, 1.051×; 4 thr 0.999×, 0.998×; 4 (0.3–0.5)

Table 3: The CPU items of snapshot cg29 (patches 0017–0027, one commit each), all bit-identical to stock v0.37.0. Paired interleaved A/B (`bench/abtest.py`), median of the per-round ratios, number of rounds and the other processes' load in cores; knob against knob in one build, 0020, 0026 and 0027 against the build before, 0021 and 0022 against `OPENEMS_FUSED=0`. The SMA launch on its 0.871 M-cell mesh. From `results/cpu_items_cg29_2026-11-05.txt`, section 1; [a] load from the commit message. Item numbers are those of Table 32.

are also from the project log, except those of cg18 below). On a Cartesian plane the phase factor $\exp(jk\,\hat{r}\cdot\vec{r})$ separates per mesh line, so the plane integral for all directions is a complex matrix product (one `cgemm` per block of directions) followed by a row dot product. The separable path needs a CBLAS, and only Apple's Accelerate is wired in: other builds, cylindrical meshes and `OPENEMS_NF2FF_LEGACY` (set to anything but empty or 0) keep the original threaded loop. 17.9 s to 1.0 s (tutorial: 1.36 s to 0.33 s), with the pattern within 7×10^{-7} of its maximum (a weak cross-polar component can differ by more relative to its own peak); the loop itself is unchanged and bit-identical. Measured again with cg18 (patches 0001–0006) on the tutorial's pattern (91×181 directions, two frequencies): 1.21 s with stock, 0.30 s with the separable path, which stays within 6.8×10^{-7} of the maximum, while the legacy loop gives stock's pattern bit for bit (`results/correctness_cg18_2026-11-03.txt`).

5.3 Dumps and probes

Field dumps and probes read the engine's fields through an interface that, for the SSE engines, located every value with per-cell division and modulo. Upstream's GPU branch, at the commit this fork is based on (7bb294f), had introduced a precomputed *gather* (entry indices of the values a dump needs); `Field_Gather_SSE` maps those entries to `f4vector` float offsets so the SSE engines use it too. The gather that upstream later merged into master is region-based and stores nothing per node, so this mapping would have to be redone there. The patch tutorial, whose nf2ff box is 12 time-domain dumps on a 0.1-million-cell model, went from 1 076 MC/s (project log) to 1 519 MC/s at 4 threads with the gather (snapshot cg2); after the Mur line hooks it ran at 1 806 MC/s (cg8), and the dumps still cost about 14 % there (2 100 MC/s without them; project log): the CPU engines evaluate the dumps on the main thread, and only the writing runs in the background. Upstream's asynchronous HDF5 writer is used as is; with it the fork's base alone runs this model 1.13× faster than stock (308.5 against 273.4 MC/s, 2 threads), and with the dumps off the two are equal (`results/patch_s1_n10500_base.txt`, Section 8). Our own background writer from the first snapshots was dropped as redundant. The Metal engine of Section 6 also leaves probes and dumps to the CPU, between its command buffers and on the shared memory; on this model its GPU was busy 78 % of the time-stepping loop (cg24, `results/diamond_engine_cg24_cg27_2026-11-04.txt`, section 6).

5.4 Where the ceiling is, and the GPU engines

This subsection is about the M5 (4 + 6 cores, 153 GB/s). On the M5 Pro (Section 10) the bus held the CPU engine back only on the largest grid measured, a 16.8 M-cell box: the fused sweep reaches 5 602 MC/s on a 2.95 M-cell box at 15 threads and scales to 15 threads on every other model. Figure 2 puts the engines measured up to snapshot cg14 on one axis: effective DRAM traffic, the product of the speed and the bytes a cell update moves in the traffic model of Section 4.5. An engine that keeps a tile's fields on chip for several time steps has no such figure, so the diamond-blocked Metal engine of Section 6 is not a bar there. The traffic is modelled, not measured with hardware counters, and the model counts the operator index as 1 B per cell; the engine allocated 4 B of index per cell up to cg28 (one entry per z-vector since cg29, Section 4.14) and on meshes with few z lines streamed all of it, so the CPU figures below are lower bounds (the whiskers in the figure show the full 4 B). On the 201^3 box three independent implementations ended within 10 % of each other in cell rate (1 562–1 670 MC/s, project log): the two unfused Metal kernels at 121–124 GB/s of an assumed 74 B/cell and the fused CPU sweep of the time (snapshot cg9) at 77 GB/s of 49 B/cell. With the standalone material-table kernel of Section 2, the GPU kernels drew 121–127 GB/s. The CPU engine of cg18 was not run on that box. On another 201^3 box, with Mur faces, the build of snapshot cg19 (cg18's CPU code and the Metal engine of Section 6) ran 1 579 MC/s on the multithreaded engine at 8 threads, 1 446 on the base's GPU engine and 2 775 on the diamond engine at $K = 4$ (rough, load average 2–7; `results/diamond_engine_cg19_2026-11-04.txt`).

- **Upstream's Metal engine** (`--engine=gpu`) as it was on the pre-merge `gpu-engine` branch at commit 7bb294f (2026-10-12): added by Sean Mollet (commit bf30fa4, 2026-10-08), with follow-up work by Thorsten Liebig. Upstream merged a reworked version into master as c3a956b on 2026-10-17 and has since deleted the branch; the merged engine was measured on 2026-11-04 (next item). 1 638 MC/s on the 201^3 box (project log); in `results/` 923–1 115 MC/s on the patch tutorial and 1 622 on the notch tutorial, 1 502–1 519 and 1 354–1 358 on the two large models, 1 337 (GCPW) and 1 135 (SMA launch, 1.071 M cells) on the real boards. Its voltage and current updates are separate passes. It was ahead of earlier CPU snapshots on some small models, where the CPU engine still paid more for per-step synchronisation, and on the 201^3 box (against cg9); against the CPU engine of Table 9 (cg14) and against cg18 (`results/*_cg18.txt`) it is slower on all six benchmark models, and on the 201^3 Mur box above as well.
- **Upstream master's merged Metal engine** (c3a956b [21], in a build of master 5fecbb5), measured on 2026-11-04 next to cg18 (best of two runs on a loaded Mac; `results/others_2026-11-04/README.md`): level with cg18 on patch x5 (1 529 against 1 510 MC/s) and on the SMA launch at 0.871 M cells (1 084 against

1 019), slower on the other four (notch x3 1 334 against 2 010, patch tutorial 782 against 1 856, notch tutorial 1 635 against 1 822, GCPW line 1 343 against 1 634): 0.42–1.06 times cg18. The GPU engines of master and of luikore's pull request differ from stock by 1.7×10^{-7} to 1.3×10^{-6} relative in the port probes (float rounding; on the SMA launch master differs by 1.5×10^{-4}, a change in its CPU engine since v0.37.0 that its GPU engine shows too; Section 3.3). The merged engine's structure was not examined here, and `results/` has no run of it next to the diamond engine on these models; on luikore's benchmark model, on 2026-11-11, master's `gpu` engine (at 1d710c2) ran 1 638 and 1 566 MC/s at 3.0 and 17.0 M cells against 3 981 and 3 309 for the diamond engine of cg31 (Table 19). On the PEC variant of the notch tutorial (Section 11) it ran at 2 374 MC/s against cg18's 2 053, the faster of two runs (the other 1 452), above the roughly 2 000 MC/s that 74 B per cell allow from the M5's memory (Section 2); whether its kernels fuse the two half steps or this 0.270 M-cell model stays in the GPU's caches was not examined.

- **A Swift/Metal kernel** written for this project (`swift_engine/`; its unblocked numbers are from the project log): the fork exports the compressed operator (`OPENEMS_EXPORT_OPERATOR=dir`: per-vector index, the four coefficient tables, the voltage excitation entries and signal) and the final fields (`OPENEMS_EXPORT_FIELDS=dir`); the kernel mirrors `Engine_sse`'s operation order, uses `fma()` exactly where clang fuses on arm64, and runs with Metal's fast math off. It is bit-identical to every CPU engine on a 41^3 box over 600 steps, and on 121^3 to the SSE-family engines once both sides flush subnormals (see below; the basic engine has no flush-to-zero switch). 1 670 MC/s at 201^3, independent of the command-buffer batch size from 20 to 400 steps. Like upstream's engine it updates voltages and currents in separate passes. On 2026-11-04 it got a diamond-blocked mode (`--diamond K`), the prototype of Section 6: identical to its unblocked kernel and to the CPU engine of cg18 (with `OPENEMS_FTZ=1` on the larger boxes) for every K tried on 41^3, 121^3 and 201^3 boxes, and 2 445.7 MC/s at $K = 4$ on 201^3 (2 574.8 at its best tile and threadgroup size) against 1 540.0 unblocked (one run each, load average 6–12; `results/diamond_swift_2026-11-04.txt`).
- **The CPU engine (cg14, cg18)**: 1 764 and 2 099 MC/s on the two large models (cg14, Table 9; cg18 1 716 and 2 046, `results/*_cg18.txt`), 86–103 GB/s at the model's 49 bytes per cell (92–109 with the full 4-B index); a standalone fused kernel of the patch x5 shape, with a compact index of 1 B per cell, tops out at 1 932 MC/s on 4 threads, and one independent sub-grid per thread (no neighbour exchange at all) at 2 273 on 6, 111 GB/s at 49 B per cell (both from the project log): that is the ceiling the cores see in a sweep that reads the fields from memory once per time step (the opt-in tiled sweep of Section 9.6, which reads a tile once for several steps, ran patch x5 at 2 305 MC/s at 4 threads on 2026-11-11). Snapshot cg29 (Section 4.14) allocates the operator index per z-vector, as the model counts it, and with its other CPU items runs notch x3 1.134× [1.018–1.140] and patch x5 1.257× [1.232–1.269] faster than cg18 at 8 threads (paired, 5 rounds, others 0.6 cores; `results/cpu_items_cg29_2026-11-05.txt`, section 4); its rates are in Table 10, where its fastest CPU rate, notch x3 at 8 threads, is 2 177 MC/s, below that ceiling.

The unfused GPU engines measured here (the base's and the Swift/Metal kernel) are therefore at best about as fast as the fused CPU sweep on this machine, and upstream master's merged engine is level with cg18 on two of the six models and slower on four. At the 121–127 GB/s the GPU kernels drew, 49 B per cell would bound a fused GPU sweep at about 2 470–2 590 MC/s (computed from the traffic model; a bound, not a prediction): 1.2–1.5 times the CPU engine of cg14 on the large models and about 1.1 times the cores' 2 273 MC/s. Until 2026-11-04 this section said that such a sweep was not tried and that it would need a grid-wide synchronisation between the voltage and the current update, or halo-tiled threadgroups. The diamond engine of Section 6 needs neither: a tile updates its voltages and then its currents with a threadgroup barrier between them, and four dispatches per block of steps (mountain and valley tiles in x and y; three since cg31, Section 9) complete the tile edges. Run with one time step per block ($K = 1$) it is a fused, unblocked GPU sweep, and it reached 1 716–1 733 MC/s on notch x3 and up to 1 844 on patch x5 (cg27; paired, 4 rounds, others' CPU 2.5–4 cores; `results/fused_sweep_k1_2026-11-04.txt`), 67–75 % of the bound or 85–90 GB/s at 49 B per cell: the four dispatches, the tile edges and the barrier between the half steps cost more than the bytes they save. Blocking over time steps passes the bound where fusing alone does not. With $K = 2$ the two models ran at 2 123 and 2 408 MC/s, still below it; with $K = 4$ at 2 986 and 2 817, above its upper end; and with the tiling chosen at run time (cg28) notch x3 reached 3 226 MC/s on a quiet Mac (Table 5). A one-dispatch fused kernel was not built; it would need a second copy of both fields. fp16 field storage was not tried (the half-precision normal range ends at 6×10^{-5} while late-run currents are 10^{-6} A, project log, so fields need scaling; and it is not bit-identical by construction). On the CPU, two time steps per sweep gained 10–14 % in the standalone kernel of 2026-10-28 (project log), on top of a fused sweep that already reuses the fields between the two half steps. An upper bound measured for cg29 on the fork's fused loops, with one independent sub-grid per thread and so without halos or extension hooks, gave at most 6 % for 2 to 4 steps (`results/cpu_items_cg29_2026-11-05.txt`, section 2), and a real version of that design was not built. Upstream master's `cpu` engine does block on the CPU, with all threads in one cache-sized tile, and on this machine the blocking is worth 1.15–1.49× to it (Section 9.4); the bound above is for one sub-grid per thread, not for that form. The fork has had that form since 2026-11-11, as an opt-in tiled sweep (patch

0040, Section 9.6): at 4 threads it ran patch x5 and a 16.8 M-cell box 1.08 and 1.23× as fast as the fused sweep at its best, and four other models at 0.71–0.92×. Wavefront or diamond blocking [17, 16] beyond that sweep's trapezoids along x was not tried on the CPU. On the GPU, luikore's Metal engine [20] advances a diamond of four time steps per block and reports 1.3–4.2× its own unfused path on an M4 Pro and an M5 Max, on a uniform grid without conducting sheets (Section 11). It was built and run on this M5 on 2026-11-04 (`results/others_2026-11-04/`, best of two runs on a loaded Mac). Its diamond path refused all six models before stepping (it names the Mur extension, on the GCPW line the conducting sheets). On variants of five of them with every Mur face replaced by PEC, which are speed-only cases, it ran at 0.39 to 2.00 times cg18, and its legacy path was slower than cg18 on every model. The diamond engine of Section 6 was not run on those variants, so on these models the two diamond engines have not been compared head to head. On luikore's own benchmark model they were, on 2026-11-11: cg31's engine was level with luikore's at 3.0 M cells (3 981 against 3 952 MC/s) and 1.17× as fast at 17.0 M (3 309 against 2 829; Table 19).

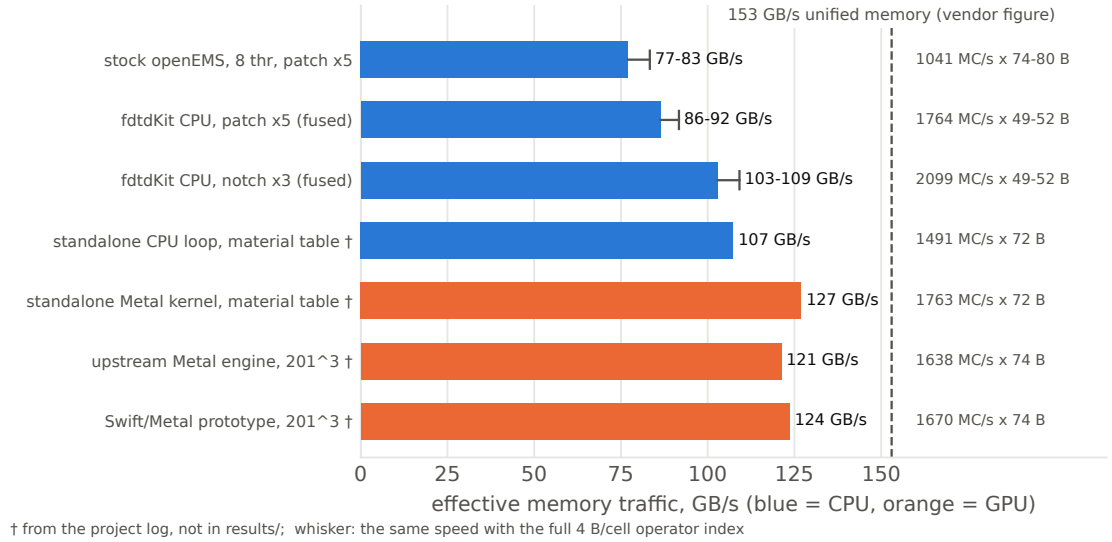


Figure 2: Effective memory traffic of the engines measured up to snapshot cg14, against the M5's bus: cell rate times the bytes per cell of the traffic model (not counter-measured). The openEMS CPU bars count the operator index as 1 B per cell, their whiskers the full 4 B per cell it was allocated with up to cg28. CPU in blue, GPU in orange (upstream's pre-merge Metal engine and the Swift/Metal prototype: assumed 74 B/cell). The two standalone material-table kernels and the two 201^3 bars are from the project log, the others from `results/`. The Metal diamond engine of Section 6 has no bar: with K time steps per tile its traffic depends on what the caches hold, and the traffic model gives no bytes-per-cell figure for it (its $K = 1$ case, a fused sweep, would be a bar at 85–90 GB/s, `results/fused_sweep_k1_2026-11-04.txt`). Upstream master's merged engine and luikore's engine, measured on 2026-11-04, are not in the figure either.

Subnormals. The Swift/Metal kernel's first differences from the CPU engines appeared as 3×10^{-43} versus 0 at step ∼50 and grew to 3×10^{-7} of the field maximum by step 600 on the 121^3 box (project log): Apple's GPU flushes subnormal floats to zero, the cores keep them. Upstream openEMS sets flush-to-zero by default, but only on x86 (a TODO in `tools/denormal.h` asks whether other CPUs need it). Apple's cores pay no penalty for subnormals (the GCPW line runs at the same speed either way, `results/kgcpw_s1_n8000_kirchftz.txt`), so the fork keeps them by default. `OPENEMS_FTZ` (set to anything but empty or 0; arm64 only) sets `FPCR.FZ` in the threads of the SSE-family engines (`sse`, `sse-compressed`, `multithreaded`; not `basic`) and gives the creating thread its previous flush-to-zero bit back when the engine is torn down; with it these engines and the Swift/Metal kernel agree on every value, and on the notch tutorial the multithreaded and sse-compressed engines' port probes and S_{11} equal those of upstream's GPU engine over 3 000 steps (without the flush the probes differ by 3.2×10^{-7} relative; `results/notch_s1_n3000_ftzgpu.txt`, Section 3.3). On 2026-11-04 the same held on all six models, for the base's GPU engine and for the diamond engine of Section 6 (`results/*_cg22gpu.txt`); the diamond engine's field dumps compare equal only when both runs set the switch, because openEMS computes them on the creating thread, which the switch also flushes (`results/diamond_engine_cg20_2026-11-04.txt`). A run with the flush is not bit-identical to the default run: only subnormal values are flushed, but the difference spreads, to the same 3×10^{-7} of the field maximum after 600 steps; on the GCPW line it moved the port probes by 4.1×10^{-7} relative and S_{21} by 1.8×10^{-6} dB

over 8 000 steps on one thread (`results/kgcpw_s1_n8000_kirchftz.txt`).

6 A diamond-blocked Metal engine

Section 5.4 ends at the memory bus. The fused CPU sweep and the unfused GPU engine of the fork's base stream every field array from DRAM at least once per time step, and at the 121–127 GB/s the GPU kernels drew, a fused GPU sweep that moves 49 B per cell is bound to about 2 470–2 590 MC/s. Temporal blocking moves that bound: a tile of the grid is advanced by K time steps while its fields stay in the GPU's caches, so a field value need not cross the bus at every step. The fork's engine for this, `--engine=metal-diamond`, was written on 2026-11-04 in ten snapshots, cg19 to cg28 (patches 0007–0016, commits 6b23756 to 68c1c82; `CHANGELOG.md`). Its tile schedule follows luikore's Metal engine, openEMS PR #228 [20] (GPL; the diamond tile schedule; no code of it is used). The kernels are the fork's own. They began as a `--diamond K` mode of the Swift/Metal prototype of Section 5.4, which matched the CPU engine bit for bit at every K tried (`results/diamond_swift_2026-11-04.txt`), and were then moved onto the engine's own arrays and given the extensions that the six models use. The engine is opt-in and Apple-only: it is built with the base's Metal option (`ENABLE_METAL`), and the default engine stays the multithreaded CPU engine. The code is `FDTD/engine_metal_diamond.h` and `FDTD/metal/engine_metal_diamond.mm`; against cg18 the ten patches change 19 files, +1 717/ − 1 lines, of which the Objective-C++ file has 1 592 (`git diff --stat cb168f7 68c1c82` in the fork). Snapshot cg29 leaves these files unchanged; its CPU changes were checked against the engine (below). The build of 2026-11-11, cg31 (patches 0028–0039, of which 0029–0038 are the engine's), changes the engine again: this section describes and measures it as of cg29, and Section 9 has what changed and the rates since.

6.1 Design

Fields in shared memory. `Engine_Metal_Diamond` is a subclass of the sse-compressed engine, and its operator is, since cg24, the multithreaded engine's (`Operator_Multithread`). The voltage and current arrays are the engine's own `f4vector` arrays, placed in shared Metal buffers in the unified memory (`ArrayENG::InitView`), so the probes, the dumps and the CPU fallback read and write the memory the kernels use, without copies. Their order is N-I-J-K, the component outermost, instead of the CPU engines' I-J-K-N, where the three components of a cell lie side by side and the loads of a SIMD group are 48 B apart: on Mur boxes of 121^3 and 201^3 cells N-I-J-K ran 7–9 % faster (GPU time, three alternating runs each, on a quieter Mac; `results/diamond_engine_cg19_2026-11-04.txt`, section 4). At the first time step the engine copies the operator's coefficient tables and the extensions' data into Metal buffers and compiles its kernel for the model, with the stages that the model does not use left out; this GPU setup took 3–33 ms on the six models (`results/diamond_engine_cg24_cg27_2026-11-04.txt`, section 5).

Mountains and valleys. Along x and along y the lines are cut into mountains of width M with valleys of width V between them; z stays whole in every tile. One threadgroup advances one tile by K time steps, $2K$ half steps, in place. A mountain $[l, h)$ shrinks as it climbs: at half step s it updates the lines $[l + \lfloor s/2 \rfloor,\ h - \lceil s/2 \rceil)$, so every value it reads at its edges is still at the step it needs. A valley between two mountains, with l the end of the mountain before it and h the start of the one after, grows by the same rule, $[l - \lceil s/2 \rceil,\ h + \lfloor s/2 \rfloor)$, and completes what the mountains left. With tiles in x and in y, a block of K steps takes four dispatches, one per kind of tile: mountain in x and in y, mountain in x and valley in y, valley in x and mountain in y, valley in both (three since cg31, which sends the two mixed kinds together; Section 9). The tiles of one dispatch are independent of each other. Inside a tile a threadgroup barrier on device memory separates the stages, and the voltage half step from the current half step. Threadgroup memory is not used: the reuse across the K steps comes from the GPU's caches, and there are no halo copies. The first mountain starts at line 2 and the last ends at line $n - 2$ at the latest, so the two lines at each domain edge (a Mur face and the line it reads) always lie in one edge valley; a remainder of at least $2K - 1$ lines becomes a narrower last mountain, and M must be at least $2K - 1$. The host puts whole blocks of K steps into a command buffer until it holds at least 64 steps (66 at $K = 3$) or the call ends, and commits the buffers in order. An `IterateTS` call waits for its last buffer, so the probes, dumps and end criteria that openEMS evaluates between calls see the fields of that step, on the CPU.

Extensions. Each extension that touches the fields during a step has a stage in the kernel, run per tile on the tile's cells, in the CPU engine's order: the Pre stages in reverse extension order, the update, then the Post and Apply stages in extension order (by priority UPML first, then Mur, lumped RLC and the dispersive materials, the excitation last; `results/diamond_engine_cg22_cg23_2026-11-04.txt`):

```
E step: [Mur Pre, Lorentz Pre, Debye Pre] -> UPML Pre -> update -> UPML Post -> [Mur Post, Debye Post]
        -> Apply list (Mur faces, lumped RLC, field -= ADE per order) -> voltage excitation
H step: Lorentz Pre -> UPML Pre -> update -> UPML Post -> dispersive current Apply -> current excitation
```

The stages in brackets only read the field and run before one barrier.

- **Excitation** (cg19): voltage and current sources with their delays and signal periods. At first thread 0 of every tile scanned all entries at every half step; on a 201^3 PEC box with 1 035 voltage entries the rate in GPU time was then 1 914 MC/s, and per-tile lists of the entries in the tile's ranges, kept in entry order, raised it to 3 135 (1 024 threads per tile, rough; `results/diamond_engine_cg19_2026-11-04.txt`, section 5). Since cg25 the entries form *runs*: all entries that write one float go to one thread, in the CPU's order, coded in one `uint4` (the float's index, x and y in 16 bits each, the first and the end entry), so x and y may have at most 65 535 lines.
- **Mur** (cg19): Pre, update, Post and Apply of the face cells in the tile's ranges; the edge valleys keep a face and the line it reads in one tile.
- **Lumped RLC** (cg20): one state record per element (I_l, $V_d[n-1]$, $V_d[n-2]$, $J[n-1]$, $J[n-2]$), in the Apply order together with the Mur faces.
- **UPML** (cg21): per component and cell an index into a table of distinct coefficient sets, as in the CPU's compressed PML tables (cg14). One barrier serves a stage of all boxes, so the boxes must not overlap; openEMS's do not (`Create_UPML` leaves out the edges and corners that another box has). Up to cg29 a stage was a loop over single floats; since cg31 it takes a z-vector at a time in the x and y slabs and the three components of a cell together in a z slab (Section 9).
- **Drude/Lorentz materials and conducting sheets** (cg22, `Engine_Ext_LorentzMaterial`: voltage and current ADEs) and **Debye materials** (cg23, `Engine_Ext_DebyeMaterial`), each only as exactly that class (`typeid`), not as a subclass. Since cg25 the ADE elements of one cell form one run, as the excitation's entries do.

The extension data sit in three buffers (read-only words, read-only floats, state) with a table of offsets, because the kernel had used 29 of Metal's 31 buffer slots before the UPML came (cg21). Since cg31 the offsets and counts are constants of the kernel, which is compiled after the data are packed, and a tiling's per-tile lists have a buffer of their own (Section 9).

Coefficients, constants and the tuner. The packed tables of cg13 hold six `f4vector`s per operator for the voltages and six for the currents, 192 B per operator. That suits the uniform tutorial meshes, which repeat a few operators, but the SMA launch's graded mesh (0.871 M cells, the mesh of the SMA figures in this section) has 64 410 operators, one for every 3.4 `f4vector`s of the grid, 12.4 MB packed. Split tables (cg27) store the distinct `f4vector`s once, 88 485 of them in 1.4 MB for the SMA launch, and per operator a `uint4` of indices for each of the two updates (32 B), 2.1 MB (the voltage-voltage and current-current factors first, at most 1 024 of them for 10-bit indices), so the caches hold the coefficients of more cells for one more dependent load per cell. They are the default where the packed coefficients of a 16×16 block of (x, y) columns exceed 32 KB on average: the SMA launch (275 KB), the GCPW line (46 KB) and the patch tutorial (58 KB); notch x3 (6 KB), patch x5 (21 KB) and the notch tutorial (9 KB) stay packed (`results/diamond_engine_cg24_cg27_2026-11-04.txt`, section 4). Since cg26 the grid sizes are constants of the compiled kernel, so its divisions by them become multiplications. Since cg28 the tiling is chosen at run time. The engine builds four tilings, $(K, M, V) = (4, 8, 8)$, the former default, $(4, 12, 8)$, $(3, 8, 6)$ and $(3, 6, 6)$, each with its own tile buffers, threads per tile and per-tile lists; the extension floats and the state are shared. While it tunes, the command buffers take the tilings in turn, in half batches of 32 steps (33 at $K = 3$) with at most two in flight, and per tiling the engine keeps the least GPU time per step of four timed buffers; then it runs the fastest tiling and releases the others (since cg31 seven tilings, timed in two stages: $(4, 10, 6)$, $(4, 9, 2)$ and $(5, 12, 8)$ are added; Section 9). Only full batches and whole calls are timed and take a turn: a first version that also gave the short last buffer of each call a turn handed one tiling that buffer on every call of notch x3, and other call lengths could have starved a tiling for a whole run (`results/diamond_engine_cg28_tuner_2026-11-04.txt`, section 4). Threads per tile follow the tile's work: the power of two at or below 3/5 of the `f4vector`s that a mountain updates per half step ($M \cdot M \cdot n_v$), 256 to 1 024 (cg21).

Knobs and fallback. `OPENEMS_DIAMOND_K` and `OPENEMS_DIAMOND_TILE` (`M,V` or `M:V`, default $2K, 2K$) fix one tiling, `OPENEMS_DIAMOND_TUNE=n` sets the timed buffers per tiling (4; `0` runs $K = 4$, 8:8 without tuning), `OPENEMS_DIAMOND_TG` the threads per tile, `OPENEMS_DIAMOND_BATCH` the steps per command buffer (64, rounded up to whole K-step blocks), `OPENEMS_DIAMOND_COEF=packed|split` the coefficient layout, and `OPENEMS_DIAMOND_TIMING=1` or `2` prints the GPU time, per phase with `2` (which also fixes one tiling). cg31 adds eight switches for its changes (Section 9; all in `BUILD.md`, section 7). Whatever the engine does not cover runs on the CPU, on the same memory, with a message: an extension without a GPU stage (named in the

message; TFSF, the steady-state detection, any subclass of the dispersive extensions), overlapping UPML boxes, more than 65 535 lines in x or y, 2^{32} or more floats per field, no Metal device or a failed allocation; more than 32 Mur faces or UPML boxes, or more than 65 535 lumped-RLC or dispersive extensions, fall back without a message. The fallback is the sse-compressed engine's generic loops on one thread; at cg19 the six models all ran it, and their speeds there mean nothing (`results/diamond_engine_cg19_2026-11-04.txt`, section 2). Cylindrical meshes run openEMS's cylinder engine whatever the option says. Table 4 lists what each snapshot put on the GPU.

snapshot	patch	commit	added on the GPU, or changed	benchmark models on the GPU	`diamond_check`
cg19	0007	6b23756	cell updates; voltage and current excitation with delays and periods; Mur	none	18
cg20	0008	7ac3438	lumped RLC elements	patch tutorial, patch x5	24
cg21	0009	2c79751	UPML; extension data in three buffers; threads per tile from the tile size	+ notch tutorial, notch x3, SMA launch	36
cg22	0010	5ea5ffd	Drude/Lorentz materials and conducting sheets	+ GCPW line: all six	48
cg23	0011	0242c5f	Debye materials	all six	54
cg24	0012	8dd56e5	operator setup on all cores (`Operator_Multithread`); `OPENEMS_DIAMOND_TIMING`	all six	54
cg25	0013	3db2791	excitation and dispersive elements as runs; per-tile offsets read once	all six	54
cg26	0014	ca7c334	grid sizes as constants of the kernel; Mur and UPML records passed by value	all six	54
cg27	0015	282707b	split coefficient tables for graded meshes	all six	54 + 54
cg28	0016	68c1c82	tiling chosen at run time	all six	72

Table 4: What runs on the Metal diamond engine, per snapshot (all of 2026-11-04). A model runs on the GPU when each of its extensions has a GPU stage; the others run the CPU loops (at cg19 all six: each has PML, lumped ports or conducting sheets). `diamond_check`: comparisons of every exported field value of the box models with the multithreaded engine run with `OPENEMS_FTZ=1`, all identical (`results/diamond_engine_cg19_2026-11-04.txt` to `results/diamond_engine_cg28_tuner_2026-11-04.txt`; the logs of cg21–cg23 are `results/diamond_check_cg2N_2026-11-04.txt`); on cg27 54 more with the split tables forced. cg24's 54 of 54 is recorded only in its commit message (patch 0012), not in `results/`; the 180 runs of the cg24–cg27 session, all equal to the first run of cg24, the CPU engine with `OPENEMS_FTZ=1` among them, show its identity on the six models as well. cg29 changes only CPU code; its `diamond_check` is 72 of 72 too (`results/cpu_items_cg29_2026-11-05.txt`, section 3). Never on the GPU: TFSF, steady-state detection and subclasses of the dispersive extensions (CPU loops), cylindrical meshes (openEMS's cylinder engine).

6.2 Bit identity

The engine is held to the fork's rule (Section 3.3) against the CPU engine whose arithmetic it repeats. Every value gets the CPU engines' operations in the same order. Each component of the cell update is the rounded product and the one fused multiply-add that Apple clang makes of the CPU expression on arm64:

```
v = volt[...];  v = v * vv;  v = fma(vi, ((c2 - c2ym) - c1) + c1zm, v);  volt[...] = v;   // the currents
    alike
```

`fma()` is written where clang fuses, `#pragma METAL fp contract(off)` holds everywhere else, and Metal's safe math mode is set (fast math off). The extension stages keep the CPU engine's order and their own operation order, and an excitation run applies the entries of one float in entry order. The tiles, the tiling the tuner picks and the batch size change only the order in which independent values are computed. One difference remains: Apple's GPU flushes subnormal floats to zero, while the CPU cores keep them unless `OPENEMS_FTZ=1` sets `FPCR.FZ` (Section 5.4). The reference is therefore the multithreaded engine run with `OPENEMS_FTZ=1`:

- `scripts/checks/diamond_check.sh` compares every exported field value of box models after the run: PEC and Mur boxes; mixed faces with a delayed voltage excitation, a soft current excitation and an excitation on a Mur plane; 16 lumped RLC elements; PML on all six faces and PML mixed with Mur; two conducting sheets; Drude/Lorentz and Debye blocks. Each runs at $K = 1$, 2, 3, 4 and 8 and with an odd tile (7,0), 96 threads and batches of 3 steps, and since cg28 also with the tuner changing tilings during the run. Every

comparison was identical, 18 at cg19, 24 at cg20, 36 at cg21, 48 at cg22, 54 at cg23 to cg27 (cg24's in its commit message), 72 at cg28 and cg29 (Table 4), and 72 at every snapshot of cg31 (Section 9).
- The six benchmark models: port probes and S-parameters identical to the multithreaded engine with `OPENEMS_FTZ=1` on every model that ran on the GPU, two at cg20, five at cg21 and all six from cg22 on (`results/*_cg2Ngpu.txt`; for cg29 `results/*_cg29gpu.txt`). The 180 runs of the cg24–cg27 session were all identical to the first run of cg24, and the 190 runs of the two cg28 sessions to the first run of cg27 (`results/diamond_engine_cg24_cg27_2026-11-04.txt`, section 3; `results/diamond_engine_cg28_tuner_2026-11-04.txt`, section 1). The Metal runs of cg31 give the probes of cg29's (`results/metal_engine_cg31_2026-11-11.txt`).
- The nf2ff time-domain dumps of the patch tutorial (984 datasets) and of patch x5 (120) are identical when both runs set `OPENEMS_FTZ=1`. With the switch in the CPU run only, 27 of the tutorial's 984 differed while the fields were identical, because openEMS computes the dumps on the creating thread, which the switch also flushes (`results/diamond_engine_cg20_2026-11-04.txt`).
- The closed-form validation suite (`bench/validation/`) passes 15 of 15 on the engine at cg23 and at cg27 (and at cg30l, the snapshot of patches 0001–0038 on the way to cg31, Section 9). The Debye, Drude and Lorentz slabs, the rectangular cavity, the WR-90 guide with PML and the microstrip line ran on the GPU without a fallback, and the three dispersive slabs equal the basic engine (difference 0); the cylindrical cavity runs openEMS's cylinder engine (`results/validation_cg23_metal.md`, `results/validation_cg27_metal.md`).

The base's GPU engine (`--engine=gpu`, the pre-merge branch at 7bb294f), compared the same way, is identical to the multithreaded engine with `OPENEMS_FTZ=1` on all six models too (`results/*_cg22gpu.txt`); upstream master's merged engine was not compared with a run that flushes subnormals (Section 3.3). Against stock v0.37.0, which keeps subnormals, the diamond engine's differences are those of the flush. They are measured, not acceptance criteria: 1.7×10^{-7} to 9.4×10^{-7} relative in the port probes, at most 2.3×10^{-4} dB in S_{11} (notch tutorial; at most 2.3×10^{-5} dB on the other models) and at most 9.4×10^{-5} dB in S_{21} (`results/*_cg22.txt`; the GPU results have not changed since). End to end through the toolkit (cg23, CPU runs without the flush) the runs stop at the same steps; S_{11} of the SMA launch differs by at most 7.3×10^{-3} dB (-22.69 dB at 1 GHz in all four runs), S_{21} by at most 2.0×10^{-4} dB, and on the GCPW line S_{21} by at most 3.9×10^{-6} dB (`results/rfsim_e2e_gpu_2026-11-04.txt`).

6.3 Speed

The Mac was shared during this work. The cg19 to cg23 runs were made on a loaded Mac (load averages of 2 to 8 where the files give them), and in the cg24–cg27 sessions other jobs took 0.5–7.7 cores and up to 31 % of the GPU; the files call single runs rough, and the comparisons that count are paired. Of the comparisons of cg19 to cg28 only cg28's ran on a quiet Mac; the cg29 size sweep below waited for one before every run. Ratios are paired and interleaved (Section 3): the median of the per-round ratios, with their range.

When the GPU wins. On Mur boxes at cg19 (rough, load average 2–7) the engine at $K = 4$ ran 2 447 MC/s on 121^3 cells and 2 775 on 201^3, against 1 795 and 1 579 for the multithreaded engine of the same build (the CPU code of cg18) at 8 threads and 1 472 and 1 446 for the base's GPU engine. On 61^3 cells (0.23 M) the CPU at 4 threads was faster, 2 281 against 2 099 (`results/diamond_engine_cg19_2026-11-04.txt`, section 3): that model did not fill the GPU, and whether a small model loses on the GPU also depends on the CPU's rate there (on cg29 the GPU led on the GCPW line at 0.11 M cells, below). The six models' rates on this engine at cg29, next to stock and cg29 on the CPU, are in Table 10, which runs the SMA launch on its 1.071 M-cell mesh; their rates at cg31 are in Table 18.

Where the GPU overtakes the CPU. There is no single size at which the GPU becomes the faster engine; it depends on the model. Three models were run at several mesh scales on cg29 (Figure 3; `results/gpu_crossover_cg29_2026-11-05.txt`): the patch antenna of the patch tutorial from 0.10 to 7.9 M cells (scale 1 is the patch tutorial, scale 5 patch x5), the notch filter of the notch tutorial from 0.27 to 5.2 M (scale 3 is notch x3) and the GCPW line from 0.11 to 5.1 M, 25 points, each in 4 paired, interleaved rounds of the multithreaded engine at 8 and 4 threads and the GPU engine with and without its timing (others' CPU 0.5–1.3 cores and GPU 1–5 % in the median of a point; every run waited, for up to 30 minutes, until the others used at most 1.5 cores and 10 % of the GPU, and was repeated up to three times when they used more; a round still loaded after that was left out where two clean rounds remained, so 9 points rest on 3 rounds and 2 on 2). The multithreaded engine was fastest at 8 threads at every point. The GPU was ahead in every round from 1.7 M cells on the notch filter, from 2.4 M on the GCPW line and from 2.9 M on the patch antenna, by 1.10–1.52× from there and by 1.25–1.52× from 3.1 M cells on; the medians cross for the last time at about 0.66, 1.45 and 1.91 M cells. Below that the two engines were within 0.89–1.23× of each other and the side depended on the model: the GPU led on the GCPW line at 0.11–0.42 M cells (1.03–1.23×),

trailed on the notch filter (0.89× at 0.27 M, 0.97× at 0.50 M) and changed sides from size to size on the patch antenna (0.94–1.11× up to 1.85 M). On grids this small the session counts as well: the patch tutorial, 1.107× [1.09–1.17] in the sweep, ran 0.95–0.97× in two later runs of the same 36 500 steps on a quieter Mac and 0.91–0.93× at the bench's 10 500 steps, with the CPU 8 % faster than in the sweep and the GPU 6 % slower (section 5 of the file). Every CPU run gave the CPU reference's probes; the GPU's differed from them by at most 1.7×10^{-6} relative (no FTZ in either), except at one point where the wave had not yet reached a port and the CPU's values there were subnormal, which the GPU flushes to zero.

Three things narrow the GPU's lead on smaller grids, or reverse it. *It waits for the CPU.* Between `IterateTS` calls the CPU records the probes and evaluates the dumps while the GPU idles, and starting and finishing a call's command buffers leaves it idle too: 0.3–0.5 ms per call on the smallest grids. On the patch tutorial a call ends every 31 steps, at the nf2ff dumps, and holds 1.3 ms of GPU work, so the GPU was busy 73–82 % of the time-stepping loop; at 0.62 M cells 94 %, from 4.2 M 98–99 %. On the patch antenna, up to 0.62 M cells, its rate over busy time alone, 2 413–2 898 MC/s, was above the CPU's at every point (1 766–2 825): on those grids the waiting decides. The notch filter and the GCPW line end a call every 81 to 750 steps and kept the GPU busy 97–99 % at every size, the notch tutorial included. *Its own rate is lower on small grids:* over busy time 1 959 MC/s on the notch filter at 0.27 M cells against 3 239 at 5.2 M, and 1 799–1 956 on the GCPW line at 0.11–0.16 M against 3 053 at 5.1 M. A dispatch there has few tiles for the 10 GPU cores (75–96 tiles of 256 threads on the notch filter at 0.27 M, 18–30 on the GCPW line at 0.11 M) and short columns (4 z-vectors), and the edge work makes up more of the cells: the 8-cell PML and the Mur faces on the notch filter, the conducting sheets and the Mur faces on the GCPW line. These causes were not separated; the patch tutorial, with 16–25 tiles and 12 z-vectors, ran 2 420–2 510 MC/s over busy time. *The CPU is at its best in the middle of the range:* 1 573–2 208 MC/s at 0.10–0.27 M cells, 2 570–3 003 at 0.62–1.85 M, above the 2 273 MC/s the cores reach from DRAM (Section 5.4), presumably because the caches then hold part of the traffic (not measured), and 1 919–2 263 at 4.2–7.9 M, while the GPU ran 2 818–3 220 there. The GPU's lead is largest where the CPU is held to the bus. Refined to 3.2 and 5.1 M cells, the GCPW line ran 2 933 and 3 028 MC/s on the GPU, between patch x5 (2 908) and notch x3 (3 220): the board's lower rate at its own size, 2 099 at 0.24 M cells, comes mostly from that size and what comes with it (fewer tiles, shorter columns, a larger share of sheet and Mur cells; not separated).

Tiles and K. No one tiling is best for every model. In a sweep on cg23 (the faster of two runs, others' CPU 0.4–2 cores, GPU up to 15 %) the default $K = 4$ with 8:8 tiles was best or within 8 % on the four models built from the tutorials and 8–9 % below the best setting on the two boards. In paired runs against the default the SMA launch gained 1.132× [1.107–1.144] with $K = 2$ and 6:4 tiles (cg24) and the GCPW line 1.114–1.117× per round with $K = 4$ and 12:8 tiles (cg23; `results/diamond_engine_cg24_cg27_2026-11-04.txt`, section 8). The run-time tuner of cg28 (above) chooses among four tilings. In the cg23 sweep four of the six models ran best with one of them, and the SMA launch and the patch tutorial with a $K = 2$ tiling, 0.3 and 1.2 % faster than the best of the four; in a K sweep on cg27 (the faster of two runs, not paired; `results/fused_sweep_k1_2026-11-04.txt`, part B) the default $K = 4$ with 8:8 tiles beat $K = 2$ with 4:4 tiles and every $K = 1$ tiling on all six models. $K = 2$ with 6:4 tiles, the SMA launch's best in the paired runs of cg24 (against 1.065× for $K = 3$ with 8:6 tiles), is not among the four, nor among the seven of cg31 (Section 9), and was not timed after cg24 up to cg29; the record of the sweep of thirteen tilings of 2026-11-11 does not list its tilings (`results/diamond_engine_cg28_tuner_2026-11-04.txt`, header; `results/metal_engine_cg31_2026-11-11.txt`). The earlier fixed 1 024 threads per tile cost 44 % on the notch tutorial and 21 % on the SMA launch against the rule above (loaded Mac; `results/diamond_engine_cg21_2026-11-04.txt`, section 4).

From cg24 to cg27. The operator setup of cg24 runs on all cores: before it, the diamond engine set up notch x3 in 5.55 s and patch x5 in 6.48 s, now in 2.74 and 1.69 s, against 2.61 and 1.60 for the multithreaded engine (loaded Mac; `results/diamond_engine_cg24_cg27_2026-11-04.txt`, section 5). A diagnostic build that leaves parts of the work out (results not identical by design; 3 rounds) showed where the boards' time went: the SMA launch ran 1.343× faster without the UPML, 1.257× with every cell on one operator and 1.167× without the excitation, the GCPW line 1.514× without the sheets and 1.283× without Mur (section 7). The excitation and the sheets became runs (cg25), the coefficients split tables (cg27); in one build, split against packed gave 1.053× [1.03–1.07] on the SMA launch and 1.031× [1.03–1.04] on the GCPW line (8 rounds, others 4–5 cores). Paired in one session against cg24 (5 rounds; others 4–5.6 cores, load average 11.7–25.2), cg27 ran the SMA launch 1.265× [1.22–1.28] faster (median 1 184 to 1 482 MC/s) and the GCPW line 1.303× [1.29–1.37] (1 361 to 1 773). The notch tutorial, notch x3 and patch x5 stayed within noise (0.983–1.012×); the patch tutorial's 1.251× [0.93–1.47] rests on 343 short calls under that load and is noise (section 3). Folding the UPML stages into the cell update gave nothing (paired against cg24, 4 rounds: SMA launch 1.023× [0.934–1.106], notch tutorial 0.948×, notch x3 0.988×) and was removed (section 1).

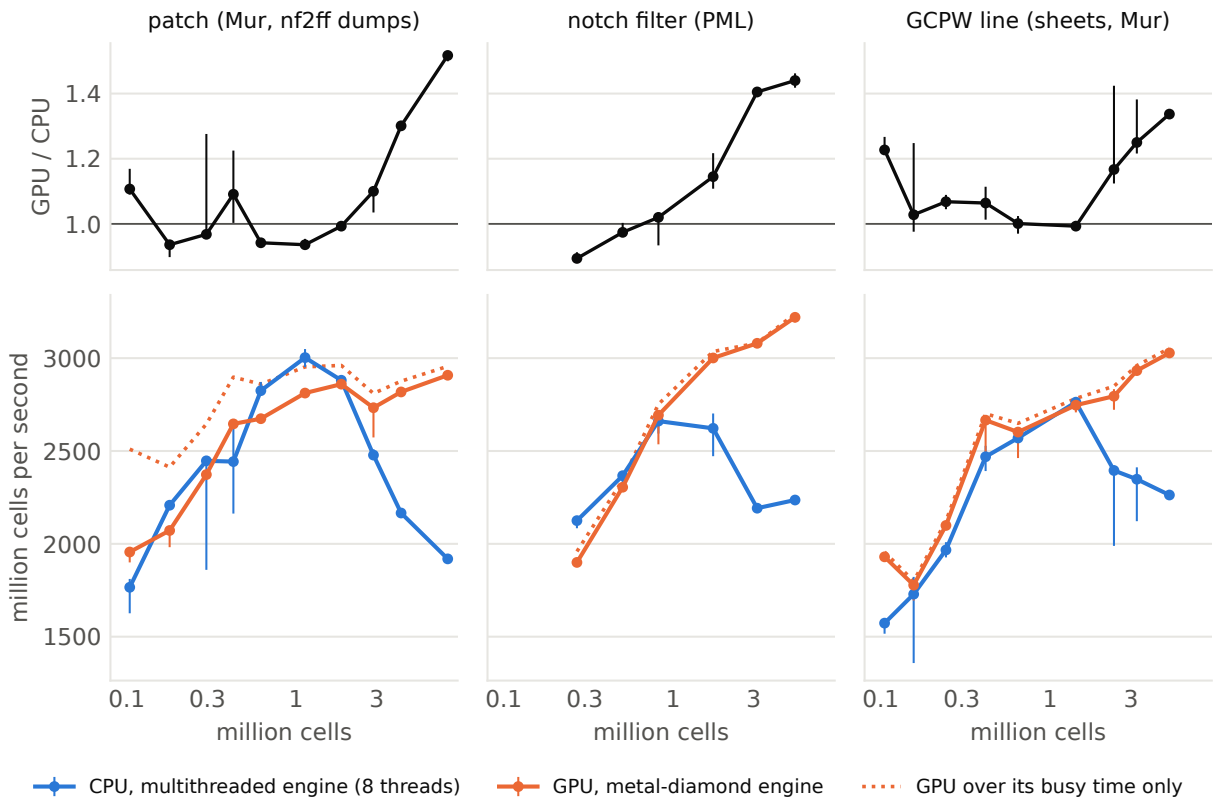


Figure 3: GPU against CPU by grid size on cg29 (`results/gpu_crossover_cg29_2026-11-05.txt`). Top: the GPU engine's speed over the multithreaded engine's at 8 threads, median of the paired rounds used and their range (4 rounds per point; 9 points rest on 3 and 2 on 2, see the text). Bottom: the medians in MC/s with the range over the same rounds; the dotted line is the GPU's rate over its busy time alone (`OPENEMS_DIAMOND_TIMING=1`), so the gap down to the GPU's own line is time it waited for the CPU. The patch and notch meshes are refined by the bench's scale factor (the tutorials at scale 1, patch x5 and notch x3 at scales 5 and 3); the GCPW line's scale multiplies its cell size (scale 1 is the 0.242 M-cell line of the tables, scales 2 to 0.3 give 0.105 to 5.1 M cells); logarithmic cell axis.

Where the GPU waits. With `OPENEMS_DIAMOND_TIMING` the engine reports its GPU time. On cg24 the GPU was busy 96–99 % of the time-stepping loop on the two boards and notch x3 and 91 % on patch x5, but 78 % on the patch tutorial and 76 % on the notch tutorial, whose `IterateTS` calls are short (31 steps between the patch tutorial's nf2ff dumps, 81 on the notch tutorial) and leave the GPU idle while the CPU works between them. On cg29, in the size sweep above, the patch tutorial kept the GPU busy 79–82 % of the loop (73–79 % in later runs), but the notch tutorial 97 % with the same 81-step calls: its cg24 figure, from a loaded Mac, was not reproduced. Timed per phase, notch x3 spent 12/17/18/53 % of the GPU time in the four kinds of tile, against a work split of 10/18/18/54 % at $M = V = 8$, $K = 4$: per cell update the four cost the same (`results/diamond_engine_cg24_cg27_2026-11-04.txt`, section 6).

The run-time tuner (cg28). Table 5(a) gives the paired comparison with cg27 on a quiet Mac: 1.094× on the SMA launch, 1.114× on the GCPW line, 1.061× on notch x3 and 1.064× on the patch tutorial. On the notch tutorial and patch x5 the default is the best tiling; there the tuning, 575 and 623 steps with the other tilings in turn, costs 0.4–0.5 % of these short runs, and less in real runs of 34 000 to 190 000 steps. The choices were those of the best fixed tiling in the paired sweeps and the same in every repeat (`results/diamond_engine_cg28_tuner_2026-11-04.txt`, sections 2 and 3).

Fused but not blocked ($K = 1$). At $K = 1$ a threadgroup updates the voltages and then the currents of its tile in one pass, and the four kinds of tile complete the tile edges, so each field value is read and written once per step: the 49 B per cell of the CPU's fused sweep, the case that Section 5.4 bounds at 2 470–2 590 MC/s. Measured on cg27 (Table 5(b); paired, 4 rounds, others' CPU 2.5–4 cores, GPU 3–5 %), it ran notch x3 at 1 716–1 733 MC/s and patch x5 at 1 530–1 844 over three tile sizes, 0.54–0.65 times $K = 4$. At each model's best tile that is 1.25–1.30 times the base's unfused GPU engine (ratio of the medians), 67–75 % of the bound, and 85–90 GB/s at 49 B per cell, where the streaming kernels drew 121–127 GB/s; against the CPU best of cg18 on a quiet Mac, a comparison across sessions, it is 0.85 (notch x3) and 1.07 times (patch x5) (`results/fused_sweep_k1_2026-11-04.txt`). The per-step structure (four dispatches, the tile edges, a threadgroup barrier between the two half steps) costs more than the bytes it saves; the bound is a traffic bound only. Blocking over time steps does what fusing alone does not: $K = 2$ (2 123 and 2 408 MC/s) is still below the bound, $K = 4$ (2 986 and 2 817) above its upper end on both models. A one-dispatch fused kernel was not built: tiles that compute their own edges would overwrite values their neighbours still read in the same dispatch, so it needs a second copy of both fields, and every extension would have to follow.

End to end. Through the author's toolkit (cg23; one run each, another session's job on one core, load average 2.6–8.0) the engine cut the toolkit's default runs (probe end −60 dB, hold 0.5) from 7.43 to 5.95 s on the GCPW line (1.25×) and from 43.71 to 34.23 s on the SMA launch at 0.871 M cells (1.28×), and the full runs to `max_time` by 1.15× and 1.26× (`FDTD.Run` wall time), against the multithreaded engine of the same build with its automatic thread count (`results/rfsim_e2e_gpu_2026-11-04.txt`). On cg27, under a heavier load, the GPU runs gave the same port data (`results/diamond_engine_cg24_cg27_2026-11-04.txt`, section 9). The end-to-end runs of cg29 are in Section 8.

6.4 Limits and open questions

- **One machine.** Everything here ran on the base M5 with its 10 GPU cores, whose bus the CPU shares. The M5 Pro (307 GB/s [30]) was measured later, with cg31, as a set of its own (Section 10); the M5 Max and the M5 Ultra (1.2 TB/s [31]) were not measured; for luikore's engine, 1.3–4.2× its own unfused path is reported on an M4 Pro and an M5 Max [20].
- **No head-to-head with luikore's engine on the six models.** On this M5 its diamond path refused all six models (Mur faces; on the GCPW line also the conducting sheets) and ran only variants with every Mur face replaced by PEC, on which this engine was not run (`results/others_2026-11-04/`, Section 5.4). Since 2026-11-11 there is one on luikore's own benchmark model, a uniform grid with PEC walls: cg31's engine ran 3 981 and 3 309 MC/s at 3.0 and 17.0 M cells, luikore's 3 952 and 2 829 (Section 9.4, Table 19). Its diamond path still refuses the six models.
- **No bytes per cell.** The traffic model of Section 4.5 gives the bytes of a sweep that streams every array once or twice per step. With K steps per tile the traffic depends on what the caches keep, and it was not measured with hardware counters, so this engine has no bytes-per-cell figure and no bar in Figure 2. Its rates above the bound show only that it moves less than 49 B per cell update. Section 9 adds a count from the tile schedule, 20–29 B of field per cell and step for a tile that stays cached through its block; it is a count, not a measurement.
- **GPU memory** was not measured: the shared field buffers, the four tilings' buffers while tuning (seven tilings since cg31), the extension buffers. Only the coefficient tables are known (above).

(a) cg28 (tiling chosen at run time) against cg27, paired, 5 rounds; quiet Mac (others' CPU 0.5–1.3 cores, GPU 2–4 %)

model (cells, steps)	cg27	cg28	cg28 / cg27	`TUNE=0` / cg27	tiling chosen	tuning steps
SMA launch (0.871 M, 6000)	1671	1828	1.094 [1.092–1.097]	1.001 [0.999–1.003]	$K = 3$, 8:6	584
GCPW line (0.242 M, 15000)	1896	2113	1.114 [1.068–1.117]	0.999 [0.963–1.003]	$K = 4$, 12:8	588
notch x3 (5.247 M, 3000)	3039	3226	1.061 [1.061–1.064]	0.998 [0.996–1.000]	$K = 4$, 12:8	614
patch tutorial (0.104 M, 10500)	1640	1764	1.064 [1.010–1.152]	1.000 [0.942–1.042]	$K = 3$, 6:6	532
notch tutorial (0.270 M, 16000)	1908	1902	0.996 [0.994–1.001]	0.998 [0.994–1.002]	$K = 4$, 8:8	575
patch x5 (7.863 M, 2400)	2930	2917	0.995 [0.994–0.997]	1.003 [1.002–1.005]	$K = 4$, 8:8	623

(b) The fused, unblocked sweep ($K = 1$) on cg27, paired, 4 rounds (others' CPU 2.5–4 cores, GPU 3–5 %); MC/s (ratio to $K = 4$)

model (cells)	base GPU engine (unfused)	$K = 1$, 8:8	$K = 1$, 16:8	$K = 1$, 32:16	$K = 2$, 4:4	$K = 4$, 8:8
notch x3 (5.247 M)	1330 (0.446)	1716 (0.573)	1729 (0.579)	1733 (0.579)	2123 (0.710)	2986
patch x5 (7.863 M)	1469 (0.515)	1799 (0.637)	1844 (0.649)	1530 (0.541)	2408 (0.852)	2817

Table 5: Speed of the Metal diamond engine, MC/s (medians of the rounds; a ratio is the median of the per-round ratios, with its range). Tiles are M:V. (a) `results/diamond_engine_cg28_tuner_2026-11-04.txt`, section 3; the tiling chosen and the steps the tuning took come from one separate run per model (section 2; others about 0.5–1 core), and the choice was the same in every repeat. `TUNE=0` is cg28 with `OPENEMS_DIAMOND_TUNE=0`, the fixed $K = 4$, 8:8 of cg27. (b) `results/fused_sweep_k1_2026-11-04.txt`, part A. The base GPU engine is `--engine=gpu` of the fork's base (7bb294f), not upstream master's merged engine. The traffic bound of a fused sweep at 49 B per cell is about 2 470–2 590 MC/s (Section 5.4). Every run of both sessions equals its session's first run, and so the multithreaded engine with `OPENEMS_FTZ=1`.

- **Graded meshes and extension work.** The boards run below the large uniform models, for their UPML, sheet and excitation work and their many distinct operators (the ablation above), and for their size: refined to 3.2 and 5.1 M cells, the GCPW line ran between patch x5 and notch x3 (Figure 3). The SMA launch, 1 829 MC/s at 1.07 M cells in Table 10 against 2 747–2 812 for the GCPW line and the patch antenna at 1.1–1.4 M cells in the size sweep (another session), was not scaled. The tuner picks among four tilings only. Since cg31 (Section 9) a block takes three dispatches, the UPML and the dispersive stages take a z-vector or the three components of a cell at a time, Mur faces of one axis share a barrier and the tuner has seven tilings: the GCPW line and the SMA launch (0.871 M cells) gained 1.09× and 1.14× (Table 18). The extensions still cost, and the cost is tied to their code being compiled into the kernel, not to the loops over their cells; that is not explained.
- **Coverage.** TFSF, steady-state detection and subclasses of the dispersive extensions put a model on the one-thread CPU loops; probes and dumps run on the CPU between calls, which, with the start and finish of each call's command buffers, left the GPU idle for about a quarter of the loop on the two tutorials at cg24, and at cg29 still for 18–27 % on the patch tutorial (3 % on the notch tutorial; Figure 3). Up to cg29 the engine also waited for the base's energy estimate, one CPU thread over both fields every Nyquist period of a run's first 60 s; since cg31 a run without an end criteria makes none (Section 9).
- **fp16 field storage** was not tried: late-run currents of 10^{-6} A lie below the half-precision normal range, which ends at 6×10^{-5} (project log), so the fields would need scaling, and the result could not be bit-identical.
- **Load.** The speeds of cg19 to cg27 and of the $K = 1$ runs come from a loaded Mac (each with its load above). Of the GPU-against-GPU comparisons of this section only cg28's is quiet (that of cg31 with cg29, Table 18, ran with the other sessions paused); the GPU against the CPU engine and stock is Table 10, and against the CPU engine by grid size the size sweep, which waited for a quiet Mac before every run; its ratios on grids below 0.3 M cells still moved by up to 16 % between sessions.

7 The modelling side: shorter runs for the same answer

Once the engine is at the bus, a run's wall time is cells × time steps ÷ speed, and the first two are set by the model and by openEMS's time-step criterion. Three changes act there; all live in the author's simulation toolkit (rfsim) except the end criterion, which is in openEMS. The changes made to the toolkit in this project are not published, since the toolkit itself is private; this section and `CHANGELOG.md` describe them.

7.1 A run-end criterion on the port probes

openEMS stops a run when the total field energy has fallen below a threshold relative to its peak (default −50 dB) or at a maximum number of steps or time. On both boards the energy criterion never fires: a Gaussian pulse has a DC component, the open structure stores it as a static charge, and the energy stays far above the threshold. At the end of the full runs of Table 6 it still reads −11.1 dB (SMA) and −14.9 dB (GCPW) of its peak (`results/rfsim_e2e_2026-11-03.txt`), while every port signal is below −80 dB of its peak after about a quarter of the run. Runs therefore always went to `max_time`, and the toolkit proved settling after the fact from the port files. Upstream master has since added an opt-in DC-free Gaussian pulse (commit d686f1d, 2026-10-24: `SetGaussExcite(f0, fc, dc_free=True)`, a sine instead of a cosine carrier); it is not in v0.37.0 or in the fork's base. It leaves no static charge, so the energy criterion can fire, but it changes the excitation: its spectrum falls to zero at DC, so the bottom of the band (50 MHz for these boards) is driven more weakly. The criterion below keeps the model and its pulse as they are. A baseband pulse of half the length was tested with it on 2026-11-11 and not adopted (Section 9.7).

The fork adds a second criterion, off unless `OPENEMS_PROBE_END_DB` is set to a negative level in dB; an invalid value of it or of `OPENEMS_PROBE_END_HOLD` prints a warning. Every integral probe (port voltages and currents, field probes) tracks the largest absolute value it has seen and its latest value; a waveguide (mode-matching) probe counts with its mode amplitude only, not its mode purity. At every step at which one of the processings runs (a probe or dump sample, or an energy estimate; on the GCPW line the probes sample every 490 steps, a quarter of the Nyquist interval), the engine computes the worst ratio latest/peak over all probes, so a run can only stop on such a step. It stops once that ratio has stayed below the threshold for a hold time of `OPENEMS_PROBE_END_HOLD` excitation lengths (default 1). The hold starts again at every check at which a probe is above the threshold, a probe has seen no signal yet (its peak is still exactly zero), or the excitation is still running; the excitation counts as running until the end of its signal plus the largest excitation delay, taken over every grid (the child grids of a cylindrical multigrid included) and including the propagation delay of a total-field/scattered-field (TFSF) source. A probe that never sees a signal holds the run to its end; once all the others have settled, a one-time notice names it. A far probe can pick up tiny numerical precursors shortly before the pulse arrives, and those count as a signal, so the hold should be longer than the longest one-way delay from the excitation to a probe. A probe whose own peak is below 10^{-6} of the largest peak of its kind (a current probe on an open end, say) is measured against 10^{-6} of that largest peak instead of its own near-zero peak. The criterion needs a Gaussian excitation and at least one probe; if either is missing it prints a warning and ignores the setting. The time-domain probe and port files of a stopped run are a bit-identical prefix of the full run's (checked for every stopped run of Table 6; only the stop point changes); frequency-domain dumps and nf2ff data are sums over the shorter record and change accordingly. The toolkit sets the variables around every `FDTD.Run()`: by default `probe_end_db=-60` and, since toolkit commit 4a7e20e of 2026-11-04, `probe_end_hold=0.5` (`RFSIM_PROBE_END_HOLD=1` restores the earlier default of 1). Table 6 shows the effect on both boards.

model	setting	steps	time	change vs the full run
GCPW line, 12 ns	full run (max_time)	94 217	13.7 s	–
	−60 dB, hold 1	45 570	8.6 s	S_{21} 8.0×10^{-4} dB, S_{11} 1.3×10^{-4} lin., Z_0 −0.0003 Ω
	−80 dB, hold 1	47 040	7.1 s	S_{21} 7.9×10^{-4} dB, S_{11} 1.2×10^{-4} lin., Z_0 −0.0009 Ω
	−60 dB, hold 0.5	34 300	5.2 s	S_{21} 7.1×10^{-4} dB, S_{11} 2.2×10^{-4} lin., Z_0 −0.0005 Ω
SMA launch, 10 ns	full run (max_time)	182 622	168.5 s	–
	−60 dB, hold 1	103 831	97.6 s	S_{11} 0.0089 dB at −22.8 dB, S_{21} 2.3×10^{-4} dB, Z_0 −0.0066 Ω
	−80 dB, hold 1	106 113	99.8 s	S_{11} 0.0100 dB, S_{21} 1.9×10^{-4} dB, Z_0 −0.0037 Ω
	−60 dB, hold 0.5	77 588	72.9 s	S_{11} 0.0188 dB, S_{21} 2.2×10^{-4} dB, Z_0 −0.0092 Ω

Table 6: Probe end criterion on the two boards: cg18, automatic thread count, one run each, measured on 2026-11-03 (`results/rfsim_e2e_2026-11-03.txt`); the SMA launch on the 1.071 M-cell mesh of the toolkit's former 0.02 mm floor (Δt 54.76 fs). On the GCPW line the −60 dB, hold 0.5 row is the toolkit's default run since 2026-11-04; on the SMA launch that default also has the 0.05 mm mesh floor (Table 7). Time: wall time of `FDTD.Run` (operator setup and time stepping). Changes against the full run of the same board, the largest over 50 MHz–2 GHz: $|\Delta S_{21}|$ in dB; $|\Delta S_{11}|$ linear on the GCPW line (its S_{11} stays below −40 dB) and in dB where $|S_{11}| > -40$ dB on the SMA launch; ΔZ_0 is the change of $\mathrm{Re}\, Z_0$ at 640 MHz (GCPW) and 1 GHz (SMA). In the GCPW's −60 dB, hold 1 run the tuner picked 8 threads (1 304 MC/s, against 1 636–1 683 MC/s at 4 threads in the other fdtdKit GCPW runs), so it took longer than the −80 dB run, which has more steps.

7.2 z lines in multiples of four

The SSE engines pack four z lines per vector and compute $4n_v$ slots per (x, y) column ($n_v = \lceil N_z/4 \rceil$ z-vectors; the spare slots are the top lane of the last z-vectors), so a board with 17 z lines costs the time of 20. The toolkit can now halve the largest z cell (the air above the board) until the line count is a multiple of four; it never splits the boundary cells at z max (one for Mur, N for PML_N) and never shrinks the time step. The GCPW line goes from 17 to 20 lines and from 0.242 to 0.285 million cells in the *same* wall time (bench: 1.13 s versus 1.10 s for 8 000 steps at 4 threads), with the time step unchanged. It is a finer mesh, so results move, by 5×10^{-5} relative in the probes and 10^{-4} dB in S_{21}; it is opt-in. These numbers are from the project log, not in `results/`, and were taken with the first version of the padding, which also split the top boundary cell; the toolkit's current version keeps that cell, so its mesh differs at the top (the line count, and with it the timing, is the same). Rounding down instead, by dropping the top-most air line (17 to 16 lines), would save one z-vector of run time, but it lowers the lid by one air cell: a change of the model rather than of its mesh, left to the toolkit's user.

7.3 A minimum cell size for the SMA launch

The SMA launch has 69 500 distinct operators and, on the toolkit's original mesh (that of 2026-10-17), a time step of 53.0 fs set by cells of 0.025–0.045 mm (operator count and cell sizes from the project log, the time step from `results/sma_mesh_sweep_2026-11-01.txt`). Those cells are not design intent: they are where a tangent line of the pin, the PTFE bore or a via edge lands 0.025–0.045 mm from a line of the regular 0.075 or 0.15 mm grids. The builder merged lines closer than 0.02 mm by keeping the first; it now takes a `min_cell` floor and merges with priorities: strip and gap lines (the port's 1/3–2/3 edge rule) always stay, a geometry line wins over a grid line, and two geometry lines meet in the middle so the error is split. The first core cell under the top plane also starts at the floor.

A first version of this merge was swept on 2026-11-01 (engine cg16, against the full run on the original mesh; the first table of `results/sma_mesh_sweep_2026-11-01.txt`). At 0.06 mm the four-cell prepreg lost two of its cells and S_{11} moved by up to 3.3 dB (−19.98 against −22.80 dB at 1 GHz); the last row of Table 7 keeps that run as history. The pre-publication review of this change then found two bugs in that first merge. The floor was not multiplied by the model's `scale` factor, so the 0.6 convergence mesh was merged with the unscaled floor and the coarse/fine pair was no longer a convergence pair; and the merge scanned the lines from $-y$ to $+y$, so chains of three or more close lines resolved differently on the two sides and a mirror-symmetric structure got an asymmetric mesh at every floor above 0.02 mm. The merge is now independent of the scan order (`core.merge_lines`): protected lines never move and are never merged; a pair or a short run of geometry lines closer than the floor is replaced by its mean, and a longer run (a grid finer than the floor) is thinned to evenly spaced lines at least the floor apart; grid lines give way; and the builder refuses a floor that is not below every fixed cell of the stack-up or the strip rule (0.0526 mm and above for this board, 0.06 mm included).

The 0.02 mm mesh, the toolkit's default until 2026-11-04, is the same line for line with both merges; against the original mesh it differs in the 14 lines that the caption of Table 7 lists, and its time step is 54.76 fs instead of 53.0 fs. With the corrected merge the 0.04 and 0.05 mm meshes gained lines (0.953 and 0.871 million cells; the first merge had dropped lines asymmetrically). A sweep of these meshes on 2026-11-02 ran while other simulations loaded the machine, so its run times could only be estimated. Figure 4 and Table 7 give the sweep measured on 2026-11-03 with cg18 and the corrected merge, against the full run on the 0.02 mm mesh; it reproduces the step counts of 2026-11-02 exactly and replaces the estimated times. The suggestion stands: 0.05 mm with a hold of 0.5 needs 2.54 times fewer cell-steps and 2.61 times less run time than the earlier defaults (the 0.02 mm mesh with a hold of 1; 37.4 s against 97.6 s), with S_{11} within 0.12 dB at −22.8 dB and Z_0 0.12 Ω higher; with a hold of 1 the floor alone gives 1.90 and 1.97 times (49.5 s). That error is far inside the 30 dB return-loss bench floor the model documents for itself: an error wave 30 dB below the incident one, on a reflection of −22.8 dB, allows any reading from −19.7 to −27.8 dB. The author adopted it on 2026-11-04: since toolkit commit 4a7e20e the toolkit's defaults are the 0.05 mm floor (0.871 million cells on this board) and a hold of 0.5 for every run, and `RFSIM_MIN_CELL=0.02` and `RFSIM_PROBE_END_HOLD=1` restore the old ones.

floor	cells	Δt	steps	run	S_{11} at 1 GHz	max $\lvert\Delta S_{11}\rvert$ / $\lvert\Delta S_{21}\rvert$	ΔZ_0
0.02 mm†	1.071 M	54.8 fs	103 831	97.6 s	−22.80 dB	0.0089 / 0.0002 dB	−0.007 Ω
0.03 mm	1.007 M	58.5 fs	97 097	85.7 s	−22.79 dB	0.0292 / 0.0006 dB	+0.03 Ω
0.04 mm	0.953 M	78.0 fs	72 891	58.0 s	−22.72 dB	0.0782 / 0.0014 dB	+0.12 Ω
0.05 mm	0.871 M	84.4 fs	67 340	49.5 s	−22.69 dB	0.1156 / 0.0021 dB	+0.10 Ω
0.02 mm, hold 0.5	1.071 M	54.8 fs	77 588	72.9 s	−22.80 dB	0.0188 / 0.0002 dB	−0.009 Ω
0.05 mm, hold 0.5*	0.871 M	84.4 fs	50 320	37.4 s	−22.69 dB	0.1157 / 0.0021 dB	+0.12 Ω
history: first merge, 2026-11-01 (engine cg16; see the caption)							
0.06 mm	0.625 M	130.1 fs	43 680	24.3 s	−19.98 dB	3.292 / 0.0237 dB	+0.43 Ω

Table 7: SMA launch mesh floor sweep, 10 ns, probe end −60 dB with a hold of 1 unless stated, measured on 2026-11-03 with cg18 (the tuner picked 8 threads in every run), one run each, with the corrected merge (the 2026-11-03 section of `results/sma_mesh_sweep_2026-11-01.txt`; the same runs as in `results/rfsim_e2e_2026-11-03.txt`). Run: wall time of `FDTD.Run`. Changes against the full run on the 0.02 mm mesh (182 622 steps of 54.76 fs, Table 6): ΔS_{11} in dB where $|S_{11}| > -40$ dB over 50 MHz–2 GHz, ΔS_{21} the largest over that band, ΔZ_0 the change of $\mathrm{Re}\, Z_0$ at 1 GHz (to 0.01 Ω where it exceeds 0.01 Ω). †The toolkit's default until 2026-11-04 (with a hold of 1). *The toolkit's default since 2026-11-04 (toolkit commit 4a7e20e). The first row's mesh differs from the toolkit's original mesh (that of 2026-10-17: 188 697 steps of 53.0 fs for 10 ns) in 14 lines: nine move by at most 0.01 mm (four in x, five in y) and five y lines between 3.6 and 4.45 mm by up to 0.10 mm, where the original y mesh was not mirror-symmetric. The last row is history: the 0.06 mm run of the first merge (2026-11-01, against the full run on the original mesh; the first table of the same file), in which the four-cell prepreg lost two of its cells; the builder refuses that floor now.

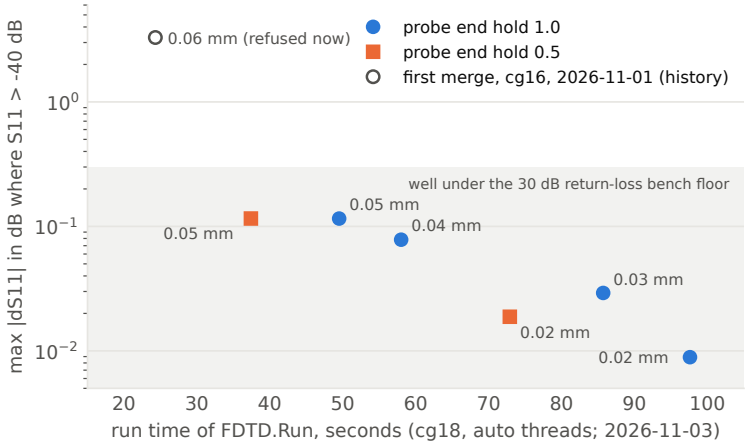


Figure 4: Run time against the S_{11} error for the mesh floors of Table 7, measured on 2026-11-03 with cg18 and the corrected merge. The hollow grey point is history: the 0.06 mm run of the first merge (2026-11-01, engine cg16, against the full run on the original mesh), a floor the builder now refuses.

8 Results

8.1 Engine speed

Figure 5 compares stock and snapshot cg14 on the six models: fdtdKit at each model's best thread count, stock at its best for the four tutorial models and at 4 threads for the two boards, whose stock thread count was not swept. Figure 6 shows how the speed-up accumulated snapshot by snapshot (Table 8 names them); Figure 7 and Table 9 give the thread scaling of cg14 and the automatic choice. These engine-speed figures and tables stop at cg14. Snapshot cg18 (patches 0001–0006, the build installed on the author's Mac from 2026-11-04 to 2026-11-06) went through the same full check on a quiet machine on 2026-11-03 (`results/*_cg18.txt`): bit-identical to stock on all six models; its best speeds differ from cg14's by −2.7 to +1.6 % on five models and by +8.2 % on the GCPW line (whose cg13 run had reached 1 745 MC/s); on the four models of up to about a million cells it reaches 1 361–1 961 MC/s, 3.1–6.8× the stock rows of the same tables. The cg17 tables

in `results/` are correctness checks, run while the machine was loaded. Snapshot cg29 is compared with stock on the CPU and on the Metal diamond engine in Section 8.2, and the installed build, cg31, with cg29 in Section 9.

Part of the patch tutorial's factor is not the patches' (`results/*_base.txt`, 2026-11-03, best of two runs on a quiet machine). The fork's base, upstream's `gpu-engine` branch at 7bb294f with stock's CSXCAD and nf2ff libraries, runs that model 1.13× faster than stock (308.5 against 273.4 MC/s, both at their best, 2 threads), all of it on the dump side: with the dumps off both reach 342.2 MC/s at 2 threads. The difference is the base's background HDF5 writer, which writes the model's 12 time-domain dumps off the main thread and keeps their files open (the CPU engines still evaluate the dumps on the main thread; only the base's GPU engine evaluates them from a field snapshot, and the Metal diamond engine of Section 6 leaves them to the CPU between its calls). Snapshot cg18 runs that model 5.98× faster than the base (1 845.9 MC/s at 4 threads), so the patches' own factor there is about 6.0×. On the notch tutorial, the GCPW line and patch x5 the base runs within 0.98–1.03× of stock; the SMA launch was not run with it. Every run in these files has stock's probes and S-parameters.

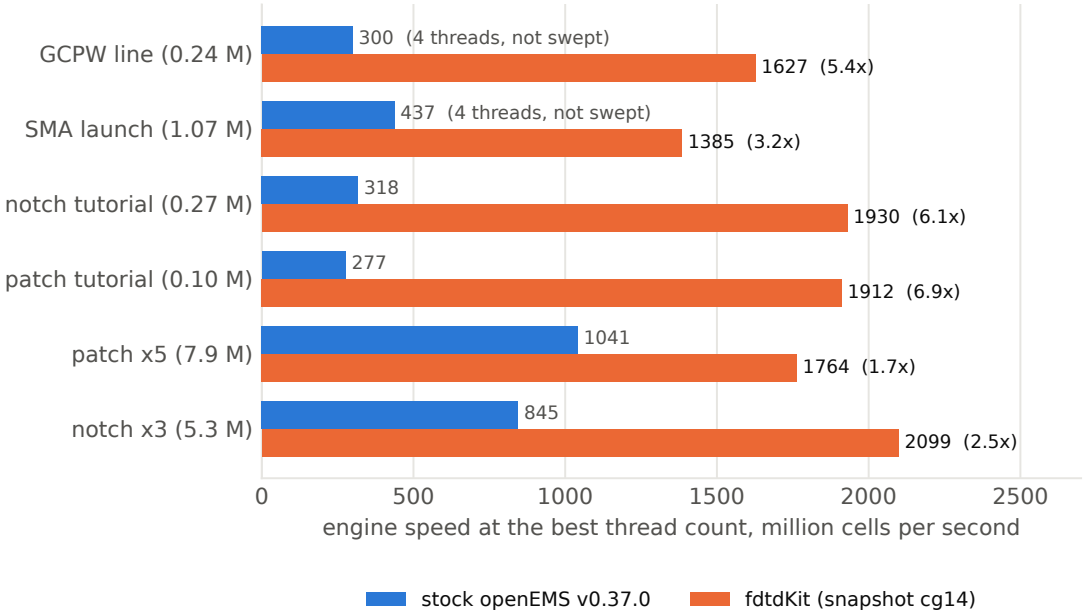


Figure 5: Stock v0.37.0 against fdtdKit snapshot cg14 (`results/*_cg14.txt`), same results bit for bit. fdtdKit at each model's best thread count; stock at its best for the tutorial models (Table 2) and, for the two boards, one 4-thread reference run each (the cg10 run for the GCPW line, the cg14 run for the SMA launch), as their stock thread count was not swept. On the patch tutorial about 1.13× of the factor comes from the fork's base (see the text).

model	stock (thr)	1 thr	4 thr	8 thr	10 thr	auto (picked)
GCPW line (0.24 M)	300 (4)	530	1627	1348	999	1540 (4)
SMA launch (1.07 M)	437 (4)	444	1241	1385	1311	1344 (8)
notch tutorial (0.27 M)	318 (4)	590	1930	1528	1160	1615 (4)
patch tutorial (0.10 M)	277 (2)	690	1912	1545	1550	1499 (–)
patch x5 (7.9 M)	1041 (8)	720	1616	1716	1764	1697 (7)
notch x3 (5.3 M)	845 (8)	648	1841	2099	2038	1962 (8)

Table 9: Snapshot cg14, MC/s; fdtdKit columns from `results/*_cg14.txt` (the run label in those files is `cgy`). Stock: the best of the Table 2 sweep for the tutorials; for the two boards one 4-thread reference run each (cg10 GCPW, cg14 SMA), as their stock thread count was not swept. The patch tutorial's 0.7 s auto run ends before the search prints its choice.

Two patterns of cg14 deserve a sentence. Single-thread speed rose 1.8 to 4.4 times over stock's sse-compressed engine on the four models that have a stock single-thread run (Table 2; the inner loop, the allocation stagger,

snapshot	added
A	spin barriers, hook plan
AB	+ inner loop (IJKN)
ABCDEG	+ work stealing, automatic thread count (0.15 s windows), Mur fast path, UPML on vectors, fused sweep with line hooks for Mur, UPML, excitation and lumped RLC, background HDF5 writer
cg, cg2	rebased on upstream's gpu-engine branch at commit 7bb294f (its gather and asynchronous dumps; the branch has since been deleted upstream), slab balancing; Field_Gather_SSE
cg6	Mur line hooks vectorised in memory order (from cg4; the dump experiments of cg3 and cg5 removed)
cg8	thread tuner timed in pieces of window/8 with slab balancing between them (cg7), refined search
cg9	threaded operator setup, nf2ff matrix product, CSXCAD lookups, operator/field export
cg10	staggered allocations (allocation-placement valley), `--disable-dumps` crash fix
cg11	scalar-SSE UPML path for z-only PML slabs
cg12	line hooks for the dispersive extensions (conducting sheets)
cg13	packed coefficient tables
cg14	compressed UPML coefficient tables
cg15	user-initiated thread QoS
cg16	probe end criterion
cg17	review fixes (patch 0005): Debye hooks, probe end arming, setup cache key, portability guards (Section 4.13)
cg18	final-review fixes (patch 0006): probe end with excitation delays and waveguide probes, packed tables always built, on/off switches, tuner clamp, flush-to-zero state restored, size limits (Section 4.13)
cg19	Metal diamond engine, `--engine=metal-diamond` (0007): cell updates, excitation and Mur on the GPU (Section 6)
cg20	+ lumped RLC elements on the GPU (0008)
cg21	+ UPML on the GPU; extension data in three buffers; threads per tile from the tile size (0009)
cg22	+ Drude/Lorentz materials and conducting sheets on the GPU: all six models run there (0010)
cg23	+ Debye materials on the GPU (0011)
cg24	GPU engine's operator setup on all cores (`Operator_Multithread`); `OPENEMS_DIAMOND_TIMING` (0012)
cg25	excitation and dispersive elements as runs; per-tile offsets read once (0013)
cg26	grid sizes as constants of the kernel; Mur and UPML records passed by value (0014)
cg27	split coefficient tables for graded meshes (0015)
cg28	tiling chosen at run time (0016)
cg29	CPU items (0017–0027, one per topic; Section 4.14): operator index per z-vector, merged ADE for conducting sheets, UPML y-rows, UPML setup on the setup threads (PR #244), line hooks for the steady-state detection and for Debye materials, geometry coefficient tables, `OPENEMS_FUSED_STAT`, neighbour flags instead of the two barriers with RLC slots by time step (Discussion #492), steady-state detection on the flags, no flag spin above the P-core count

Table 8: Snapshots. Each is a complete set of dylibs plus the cumulative patch, kept for bisection on the author's machine. `patches/openems/` publishes the fork as patches 0001–0043: 0001–0006 end at cg18 (0001 = cg8, 0002 = cg9, 0003 = cg10–cg15, 0004 = cg16, 0005 = cg17, 0006 = cg18), the build installed on the author's Mac from 2026-11-04 to 2026-11-06; 0007–0016 are cg19–cg28, one per snapshot (Table 4); 0017–0027 are cg29, one per topic (Table 3), the build installed there from 2026-11-06 to 2026-11-11; 0028–0039 are cg31 (Section 9), the build installed since, 0040 is the opt-in tiled sweep (snapshot cg32d, Section 9.6), and 0041–0043 are the thread-search start, the packed dispersive hooks and their guard, the fork's head 13350ca (snapshots cg33a, cg33c and cg33d, Section 10.1); none of these three groups has a row here. A row lists what the snapshot added over the row above; the intermediate snapshots cg3–cg5 and cg7 appear inside the cg6 and cg8 rows.

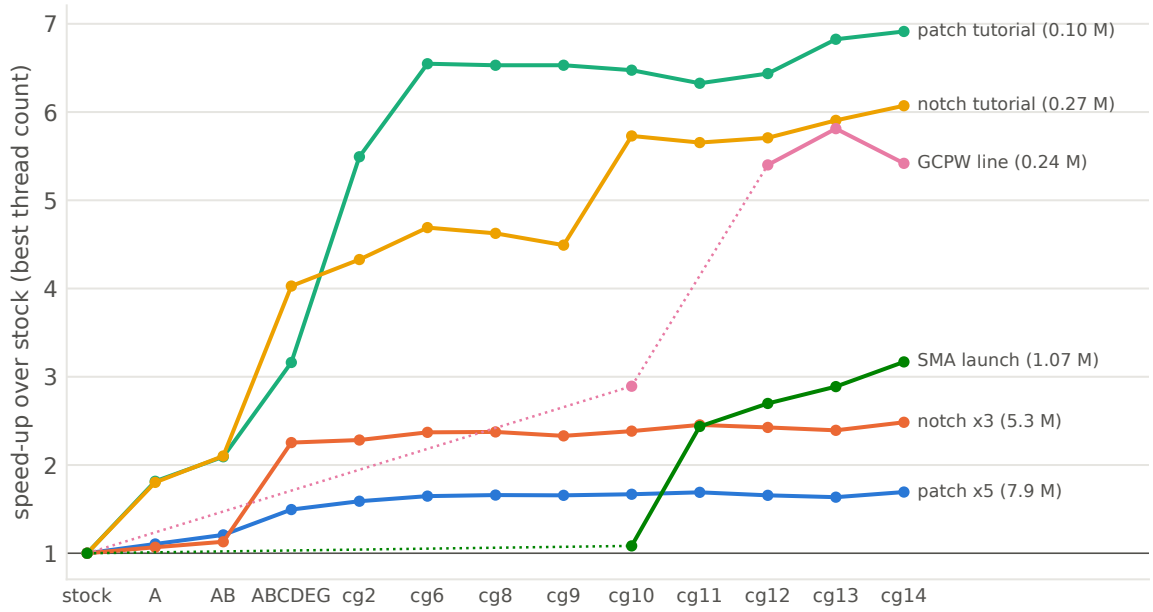


Figure 6: Speed-up over stock per snapshot (the best thread count of each check run; `results/summary.json`, which `paper/figures/parse_results.py` collects from the tables in `results/`). Stock as in Figure 5. The two boards joined the suite at cg10 and the GCPW line was not run at cg11; dotted lines bridge those gaps. The two runs behind each value differ by 1.2 % in the median and by 9 % or more in one pair in ten (399 pairs in the tables of 2026-10-28 and 2026-10-29 in `results/`), so steps of that size are noise: the GCPW's from cg13 to cg14 (1 745 to 1 627 MC/s at 4 threads; the two cg13 runs gave 1 745 and 1 608) is one. The patch x5 point of A (1 151 MC/s, 8 threads) is a run whose load figure is invalid in its table; load can only slow a run, so it is kept.

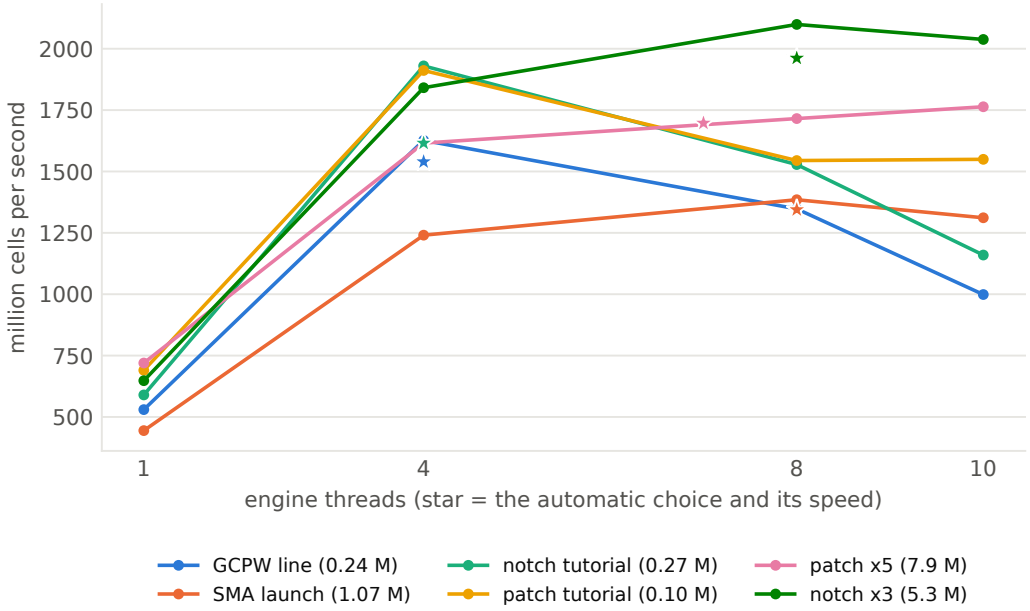


Figure 7: Thread scaling of snapshot cg14 (the check runs of 2026-10-29, `results/*_cg14.txt`). Stars: the automatic choice and the speed it reached, which includes the search of about 1.5 s (project log; none for the patch tutorial: its 0.7 s run ends before the search prints its choice).

the boundary paths); of the patch tutorial's 2.7 times, the fork's base alone gives 1.18 times at one thread (296.6 against 251.3 MC/s, both with the multithreaded engine on one thread; `results/patch_s1_n10500_base.txt`). And the best thread count is 4 for models up to a few hundred thousand cells, where the four P cores saturate what the per-step synchronisation leaves, and 8 to 10 above a million, where the E cores add bandwidth; the automatic choice finds this within 3–7 % on the two boards and the two large models, while on the 0.6–2.7 s tutorial runs the search itself costs 16–22 % (in the cg18 check the GCPW line's 2.4 s automatic run also lost 13 %). In cg29, whose neighbour flags replace the two barriers per time step (Section 4.14), 8 threads was the fastest setting measured on all six models, the smallest included (Section 8.2).

8.2 The build of 2026-11-05 on the CPU and on the GPU (cg29)

Table 10 sets cg29 (patches 0001–0027; the current build until cg31 followed on 2026-11-11, whose rates are in Table 18) against stock v0.37.0 on the six models, on the CPU and on the Metal diamond engine of Section 6, in one paired interleaved run per model on 2026-11-05 (`results/speed_cg29_2026-11-05.txt`; 4 rounds, the step counts of `bench/check.sh`). On the CPU, cg29's best setting of the three measured (4 and 8 threads and the automatic count) is 8 threads on every model, the patch tutorial's 0.104 M cells included; the automatic count ran 1.2–7.5 % below it (the medians of the rows). At 8 threads cg29 runs the four models of up to about a million cells at 1 808–1 961 MC/s, 4.73–6.89× stock at stock's best of 2, 4 and 8 threads, and the two large models at 1 958 (patch x5) and 2 177 MC/s (notch x3), 1.95× and 2.72×. The Metal diamond engine runs the four at 4.83–7.40× stock and the two large models at 2.87× and 3.99×. Against the CPU engine at its best, paired in the same rounds, it is ahead on patch x5 (1.47×), notch x3 (1.46×) and the GCPW line (1.17×), level on the SMA launch (1.00×; 0.99–1.13 over the rounds) and behind on the notch tutorial (0.97×) and the patch tutorial (0.93×). Grid size decides most of this (Figure 3): in a sweep of three models over 0.1–7.9 M cells the GPU led in every round from 1.7–2.9 M cells on and by 1.25–1.52× above 3 M, and below that the two engines were within 0.89–1.23× of each other, the side depending on the model and on the session. On the patch tutorial, whose nf2ff dumps end a GPU call every 31 steps, the GPU stands idle for 18–27 % of the time-stepping loop, while the CPU evaluates the dumps and while each call's command buffers start and finish; on the notch tutorial it is busy 97 % of the time but runs slower on so small a grid (Section 6).

Every CPU run's port probes are identical to stock's at 4 threads. The GPU runs differ from them by 1.7×10^{-7} to 9.4×10^{-7} relative: Apple's GPU flushes subnormal numbers to zero and the CPU keeps them, and with `OPENEMS_FTZ=1` on the CPU the two engines are identical on all six models (`results/*_cg29gpu.txt`; Section 3.3). The SMA launch runs here on the 1.071 M-cell mesh of the earlier tables (`RFSIM_MIN_CELL=0.02`), not on the 0.871 M cells that the toolkit's default has built since 2026-11-04 (Table 1); rates on the two meshes do not compare. A round started once the other processes were quiet and was run again (on the SMA launch, a single run) when they rose above the limit during it (the caption gives the rule); during the runs kept the others used a median of 0.5–0.8 cores per model, at most 1.5.

Two things set Table 10 apart from Table 9: its stock reference ran in the same rounds, at 2, 4 and 8 threads on every model, while Table 9 took stock's rates for the tutorials from the step-1 sweep and one 4-thread run for each board; and its snapshot is cg29, not cg14. Stock's rates also differ between the two sessions (the SMA launch at 4 threads 437 against 379 MC/s, both on the 1.071 M mesh), so the ratio of the two tables' factors is not cg29's gain over cg14. For cg29 against cg18, paired with each other in rounds of their own over the six models (5 rounds, other processes on 0.3–0.6 cores, the SMA launch on its 0.871 M mesh), Section 4.14 gives 1.03–1.23× at 4 threads and 1.13–1.44× at 8 (`results/cpu_items_cg29_2026-11-05.txt`, section 4).

	stock v0.37.0	cg29, CPU (multithreaded)			cg29, GPU	paired ratio		
model (cells)	best (thr)	4 thr	8 thr	auto	diamond	CPU / stock	GPU / stock	GPU / CPU
GCPW line (0.242 M)	286 (4)	1548	**1808**	1689	2113	6.33	7.40	1.17
SMA launch (1.071 M)	379 (4)	1388	**1833**	1799	1829	4.73	4.83	1.00
notch tutorial (0.270 M)	303 (4)	1595	**1961**	1814	1893	6.56	6.26	0.97
patch tutorial (0.104 M)	268 (2)	1829	**1854**	1832	1729	6.89	6.48	0.93
patch x5 (7.86 M)	1003 (8)	1705	**1958**	1889	2879	1.95	2.87	1.47
notch x3 (5.25 M)	800 (8)	1736	**2177**	2097	3193	2.72	3.99	1.46

Table 10: Snapshot cg29 (patches 0001–0027) against stock v0.37.0 on the six models, MC/s, measured on 2026-11-05 (`results/speed_cg29_2026-11-05.txt`). One paired interleaved run per model (`bench/abtest.py`, 4 rounds, the step counts of `bench/check.sh`): every round runs stock at 2, 4 and 8 threads, cg29's multithreaded engine at 4 and 8 threads and with the automatic thread count, and cg29's Metal diamond engine, in turn, once the other processes use at most 1.5 cores and 10 % of the GPU, and is run again, up to three times, when they rose above that during one of its runs; the SMA launch row comes from a re-run that repeats the single loaded runs instead (`--run-redo 3`). Rates: the median over the rounds; ratios: the median of the per-round ratios. Stock: the best of its three thread counts. Bold: cg29's best CPU setting, which the two CPU ratios use. Every CPU run's port probes are identical to stock's at 4 threads; the GPU runs differ from them by at most 9.4×10^{-7} relative, because the GPU flushes subnormal numbers (with `OPENEMS_FTZ=1` on the CPU they are identical, `results/*_cg29gpu.txt`). The SMA launch on the 1.071 M-cell mesh of the earlier tables (`RFSIM_MIN_CELL=0.02`); the toolkit's default since 2026-11-04 builds 0.871 M cells. The other processes used a median of 0.5–0.8 cores per model during the runs kept.

8.3 End-to-end run time

Figure 8 and Table 11 give the number the author cares about: the wall time of a complete simulation of each board from the toolkit, timed around its `FDTD.Run` call (operator setup and time stepping). All runs were made on 2026-11-03 with the toolkit as it was that day and the meshes it built, one per board with stock v0.37.0 and one per setting with cg18 (`results/rfsim_e2e_2026-11-03.txt`). The columns are: the full run with stock, which goes to `max_time` because the energy criterion never fires (Section 7.1); the same full run with fdtdKit, same steps, so the difference is the solver's alone; fdtdKit with the probe end at −60 dB and a hold of 1, the toolkit's default until 2026-11-04; and a hold of 0.5 with, on the SMA launch, the 0.05 mm mesh floor of Section 7.3, the shortest setting tested, which the author made the toolkit's default on 2026-11-04 (toolkit commit 4a7e20e). Model and Gaussian pulse are the same in every column (the SMA launch's mesh is coarser in the last). The solver alone gives 5.6× on the GCPW line and 3.3× on the SMA launch; the probe end with the earlier defaults brings this to 8.9× and 5.8×, the toolkit's current defaults to 14.7× and 15.0×, at the accuracy costs of Tables 6 and 7. Stock ran with the thread counts the toolkit uses for a stock library, 4 on the GCPW line and 8 on the SMA launch, the same counts the tuner picked for the two full runs. On the SMA launch this 8-thread stock run reached 348 MC/s, while stock's 4-thread bench run on the same board reached 437 MC/s (Table 9): the SMA factors are against the toolkit's stock setting.

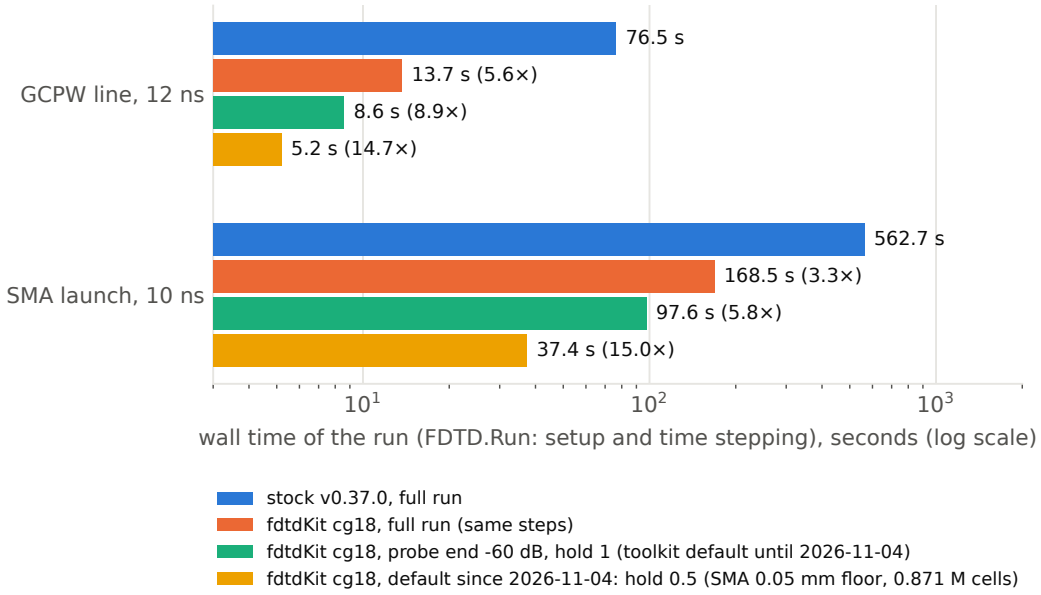


Figure 8: Wall time of `FDTD.Run` for a complete simulation of the two boards, measured on 2026-11-03 (Table 11): stock v0.37.0 and fdtdKit (snapshot cg18) for the full run, cg18 with the probe end at −60 dB and a hold of 1 (the toolkit's default until 2026-11-04), and cg18 with the toolkit's defaults since 2026-11-04 (a hold of 0.5; on the SMA launch also the 0.05 mm mesh floor). Labels: the time and, for fdtdKit, the factor against stock.

		stock v0.37.0	fdtdKit (cg18)		
board	cells	full run	full run (same steps)	probe end −60 dB, hold 1†	toolkit default since 2026-11-04*
GCPW line, 50 mm, 12 ns	0.242 M	94 217 steps 76.5 s	94 217 steps 13.7 s, 5.6×	45 570 steps 8.6 s, 8.9×	34 300 steps 5.2 s, 14.7×
SMA launch, 10 ns	1.071 M	182 622 steps 562.7 s	182 622 steps 168.5 s, 3.3×	103 831 steps 97.6 s, 5.8×	50 320 steps 37.4 s, 15.0×

Table 11: End-to-end runs of the two boards from the author's toolkit, measured on 2026-11-03 with snapshot cg18 (patches 0001–0006, the build installed on the author's Mac from 2026-11-04 to 2026-11-06) and with stock v0.37.0, one run each on a quiet machine (before each run the other processes used less than one core for 10 s; the file gives their load during each run), all on meshes that the toolkit built that day (`results/rfsim_e2e_2026-11-03.txt`). Time: wall time of `FDTD.Run` (operator setup and time stepping); the model build (0.11–0.79 s) and the port evaluation (`CalcPort`, at most 0.03 s) are not included. Factor: stock time over fdtdKit time. Stock ran with the thread counts the toolkit uses for a stock library (4 on the GCPW line, 8 on the SMA launch), fdtdKit with its tuner; the stock full runs' port files are byte-identical to fdtdKit's. In the GCPW's −60 dB, hold 1 run the tuner picked 8 threads (1 304 MC/s, against 1 636–1 683 MC/s at 4 threads in the other fdtdKit GCPW runs). †The toolkit's default until 2026-11-04. *Probe end −60 dB with a hold of 0.5; on the SMA launch also the 0.05 mm mesh floor of Section 7.3 (0.871 M cells); the GCPW line has no mesh floor. The shortest setting tested on 2026-11-03; the author made it the toolkit's default on 2026-11-04 (toolkit commit 4a7e20e).

Table 12 has the same boards with cg29 on 2026-11-05, with the toolkit's current defaults and as full runs (the SMA launch on its 0.871 M mesh in both), on the CPU (the multithreaded engine with the automatic thread count) and on the Metal diamond engine, in interleaved rounds (`results/rfsim_e2e_cg29_2026-11-05.txt`). Paired per round, the GPU ran the default runs 1.37× (GCPW line) and 1.22× (SMA launch) as fast as the CPU engine and both full runs 1.17×; over the rounds in which both runs were clean the ratios were 1.27× and 1.21× for the default runs (two rounds each) and 1.23× and 1.08× for the full runs (one round each). Both engines stopped at the same steps, and the GPU's results differ from the CPU's by the amounts in the

caption, as the GPU flushes subnormal numbers and the CPU runs here keep them. These runs saw more load than those of Table 11: the other processes used a median of 1.0–1.6 cores per row, and in 5 of the 20 runs kept they stayed above the limit through the third redo. In the one round in which both of the SMA launch's default runs were loaded, the CPU took 36.2 s against 27.9–28.9 s in the clean rounds and the GPU 24.4 s against 23.5–23.6 s. With that load the CPU columns are not a comparison with cg18: the GCPW line's default run took 5.6 s on cg29's CPU engine here and 5.2 s on cg18's quiet run of Table 11, while in bench runs that pair the two builds round by round (5 rounds, other processes on 0.4 cores) cg29's automatic thread count runs that board 1.30× and the SMA launch, on the same 0.871 M mesh, 1.42× as fast as cg18's (`results/cpu_items_cg29_2026-11-05.txt`, section 4). From 2026-11-06 cg29 was the installed build and since 2026-11-11 cg31 is (Section 3), so the toolkit's runs, on the CPU engine with the automatic thread count and the probe end, run on the fork with no change on the toolkit's side; the Metal engine stays opt-in, and the toolkit does not select it.

			cg29, `FDTD.Run` wall time			2026-11-03, same steps	
run	cells	steps	CPU	GPU	CPU / GPU	stock	cg18
GCPW line, full run	0.242 M	94 217	13.1 s	11.2 s	1.17×	76.5 s	13.7 s
GCPW line, toolkit default	0.242 M	34 300	5.6 s	4.1 s	1.37×	–	5.2 s
SMA launch, full run	0.871 M	118 475	68.1 s	58.5 s	1.17×	–	–
SMA launch, toolkit default	0.871 M	50 320	28.9 s	23.6 s	1.22×	–	37.4 s

Table 12: End-to-end runs of the two boards from the author's toolkit with snapshot cg29 on 2026-11-05 (`results/rfsim_e2e_cg29_2026-11-05.txt`): the multithreaded engine with the automatic thread count (CPU) against the Metal diamond engine (GPU), with the toolkit's defaults since 2026-11-04 (SMA mesh floor 0.05 mm, 0.871 M cells; probe end −60 dB with a hold of 0.5 excitation lengths: "toolkit default") and with the probe end off ("full run", to `max_time`: GCPW 12 ns, SMA 10 ns). Time: wall time of `FDTD.Run` (operator setup and time stepping), the median over 2–3 interleaved rounds; each run started once the other processes used at most 1.5 cores and 10 % of the GPU and was run again, up to three times, when they rose above that during it; in 5 of the 20 runs kept the others stayed above that through the third redo (at most 4.4 cores): the medians include them, and the file gives the medians of the clean runs as well. CPU / GPU: the median of the per-round ratios of the run times. In every round both engines stop at the same step; the GPU's results differ from the CPU's by at most 7.3×10^{-3} dB in S_{11} (where $|S_{11}| > -40$ dB), 2.0×10^{-4} dB in S_{21} and $5.4 \times 10^{-3}\,\Omega$ in Z_0 at the reference frequency, as the GPU flushes subnormal numbers (the file gives each round). The last two columns are the runs with the same settings and steps in Table 11 (one run each, stock with the toolkit's 4 threads, cg18 with its tuner, on a quiet Mac: the others below one core for 10 s before each run); the SMA's full run on the 0.871 M mesh was not made there. During the runs here the others used a median of 1.0–1.6 cores per row, so the CPU columns are not a comparison with cg18 (paired, cg29 against cg18: Section 4.14).

8.4 Common benchmarks

The tables so far use this project's own models. Four measurements of 2026-11-10 use tests that readers outside the project know by name: upstream's own benchmark on current master, STREAM, a free-space cube after the gprMax benchmark models, and the radar cross-section of a metal sphere against the Mie series (`results/common_benchmarks_2026-11-10.txt`; `scripts/common_bench/`). The fork is cg29, the installed build of that day, in all four.

Upstream's benchmark, and master's new engine. Upstream master has a benchmark tool, `benchmark/bench.py`: six cases (an empty box, a box with PML, an inhomogeneous one, a dispersive one, an antenna with field dumps, a cylindrical mesh) at about 1 M (size S) and 8 M cells (size M), each timed for 3 s of time stepping with the `openEMS` binary. Since the fork's base, master has also gained a third engine, `--engine=cpu`, beside `multithreaded` and `gpu` [28]. The tool ran twice on master 1d710c2: master alone on its three engines (part A), and master against the fork on the `multithreaded` engine, the one engine both have, interleaved as upstream asks (part B); 4 and 8 threads, three repeats (Table 13). During both parts the other processes used a median of 0.22 cores (three of 75 samples above 0.8). The tool's column that says whether two builds compute the same thing was not produced: it needs master's Python module, which was not built.

		part B, interleaved			part A, master alone		
case	size	master `multithr.`	fdtdKit `multithr.`	ratio	master `multithr.`	master `cpu`	master `gpu`
ideal	S	1 181	2 456	2.08×	1 267	1 907	1 703
pml	S	189	1 056	5.60×	201	709	1 163
inhomogeneous	S	187	1 014	5.42×	203	678	1 153
dispersive	S	48	479	10.0×	53	921	737
antenna	S	148	871	5.87×	162	679	1 078
cylindrical	S	911	1 544	1.69×	1 029	1 304	1 537
ideal	M	1 088	1 739	1.60×	1 360	2 099	1 689
pml	M	277	1 061	3.83×	295	1 225	1 414
inhomogeneous	M	266	967	3.63×	286	1 188	1 413
dispersive	M	53	448	8.47×	52	1 109	739
antenna	M	214	908	4.24×	230	1 088	1 334
cylindrical	M	823	1 012	1.23×	921	1 755	1 590

Table 13: Upstream's `benchmark/bench.py` on the M5, 2026-11-10, in MC/s: the better of 4 and 8 threads per cell of the table, each the median of three repeats as the tool reports it. Master: openEMS 1d710c2 with CSXCAD fdff4b5; fdtdKit: cg29, as installed that day. Part B ran the two builds interleaved; part A ran master alone, 15 minutes earlier, so its `cpu` and `gpu` columns are not paired with the fork. The tool's spread was below 5 % in most rows and above 10 % in ten of the 120 (master's dispersive S rows in part B, 23 and 26 %, among them); the file has every row. The ideal, pml and dispersive rows ran on zero fields (see the flaw described below).

On the engine both builds have, the fork runs 1.23–8.5× as fast as master at 8 M cells and 1.69–10.0× at 1 M, the least on the cylindrical mesh and the most on the dispersive case, as against v0.37.0 (Section 4.14). Master's new `cpu` engine changes the picture at 8 M cells: taken as measured, its rates are 1.15–1.23× the fork's on the ideal, pml, inhomogeneous and antenna cases, 1.73× on the cylindrical mesh and 2.5× on the dispersive case. The two columns come from different parts, and master's own `multithreaded` rates were 6–25 % higher in part A than in part B (dispersive M: 2 % lower); with each engine taken relative to master's `multithreaded` rate of its own part, the `cpu` engine is level with the fork on the ideal case (0.97×), 1.08–1.14× on pml, inhomogeneous and antenna, 1.55× on the cylindrical mesh and 2.5× on the dispersive case. At 1 M cells the fork is ahead of the `cpu` engine on five cases (1.18–1.50× as measured) and behind on the dispersive one (0.52×). So on large grids upstream's own new engine is level with or ahead of this fork's CPU engine, and well ahead with dispersive material; a paired run of the two on these cases with the probes compared was made later, on the M5 Pro, with a source on the mesh (Section 10.1); on this M5 none has been made (on another model, run interleaved on 2026-11-11, cg31's CPU engine was 1.27× master's `cpu` engine at 3.0 M cells and 0.86× at 17 M, with the fork's row the better of 8 and 10 threads and master's at 8; Table 19). The fork's tiled sweep of 2026-11-11, which blocks in time as that engine does, was not run on these cases or next to it on the M5; on the M5 Pro it was run on two of them at 8 M cells (Section 10.1). The fork's Metal diamond engine was not part of this run: every case of the tool uses the steady-state detection, which that engine runs on its CPU loops (Section 13).

A flaw in upstream's cases, found on 2026-11-13. In `benchmark/cases.py` of master 1d710c2 the function that builds the uniform meshes puts its lines at `arange(n) - (n-1)/2`. With an even number of lines, which the default sizes S (100 lines) and M (200 lines) have (the sizes XS, 37 lines, and L, 311, are odd), no mesh line passes through the origin, so the dipole that the cases place at $(0, 0, z)$ excites nothing: openEMS prints "Voltage excitations: 0", and the cases `ideal`, `pml` and `dispersive` run with all fields zero. The other three cases have their own sources and are not affected. Two consequences for this report. First, every "probes identical" of those three cases at S or M, here and in the record of 2026-11-10, is an empty statement. Second, the S and M rates of Table 13 for those cases are rates on zero fields. On the M5 Pro the cases were run again with a source on the mesh (101 and 201 lines) and, at 8 M cells and 15 threads, the empty case and the dispersive case ran at the same rates with and without a source within the spread of the rows (Section 10.1), so the rates stand there; on this M5 they were not run again, so the same equality is expected, not measured. The fork's bit identity on these cases rests on the XS size and on the runs with 101 lines (Section 10.1). The finding has not been reported upstream.

STREAM. The introduction takes the machine's memory bus as about 150 GB/s from Apple's figure. STREAM [32], the usual measure of sustained memory bandwidth (version 5.10, 100 M elements per array, OpenMP, the best of 20 passes), gives Table 14: 120–130 GB/s at 4 threads and 125–138 GB/s at 8 and 10, and one thread alone already draws 110–115 GB/s. STREAM counts 16 or 24 bytes per element and not the read that a store to a new cache line may cost, so its figures are a lower bound of the traffic. They sit where this paper's kernels do: the Metal kernels drew 121–127 GB/s (Section 6), and the fused CPU sweep 95 GB/s at 4 threads (Section 4). A system process held one core during the run.

threads	Copy	Scale	Add	Triad
1	114.3	110.4	114.5	115.3
4	129.0	130.3	120.3	120.1
8	135.9	125.5	124.7	125.0
10	137.7	125.7	126.2	126.1

Table 14: STREAM 5.10 on the M5 (16 GB), best rate in GB/s (10^9 bytes per second), 2026-11-10; Apple clang `-O3 -mcpu=native`, OpenMP, 100 M elements per array.

A free-space cube. The gprMax project times its solver on cubes of free space with one dipole source and its default 10-cell absorbing boundary, from 100^3 to 800^3 cells [14, 33]. The model `cube` of `bench/models.py` builds the same problem in openEMS: N^3 cells of 1 mm, one excited cell edge at the centre, `PML_10` on all six faces, one voltage probe. Table 15 has $N = 100$ to 400, the largest that 16 GB hold with room to spare, in three paired rounds per size. These are rates of one model on three builds of one solver; they are not set against the rates that gprMax publishes for its own code.

N	cells	steps	stock best	fdtdKit 4 thr	fdtdKit 8 thr	Metal diamond	CPU / stock	Metal / CPU
100	1.0 M	6 000	181	923	1 222	992	6.77×	0.81×
150	3.4 M	1 800	256	1 038	1 339	1 353	5.22×	1.01×
200	8.0 M	800	299	1 030	1 228	1 470	4.11×	1.20×
300	27 M	600	370	946	1 097	1 703	2.97×	1.55×
400	64 M	600	439	875	1 034	1 834	2.35×	1.77×

Table 15: The free-space cube with a 10-cell PML on every face, 2026-11-10, in MC/s: medians of three paired rounds (`bench/abtest.py`). Stock: v0.37.0 at the better of 4 and 8 threads ($N \geq 300$: 8 threads only); fdtdKit: cg29, `multithreaded`. Ratios: of the medians, the CPU ratio with fdtdKit at 8 threads. Every CPU run's probe is identical to stock's; the Metal engine's differs by 5.9×10^{-7} to 3.9×10^{-6} relative (it flushes subnormals). Other processes: a median of 1.3–1.6 cores during the runs of $N = 100$ and 150, where a run that saw them above the limit of 1.5 cores was kept after one redo in every round, and 0.3–0.7 cores for $N \geq 200$, where that happened in one round each at $N = 200$ and 300.

The absorbing boundary sets the scale of these rates. At $N = 100$ the six PML slabs hold 49 % of the cells and at $N = 400$ 14 %, and stock updates PML cells far more slowly than plain ones, so stock's rate rises from 181 to 439 MC/s with the size, the opposite of the six models. The fork's CPU engine stays at 1 034–1 339 MC/s, 2.4 to 6.8 times stock. The Metal diamond engine passes the CPU engine at about 3 M cells, as in Figure 3, and at 64 M cells it runs 1.77× as fast, the largest lead in a paired run of this project up to that day; its rate still rises at the largest size. With cg31 the Metal engine runs the 1.0 and 8.0 M-cell cubes at 1 546 and 1 839 MC/s, 1.58× and 1.24× its rates with cg29 in the same rounds and 1.15× and 1.42× the CPU engine (Table 18).

A metal sphere against the Mie series. The monostatic radar cross-section of a perfectly conducting sphere has an exact series solution [34], and openEMS ships the case as a tutorial (`RCS_Sphere`: radius 200 mm, a plane wave from a total-field/scattered-field box, `PML_8`, the far field through the nf2ff box). `scripts/common_bench/mie.py` runs it with a fixed number of steps and compares 95 frequencies, $ka = 0.41$ to 4.19, with the series (Table 16).

cell at 1 GHz	build, engine	cells	mean |error|	largest
$\lambda/20$ (the tutorial's)	stock v0.37.0	0.57 M	0.770 dB	2.943 dB
$\lambda/20$	fdtdKit, multithreaded	0.57 M	0.770 dB	2.943 dB
$\lambda/40$	fdtdKit, multithreaded	4.33 M	0.357 dB	0.880 dB
$\lambda/40$	fdtdKit, Metal diamond*	4.33 M	0.357 dB	0.880 dB

Table 16: Backscatter of a PEC sphere of 200 mm radius against the Mie series, 95 frequencies with ka from 0.41 to 4.19, 2026-11-10; 1 400 steps at $\lambda/20$ and 2 000 at $\lambda/40$. *The engine has no GPU version of the plane-wave source and ran its CPU loops.

The error is the method's and the mesh's: a staircased sphere on a uniform mesh, with a mean that halves when the cell is halved. It is not the fork's. Stock and fork give the same error to the digits shown; their cross-sections differ by at most 2.5×10^{-6} relative, all of it from the fork's separable far-field routine (Section 5), and with that routine switched off (`OPENEMS_NF2FF_LEGACY=1`) the two are identical at all 100 frequencies. One thing this case showed about the tutorial itself: with its energy end criterion the stop step is not reproducible (1 104 and 1 340 steps in two stock runs), because the engine tests the criterion between chunks of steps whose length follows the measured speed, and two such runs differ by up to 1.6×10^{-3} in the cross-section. Comparisons across builds need a fixed step count, as everywhere else in this paper.

9 After publication: the build of 2026-11-11 (cg31)

Every table before this section is of snapshot cg29 or earlier. On 2026-11-11, the day after the repository was published, twelve commits followed (patches 0028 to 0039): one on the energy schedule of a run without an end criteria (0028), ten on the Metal diamond engine (0029 to 0038) and one on the cylinder extension (0039). The build, cg31, has been the installed one on the author's Mac since that day. This section records what was looked for, what changed and what it gave (`results/metal_engine_cg31_2026-11-11.txt`). A thirteenth commit of that day, patch 0040, adds an opt-in tiled sweep to the CPU engine (Section 9.6); it is not part of the installed build, and three more commits of 2026-11-13 (patches 0041 to 0043, now the fork's head, 13350ca) are the subject of Section 10.1.

9.1 Where the Metal engine's time went

Three observations set the work list.

An energy sum on one thread. The fork's base estimates the field energy every Nyquist period during the first 60 s of a run, so that a run with an end criteria stops at a reproducible step. The estimate is one pass of one CPU thread over both fields, and the engine waits for it. On a benchmark model with a Nyquist period of 51 steps this was 4–6 % of the time-stepping loop on 3–17 M cells; with a period of 519 steps the time lost outside the engine fell from 0.23 s to 0.03 s per 3 000 steps (project log). A run with a fixed number of steps and no end criteria has nothing to stop on, so it now makes no such estimates (patch 0028), as in v0.37.0. The author's board runs have a criteria and Nyquist periods above a thousand steps: for them the estimate costs 0.1–0.2 % (project log) and stays.

Extensions cost by being present. In a timing-only build that leaves parts of the kernel out (results wrong by design), the GCPW line ran 1.35× as fast without its Mur faces, 1.32× without its conducting sheets and 2.0× without any extension; the SMA launch 1.69× without its UPML slab on one z face (the record gives no load of the machine for these timing-only runs). But on a later snapshot, where the same builds of the GCPW line gave 1.26×, 1.22× and 1.94×, skipping one stage at run time, with the code still in the kernel, gave only 1.02–1.07× each (the sheets' Pre and Apply, Mur's Pre and Post). The cost is tied to the code being compiled into the kernel, not to the loops over the few cells it handles; what in the GPU's execution makes it so was not found. Empty extra barriers cost nothing measurable, and moving the Pre stages of Mur and of the sheets into the half-step before, beside the cell update, which saves one barrier per half-step with identical fields, ran 2–4 % *slower* and was dropped.

Not at the bus on mid-size grids. Counted from the tile schedule, a tile that stays cached for its block loads and stores 20–29 bytes of field per cell and time step, which at the measured rates is 79–105 GB/s at 3 M cells, below the 125–138 GB/s that STREAM measures on this machine (Section 8.4); the count assumes perfect reuse within a block and was not measured. At 17 M cells with 257 z lines a tile is 0.9–1.5 MB, because a tile holds every z line, and the shallower $K = 3$ tilings win although they move more bytes (the byte counts and tile sizes of this paragraph are from the project log).

9.2 What changed

patch	change	switch (default on)
0028	no energy schedule in a run without an end criteria	
0029, 0032	Mur faces of one axis share a barrier; Post and Apply in one pass where a tile's faces are on one axis	`DIAMOND_MURMERGE`
0029	a call's rest in blocks of nearly equal depth (10 steps at $K = 4$: 4, 3, 3)	`DIAMOND_EVEN`
0030	three dispatches per block: the mountain×valley and valley×mountain tiles do not depend on each other and go together	`DIAMOND_MERGE`
0031, 0032, 0034	no divergent branch for the z neighbour; divisions by run-time widths as multiplications by reciprocals from the host; threads without work leave a Mur stage early	`DIAMOND_FASTIDX`
0031	the pipeline is told that a threadgroup is a whole number of SIMD groups	`DIAMOND_TGFLAG`
0033	UPML of the boxes with every z line (the x and y slabs) a z-vector at a time	`DIAMOND_PML4`
0035	UPML of the z slabs three components of a cell at a time	`DIAMOND_PML3`
0036	dispersive materials (sheets, Drude/Lorentz, Debye) three components of a cell at a time	
0037	offsets and counts of the extension data as constants of the kernel, which is compiled after the data are packed	
0038	seven tilings in a two-stage tuner (adds $K = 4$ with 10,6 and 9,2 tiles and $K = 5$ with 12,8); the per-tile lists in their own buffer	`DIAMOND_CAND`
0039	line hooks for the cylinder extension (Section 9.5)	

Table 17: The twelve commits of cg31, 2026-11-11 (the thirteenth of that day, patch 0040, is in Section 9.6). Switch names carry the prefix `OPENEMS_`; `=0` restores the earlier behaviour (`DIAMOND_CAND=4`: the four tilings of patch 0016). Every step was a snapshot of its own (cg30a to cg30l, cg31a) that passed `scripts/checks/diamond_check.sh` with 72 of 72 comparisons identical.

The mountain×valley and the valley×mountain tiles of a block can share a dispatch because a mountain cell reads only cells that were mountain cells one half-step earlier, in both directions: a cell leaves its x mountain or its y mountain first, so it is updated by one of the two kinds only, and neither kind reads what the other writes within the block. The UPML stages were scalar loops over single floats, three components and up to four lanes apart; an x or y slab has every z line, so its cells fill whole z-vectors and one pass can update a float4 of one component with the same operations per lane, while a z slab has one lane of a z-vector per cell and its pass now takes the three components of a cell together. One build in this series was wrong: it computed the reciprocals for the divisions on the GPU, and on this M5 (macOS 27.0.1) the GPU's own `0xffffffff / w` returns 3 579 139 454 for $w = 3$, where 1 431 655 765 is right. The identity check caught it as wrong fields on the Mur cases; the reciprocals now come from the host.

9.3 What it gave

	Metal diamond engine			CPU engine, cg31		Metal /
model	cg29	cg31	cg31 / cg29	8 threads	4 threads	CPU best
GCPW line, 0.24 M	2 118	2 307	1.09×	2 000	1 816	1.15×
SMA launch, 0.87 M	1 854	2 108	1.14×	1 955	1 509	1.08×
patch tutorial, 0.10 M	1 753	1 887	1.08×	1 910	1 868	0.99×
notch tutorial, 0.27 M	1 904	2 907	1.53×	2 129	1 909	1.37×
patch x5, 7.86 M	2 901	3 100	1.07×	2 104	1 802	1.47×
notch x3, 5.25 M	3 210	3 702	1.15×	2 330	1 853	1.59×
PEC box, 3.0 M	3 692	4 408	1.19×	2 614	2 156	1.69×
PEC box, 17 M	3 056	3 507	1.15×	1 736	1 550	2.02×
cube with 10-cell PML, 1.0 M	980	1 546	1.58×	1 343	1 012	1.15×
cube with 10-cell PML, 8.0 M	1 483	1 839	1.24×	1 293	1 049	1.42×

Table 18: The Metal diamond engine before (cg29) and after (cg31) the commits of 2026-11-11, with cg31's CPU engine beside it: median MC/s of four paired rounds per model on 2026-11-11 with the other sessions on the machine paused (the other processes used 0.2–0.9 cores; a run that saw more than 1.5 cores was repeated once). cg31 / cg29: the median of the per-round ratios, which over the rounds stayed within 3 % of the value shown except on the patch tutorial (0.92–1.13), the notch tutorial (1.42–1.53) and the two cubes (1.51–1.58 and 1.20–1.29). The PEC box is the benchmark model of luikore's engine (Table 19), the cube that of Table 15. The SMA launch is on the toolkit's current mesh. Every cg31 Metal run's probes are identical to cg29's.

The two boards gain 9 and 14 %; the notch tutorial and the PML cube gain the most (53 %, and 58 and 24 %), in good part from the UPML stages on whole faces (patch 0033); the two large tutorial models gain 7 and 15 % and the PEC box 19 and 15 %. With that the Metal engine is ahead of the CPU engine on every model but the patch tutorial, where the two are level. cg31 also passed, on the CPU side, `emskit fdtd check` on the six models against stock (all identical, 18 of 18 rows), the Debye check and both boards end to end, which stop at the same steps as with cg29 (34 300 and 50 320); the validation suite on the Metal engine (15 of 15) ran on the snapshot of patches 0001–0038 (cg30l), before the cylinder patch.

9.4 Three builds on one model

Section 6 had no head-to-head with luikore's Metal engine [20]: its diamond path refuses the six models. It does run its own benchmark model, a uniform grid with PEC walls, one dielectric box and a point excitation, so the three builds were run on that model, each binary from the same model file, interleaved, with the other sessions paused (Table 19; `scripts/luikore_compare/`, `results/engines_on_luikore_model_2026-11-11.txt`).

engine	3.0 M cells	17.0 M cells
fdtdKit (cg31), Metal diamond	3 981	3 309
luikore PR #228 (55ce11b), Metal	3 952	2 829
master 1d710c2, `gpu`	1 638	1 566
master 1d710c2, `cpu`, 8 threads	1 872	1 933
fdtdKit (cg31), `multithreaded`, best of 8 and 10 threads	2 371	1 659
luikore PR #228, `multithreaded`, 8 threads	1 296	1 042
master 1d710c2, `multithreaded`, 8 threads	1 246	1 055

Table 19: Three builds on the benchmark model of luikore's engine, one Apple M5, 2026-11-11: median MC/s of three interleaved repetitions (3 000 steps at 3.0 M cells, 1 000 at 17.0 M); the other processes used a median of 0.44 cores before a run, at most 1.14. The `multithreaded` engines of the three builds give identical probes; the others differ from them by 7×10^{-6} to 3×10^{-5} relative.

On its own model luikore's engine and cg31's are level at 3.0 M cells (1.01×) and cg31's is 1.17× as fast at 17 M; the night before, cg29's had run 3 728 and 3 092, 0.94× and 1.09× luikore's in that run (3 958 and 2 848; `results/luikore_compare_cg29_2026-11-11.txt`). cg31's engine is 2.4× and 2.1× master's `gpu` engine here, luikore's 2.4× and 1.8×. On the CPU, cg31 is 1.27× master's new `cpu` engine at 3.0 M cells and 0.86×

at 17 M: on the large grid master's engine is ahead, as in Table 13. Three things qualify the table. The CPU rows were not tuned equally: the fork's row is the better of 8 and 10 threads, and master's `cpu` engine ran at 8 threads only. It is one model, the best case of a GPU engine (a regular grid, no boundary work). And a model file whose `endCriteria` is 0 makes the `openEMS` program use its default of −60 dB, so here the fork and master make the energy estimates of Section 9 and luikore's build does not: run from Python, where 0 means no criteria, cg31's Metal engine does 4 408 and 3 507 MC/s on this model (Table 18).

Master's `cpu` engine blocks in time: all threads work in one tile of the size of the performance cores' cache for up to eight steps. With that switched off (`OPENEMS_CPU_TBLOCK=0`) it ran upstream's ideal case at 8 M cells at 1 347 and 1 436 MC/s (4 and 8 threads) against 2 010 and 2 099 blocked the day before, and the PML case at 891 and 1 069 against 1 117 and 1 225: the blocking is worth 1.46–1.49× and 1.15–1.25× on this machine (two runs a day apart). The fork's fused sweep, at 1 739 on the ideal case, is ahead of master's unblocked kernel and behind its blocked one. The experiment of Section 12 that found no gain in temporal blocking on the CPU gave every thread a sub-grid of its own, which does not fit the cache four threads share; one tile for all threads was built the same day as an opt-in sweep of the fork's CPU engine (patch 0040, Section 9.6).

9.5 A closed cylinder on the fused sweep

The cylinder extension, which closes a mesh in α and handles $r = 0$, declared no hooks, so such a mesh ran the unfused plan with the extension's two Post hooks on one thread. Both hooks stay on the x-line (r-line) they read and keep no state, so they are line hooks now (patch 0039) and the mesh runs the fused sweep with the neighbour flags. On a closed cylinder of 61×72×41 lines with $r = 0$ the engine ran 1 857 and 1 891 MC/s at 8 threads before and 2 503 and 2 553 after (two rounds in the quiet block; at 4 threads the rounds disagree, 2 126 and 2 176 against 2 125 and 2 400; stock 464–474). Five probes on the axis and the α seam equal stock's, the raw fields are bit-identical to the unfused plan and at 1, 4, 8 and 10 threads, with and without $r = 0$, and the cylindrical multigrid tutorial, which stays unfused, has 18 datasets with no difference (`scripts/cylinder_closed/`). The first version of this test had its source between two mesh lines: every probe was zero and every comparison passed. The script now puts the source and the probes on mesh lines.

9.6 A tiled sweep for the CPU engine (opt-in)

On a grid larger than the cache the fused sweep reads every field array from memory once per time step. With `OPENEMS_TBLOCK=`k (patch 0040) the multithreaded engine runs k time steps on one tile of x-lines before it moves on, and every thread works in that same tile, on its share of the y lines, so a tile is read from memory once per k steps. Within a block of k steps a tile shrinks by one x-line per step on each inner side, and the wedges between the tiles are filled afterwards; each such range of x-lines is a task, and the threads meet at one spin barrier per task. The design follows the `cpu` engine of upstream master (one cache-sized tile shared by all threads, Section 9.4); none of its code is used. `OPENEMS_TBLOCK=auto` takes the deepest $k \leq 8$ whose tile of $2k + 1$ x-lines stays within 90 % of the cache the threads share (the performance cores' L2 cache, 16 MiB on the M5), with tiles of at least 200 000 cells, and keeps the fused sweep when the fields fit that cache or two time steps of a tile do not. `OPENEMS_TBLOCK_W` sets the tile width in x-lines. With `OPENEMS_TBLOCK` unset or 0 the engine runs the fused sweep as before.

The line hooks of the extensions (Section 4.5) stay exact because they run in stages between two updates: the Post and Apply hooks of the task before, then the Pre hooks of the task after, each at its own time step. A stage runs on all threads. Its x-lines are cut once into one share per thread, so all hooks of an x-line run on one thread in the order of a one-thread sweep, and the two pairs of lines that a hook may couple (lines 0 and 1, and the last two) are never separated. No extension was changed for this. One call was added: an extension can cap the depth of a block (`GetMaxBlockDepth()`), and the steady-state detection allows one signal period. The tiled sweep runs only where the fused sweep can (every extension has line hooks). The engine refuses it with a message, and runs the fused sweep, when an extension is not `TS_LOCAL` (Section 4.14), when a hook couples other x-lines than those two pairs, or when the grid has fewer x-lines than two tiles.

The results are identical to the fused sweep's in every test (`scripts/tiled_sweep/check.zsh`). On the M5, against cg31: the probes of six models (the two boards, the patch and notch tutorials, the 3 M-cell PEC box and the 1 M-cell PML cube), 30 of 30 comparisons at 2 threads and 36 of 36 at 3, 4, 8 and 10 threads, with k from 2 to 7 and `auto`, set tile widths, the hook stages on one thread and equal shares; the raw fields of the closed cylinder of Section 9.5 at 2, 8 and 10 threads; a sinusoidal run that ends by the steady-state criterion (a period of 87 steps), with the same stop step (17 580), probes and raw fields for $k = 4$, 5, 8 and 19 at 1 to 10 threads; and the Debye check with $k = 3$ and 4. With the variable unset the reduced check is identical to stock v0.37.0 on the six benchmark models (18 of 18 rows). On a second machine, a Mac mini with an M5 Pro (5 + 10 cores), the same library files gave the same verdicts at 5 to 15 threads (Section 10). Three read-only reviewers, each finding handed to a second reader, found no defect in the results; the two faults

they did find were fixed before the commit (the share balance could leave a thread without y lines for the rest of a run, and a depth of 2^{30} or more passed the test for two tiles).

		fused sweep (cg31)		tiled sweep, `auto`			best tiled /
model	cells	4 threads	8 threads	4 threads	8 threads	10 threads	best fused
patch x5 (Mur)	7.86 M	1 841	2 138	2 305	1 872	1 881	1.08×
notch x3 (PML at the x ends)	5.25 M	1 963	2 385	1 861	1 704	1 741	0.81×
cube with 10-cell PML	8.0 M	1 092	1 342	1 138	937	896	0.85×
PEC box	16.8 M	1 648	1 867	2 289	1 995	1 995	1.23×
SMA launch	0.87 M	1 524	1 998	1 352	979	981	0.71×
PEC box	2.95 M	2 357	2 792	2 568	2 186	2 367	0.92×

Table 20: The fused sweep (cg31) and the tiled sweep (snapshot cg32d, patch 0040, with `OPENEMS_TBLOCK=auto`) of the multithreaded engine, in MC/s, on the Apple M5 of this report (4 performance and 6 efficiency cores, 16 GB) under macOS 27.2.1 (the other M5 tables of this report: 27.0.1), 2026-11-11: medians of three paired interleaved rounds. The other processes used a median of 9 % of one core during the runs (the highest row 63 %). Best tiled: the fastest of the six tiled settings of a model, which is `auto` at 4 threads except on notch x3 ($k = 8$ with tiles of 17 x-lines at 4 threads, 1 925) and on the SMA launch ($k = 8$ with tiles of 65 x-lines at 4 threads, 1 412); best fused: 8 threads on every model. The results of every run are identical to those of the fused sweep at 4 threads. Notch x3 has `PML_8` at both x ends. The PEC box is the benchmark model of luikore's engine (Table 19, there 3.0 and 17.0 M cells), the cube that of Table 15, with `PML_10` on all six faces; the SMA launch is on the toolkit's current mesh. These rows are compared only with each other (`results/tiled_sweep_cg32_2026-11-11.txt`).

Against the fused sweep at its best setting, 8 threads, the tiled sweep is ahead on the two large grids with little extension work, patch x5 (1.08×) and the 16.8 M-cell box (1.23×), and behind on the other four: the 2.95 M-cell box (0.92×), the PML cube (0.85×), notch x3 (0.81×) and the SMA launch (0.71×; Table 20). Against the fused sweep at the same 4 threads its best setting runs 1.25×, 1.39×, 1.09×, 1.04×, 0.98× and 0.93× as fast, in that order of models. Four things limit it.

- *It needs the four performance cores.* At 8 or 10 threads it is behind the fused sweep at 8 threads on every model but the 16.8 M-cell box. A task ends only when its slowest thread is done. The shares follow each thread's measured speed, and against equal shares (`OPENEMS_BALANCE=0`) that changed the rate at 8 threads by less than 3 % on every model.
- *The hook stages must run on all threads.* With them on one thread (`OPENEMS_TBLOCK_HOOKS=0`, a diagnostic) patch x5 ran at 2 044 MC/s instead of 2 305, notch x3 at 1 499 instead of 1 861 and the PML cube at 617 instead of 1 138 (4 threads). Even with them on all threads, the three models with a PML stay behind the fused sweep at its best.
- *It does not survive a loaded machine.* During the identity runs of that morning, with 5.6 cores busy with other programs, the tiled sweep at 8 to 10 threads fell to 28–600 MC/s where the fused sweep at 4 threads held 730–1 570: a barrier per task does not survive an oversubscribed machine.
- *The two boards gain nothing.* The fields of the GCPW line fit the cache, so `auto` keeps the fused sweep, and the SMA launch loses.

A standalone kernel had given the expectation that morning (`scripts/kernel/kb_tile.cpp`, 7.86 M cells, a quiet machine): at 4 threads one shared tile with $k = 7$ and 15 x-lines ran at 2 371 MC/s against 1 727 for the fused sweep (1.37×); at 8 threads the shared tile ran at 2 020 against 2 088, and at 10 threads at 1 358 against 2 025. In the engine the gain at 4 threads on the model of that size is 1.25× (2 305 against 1 841).

On the M5 Pro of Section 10, timed on its own at 5, 10 and 15 threads (Table 27), the tiled sweep wins only on the 16.8 M-cell box, 1.43× the fused sweep at its best there, and loses on the other five models (0.41–0.97×): there the fused sweep keeps gaining up to 15 threads, while the tiled sweep peaks at 10. The verdict below holds on that machine too.

The verdict: the tiled sweep is a switch for large grids with little extension work on a quiet machine, not a default. The fused sweep stays the engine's default, and the installed build, cg31, does not have the switch. On the M5 the tiled sweep was not measured against upstream master's `cpu` engine in one paired run; on the M5 Pro the fused sweep was (Section 10.1), and the tiled sweep was timed on two 8 M-cell cases beside the fused one, not run side by side with master's engine. Not built: a choice between the two sweeps at run time.

9.7 A shorter pulse (tested, not adopted)

The toolkit excites with a Gaussian of centre $f_{\max}/2$ and width $f_{\max}/2$. A baseband Gaussian (centre 0, width $f_{\max}$) covers the same band with half the length, and a toolkit run lasts the pulse plus a hold of half a pulse length. Both boards were run with both pulses and compared with the toolkit pulse run to the full time (Table 21; `scripts/pulse/`, `results/pulse_test_2026-11-11.txt`; the excitation was set on the built model, the toolkit was not changed).

board	pulse, hold	steps	max ΔS_{21}	max $\Delta\lvert S_{11}\rvert$	max ΔZ_0
GCPW line	toolkit, 0.5	34 300	0.0007 dB	1.0×10^{-4}	0.05 Ω
	baseband, 0.5	22 540	0.0042 dB	5.5×10^{-4}	0.09 Ω
	baseband, 1.0	27 930	0.0016 dB	2.2×10^{-4}	0.17 Ω
SMA launch	toolkit, 0.5	50 320	0.0003 dB	2.1×10^{-5}	0.66 Ω
	baseband, 0.5	25 900	0.0004 dB	4.8×10^{-5}	0.53 Ω
	baseband, 1.0	34 040	0.0005 dB	4.2×10^{-5}	0.51 Ω

Table 21: The toolkit's pulse and a baseband pulse of half its length on the two boards, probe end at −60 dB with the hold in lengths of the pulse used, against the toolkit pulse run to the full time (94 217 and 118 475 steps), 50 MHz to 2 GHz. $|S_{11}|$ linear: the GCPW line's S_{11} is −40 to −52 dB, the SMA launch's −17 to −23 dB.

The baseband pulse cuts the steps 1.52× on the GCPW line and 1.94× on the SMA launch; the errors grow a few-fold and stay small (S_{21} within 0.005 dB, $|S_{11}|$ at the −65 dB level and below). It is not the toolkit's default: a baseband pulse carries DC, and a board with a conductor that has no DC path, or a series capacitor, was not tested; there the signals may not settle and the run would go to its full time.

10 The same build on a second machine: an M5 Pro Mac mini

Every other speed of this report was measured on the M5 laptop of Section 2. On 2026-11-12 the installed build and the tiled sweep of patch 0040 were measured on a second machine, a Mac mini with an Apple M5 Pro, with the same library files, not rebuilt: cg31 is the fork's commit ff2538a (patches 0001–0039; the CPU engine `multithreaded` and the GPU engine `metal-diamond`), cg32d is 0ef186b (patch 0040), and stock is v0.37.0 (`results/m5pro_speed_set_2026-11-12.txt`, `results/m5pro_tiled_sweep_2026-11-12.txt`; the scripts are in `scripts/m5pro/`). On 2026-11-13 stock was measured again at every thread count from 1 to 15 (Table 25), which corrects the stock column of the speed table of 2026-11-12, and the fork's three later patches, 0041 to 0043 (the head, 13350ca), were timed there (Section 10.1; `results/cpu_items_cg33_2026-11-13.txt`). These numbers form a set of their own. They are compared only with each other: no table of this section holds a number of the M5, and no ratio between the two machines is given, since the two chips differ in cores, caches and memory at once.

The machine. Mac mini, Apple M5 Pro (Mac17,16): 5 "Super" cores (`hw.perflevel0`, one L2 cache of 16 MiB) and 10 "Performance" cores (`hw.perflevel1`, two L2 caches of 8 MiB), 24 GB of unified memory, macOS 27.2.1 (26B131). Apple's figure for the M5 Pro's memory is 307 GB/s [30]. The engine's automatic thread search (Section 4) starts, in cg31 and before, at the `hw.perflevel0` count, here 5; since patch 0041 it starts at every core outside the efficiency class, here 15 (Section 10.1). The `auto` rule of the tiled sweep reads the 16 MiB of the first cluster's L2 cache.

How it was measured. In two blocks of a night queue on the machine, with no other measured job beside them: the tiled sweep from 03:36 to 04:16 (`stageB.zsh`), the rest from 07:13 to 08:32 (`stageA.zsh`: bandwidth, kernels and thread sweep; `stageC.zsh`: speed table; `stageD.zsh`: valley and switches; `stageE.zsh`: the two boards end to end; every stage exited 0). Every rate of Tables 23, 24 and 27 and of the valley and the switches is a median of interleaved rounds of `bench/abtest.py`. Before each of those runs the gate (`scripts/m5pro/common.zsh`) waits up to 20 s for the other processes to use at most 300 % CPU, 3 of the 15 cores, and a run made above that is run once more; every row of the two records lists the other processes' median CPU and GPU use during its runs. STREAM and the two kernels ran without that gate, and the end-to-end runs went through `emskit fdtd e2e`, one run each, after a wait of up to 240 s for the other processes to use at most 70 % CPU. macOS's own indexing (`mediaanalysisd`, Spotlight) ran all night. A first try of the tiled-sweep timing at 02:23, with a stricter gate (70 %, up to 240 s), was stopped: the indexing kept the others

at 1 to 3.5 cores, so nearly every run waited its full time (8 of 162 runs in 55 minutes). Most rows ran beside 0.1 to 0.4 cores of other load. Near or above one core: the speed rows of the notch tutorial (a median of 1.15 cores) and of patch x5 (1.27), whose ranges are wider, and the patch tutorial's row with the automatic count (1.13); in the tiled sweep notch x3's tiled row at 5 threads (1.14), the 2.95 M-cell box with $k = 8$ and tiles of 33 x-lines (3.25) and with $k = 4$ (1.09), and on the PML cube the fused rows and the rows with $k = 4$ and with the hook stages on one thread (2.3 to 3.2 cores), while its tiled `auto` rows ran beside 0.15 to 0.74. A job of another program (a design-rule check of a circuit board, about 1.5 to 2 cores, no GPU) ran from about 07:29 to 07:36. It fell on round 3 of the patch x5 thread sweep, whose medians over all four rounds and over rounds 0 to 2 differ by 0.5 % at most, so all four rounds are kept; and on round 0 of the GCPW sweep, whose runs were 0.7 to 14.5 % slower than the median of rounds 1 to 3 (the others column of that round shows at most 0.9 cores), so the GCPW row of Table 23 uses rounds 1 to 3, which moves no median by more than 1.8 % (the record prints both).

Correctness on this machine. On the first evening on the machine, 2026-11-11 from 21:21 to 21:35, the same libraries passed the checks of Section 3: `emskit fdtd check installed --lite` and `check cg32d --lite` identical to stock on the six models (18 of 18 rows each), the validation suite 15 of 15 with the default engine and thread count and with `metal-diamond`, `diamond-check` 72 of 72, and the Debye check. The tiled sweep's identity checks of Section 9.6 passed at 5, 7, 10, 13 and 15 threads (42 of 42 probe sets on the six models, the closed cylinder's raw fields, the steady-state run and the Debye box; `results/tiled_sweep_cg32_2026-11-11.txt`). In the measurements below every CPU run of cg31 gave probes identical to its reference run, and so did all 162 runs of the tiled sweep; the GPU rows are within 10^{-6} relative of their reference (1.7×10^{-7} to 9.4×10^{-7}: Apple's GPU flushes subnormal numbers, the CPU reference keeps them).

Memory bandwidth and the standalone kernels

STREAM [32] (version 5.10, 100 M elements per array, 20 passes, OpenMP) gives a Triad rate of 131 GB/s on one thread and 256, 244 and 259 GB/s at 5, 10 and 15 (Table 22): the five Super cores alone reach STREAM's rate, about 256 to 262 GB/s. The standalone fused kernel (`scripts/kernel/kb_idx.cpp` with `KB_FUSED=1`, on a grid of the patch x5 size) gains with every thread count, and on one independent sub-grid per thread (`kb_tile.cpp`) it runs at 4 561 MC/s at 15 threads, 223 GB/s at the 49 B per cell of the traffic model, 86 % of STREAM Triad. The same program's shared tile, in which all threads work on one tile for several steps, is 1.21–1.30× its fused sweep at every thread count measured. The bus is therefore not what limits the engine on most of the grids measured: the fused sweep of the engine ran the 2.95 M-cell PEC box at 5 602 MC/s at 15 threads (Table 27), and it scales to 15 threads on every model but the 16.8 M-cell box. (At the 49 B per cell of the M5 traffic model that rate would be 274 GB/s, above STREAM's 259: a computed figure, which shows only that the model's byte count does not carry over to this grid on this chip.)

	STREAM, GB/s		`kb_idx.cpp`	`kb_tile.cpp`		
threads	Copy	Triad	fused, one grid	fused, a sub-grid per thread	one shared tile	ratio
1	135	131	825–829			
5	262	256	2 645–2 684	2 685 (132 GB/s)	3 486	1.30×
10	235	244	3 065–3 088	3 553 (174 GB/s)	4 417	1.24×
15	262	259	3 183–3 659	4 561 (223 GB/s)	5 512	1.21×

Table 22: M5 Pro: STREAM 5.10 (best rate of 20 passes, 100 M elements per array, OpenMP) and the two standalone kernels, in MC/s: `kb_idx.cpp` with `KB_FUSED=1` on one $209 \times 209 \times 180$ grid (7.86 M cells, the patch x5 size, 8 steps; the range of three runs), and `kb_tile.cpp` on the same grid size, 56 steps: its fused sweep with one sub-grid per thread (GB/s at 49 B per cell) and its best shared tile ($k = 8$, tiles of 17 x-lines, at every thread count; $k = 2$ with 15 x-lines falls to 2 337 at 15 threads, 0.51×). Every kernel configuration is bit-identical to its reference. `results/m5pro_speed_set_2026-11-12.txt`, sections 1 and 2.

Threads

Table 23 has cg31's CPU engine at five of the counts from 1 to 15 and with the automatic count (`numThreads=0`) on three models; the full sweep, every count from 1 to 15, is in the record. All three run fastest at 15 threads, 7.0× (notch x3), 5.1× (patch x5) and 7.5× (GCPW line) their rate on one thread. Patch x5 dips at 9 and 10 threads (3 608 to 3 618 MC/s against 3 922 at 8); this is not explained. The engine's own choice starts at 5 and tries 5, 7, 9, …, 15, then 3, then one count either side of the best (Section 4): it took 15 threads in all

four rounds of notch x3 and 9, 14, 9 and 14 on patch x5, where the dip misleads it; on the GCPW line it printed no choice, since the 15 000-step run (about 1 s) ends before the search does. On these runs of 1 to 2.7 s the automatic count runs at 0.81× (notch x3), 0.88× (patch x5) and 0.72× (GCPW line) of the best fixed count: the search costs roughly 0.3 to 0.5 s a run. That is the search of cg31; the start of patch 0041 removes most of that loss (Table 28).

model	1	5	8	10	15	auto	auto's choice	15 / 1	auto / best
notch x3, 5.25 M	690	2 611	3 639	4 102	4 860	3 948	15, 15, 15, 15	7.0×	0.81×
patch x5, 7.86 M	782	2 705	3 922	3 608	3 965	3 502	9, 14, 9, 14	5.1×	0.88×
GCPW line, 0.24 M	565	2 365	2 847	3 170	4 241	3 054	none printed	7.5×	0.72×

Table 23: M5 Pro: cg31's CPU engine at 1, 5, 8, 10 and 15 threads and with the automatic count, in MC/s, medians of 4 interleaved rounds (2 000, 1 200 and 15 000 steps); the GCPW row uses rounds 1 to 3 (see the text). Auto's choice: the count the engine printed in each round. Best: the best fixed count, 15 threads on all three. `results/m5pro_speed_set_2026-11-12.txt`, section 3.

Speed: stock, the CPU engine and the Metal engine

Table 24 sets cg31 against stock on the six models, on the CPU at 5, 10 and 15 threads and with the automatic count, and on the Metal diamond engine. Stock's column is its best rate over the seven thread counts measured on 2026-11-13 (Table 25); the cg31 columns and the Metal column are of 2026-11-12. Stock is slower at 10 threads than at 5 on four of the six models; cg31's CPU engine is fastest at 15 threads on all six. At 15 threads it runs the four models of up to about a million cells 9.33–14.06× as fast as stock at stock's best count and the two large models 4.27× (notch x3) and 2.73× (patch x5), paired round by round on 2026-11-13. The first version of this table, of 2026-11-12, took stock's best from its runs at 5 and 10 threads only and gave 9.6 to 17.3× and 4.39 and 2.74× (14.6, 9.6, 15.2, 4.39, 17.3 and 2.74 in the order of the table); that missed stock's best count on two models, and the stock rates of the two nights differ, so it overstated the gain (17.3× against 9.67× on the patch tutorial). The Metal engine leads on the two large grids (1.45× and 1.69× the CPU engine at 15 threads) and on the two tutorials (1.10× and 1.18×); on the two boards the CPU engine at 15 threads is level with it (the GCPW line) or ahead (the SMA launch, 1.10×).

	stock v0.37.0,	cg31, CPU engine				cg31,	CPU /	GPU /
model	best (threads)	5 thr	10 thr	15 thr	auto	Metal	stock	CPU
GCPW line, 0.24 M	305 (4)	2 213	3 096	4 123	3 034	4 085	14.06×	0.99×
SMA launch, 0.87 M	472 (5)	1 960	3 170	4 307	2 966	3 916	9.33×	0.91×
notch tutorial, 0.27 M	343 (5)	2 252	3 297	4 111	2 907	4 836	12.45×	1.18×
notch x3, 5.25 M	1 164 (10)	2 598	4 172	4 931	4 023	7 140	4.27×	1.45×
patch tutorial, 0.10 M	270 (2)	2 397	2 436	2 633	2 390	2 901	9.67×	1.10×
patch x5, 7.86 M	1 488 (10)	2 668	3 636	3 857	3 360	6 511	2.73×	1.69×

Table 24: M5 Pro: stock v0.37.0's best rate over 1, 2, 3, 4, 5, 10 and 15 threads (the count in brackets; the whole sweep is Table 25, measured 2026-11-13), cg31's CPU engine at 5, 10 and 15 threads and with the automatic count, and cg31's Metal diamond engine, in MC/s. The cg31 columns and the Metal column are medians of 5 interleaved rounds of 2026-11-12. CPU / stock: the median over the rounds of 2026-11-13 of cg31 at 15 threads over stock's best, which is not the quotient of the two columns shown (those are of different nights and step counts); GPU / CPU: the Metal engine over cg31's best fixed count, both of 2026-11-12, as before. The SMA launch is on the toolkit's current mesh. Most rows of 2026-11-12 ran beside 0.2 to 0.4 cores of other load, the patch tutorial's row with the automatic count beside 1.13; the notch tutorial's rows of that night ran beside 1.15 cores and patch x5's beside 1.27 (cg31 at 15 threads 3 662 to 4 083 MC/s). Every CPU run's probes are identical to stock's at 5 threads; the Metal rows differ by 1.7×10^{-7} to 9.4×10^{-7} relative. `results/m5pro_speed_set_2026-11-12.txt`, section 4 (with a dated correction at its end), and `results/cpu_items_cg33_2026-11-13.txt`, section 3.

	stock v0.37.0, threads							stock	cg31,	cg31 / stock
model	1	2	3	4	5	10	15	best (thr)	15 thr	best, paired
GCPW line	154	247	291	305	280	125	82	305 (4)	4 287	14.06× [13.86–14.52]
SMA launch	155	275	371	432	472	349	255	472 (5)	4 431	9.33× [9.29–9.63]
notch tutorial	135	236	305	342	343	170	112	343 (5)	4 258	12.45× [12.43–12.92]
notch x3	288	541	757	897	1 021	1 164	1 084	1 164 (10)	4 944	4.27× [4.20–4.31]
patch tutorial	254	270	252	213	166	61	36	270 (2)	2 615	9.67× [9.66–9.99]
patch x5	398	756	1 030	1 245	1 326	1 488	1 303	1 488 (10)	4 050	2.73× [2.71–2.76]

Table 25: M5 Pro: stock v0.37.0 at 1, 2, 3, 4, 5, 10 and 15 threads on the six models, in MC/s, medians of 3 interleaved rounds on 2026-11-13 (21:03 to 21:49) with half the steps of the speed table of 2026-11-12 (patch tutorial 5 000, patch x5 600, notch tutorial 8 000, notch x3 1 000, GCPW line 7 500, SMA launch 3 000 steps); cg31 at 15 threads ran in the same rounds. Last column: the paired ratio of cg31 at 15 threads to stock's best count, median and [smallest–largest] of the rounds. The other processes used a median of 26 to 47 % of one core per model; the highest single readings were 182 % (patch x5) and 171 % (notch tutorial). All probes are identical to stock's on one thread. `results/cpu_items_cg33_2026-11-13.txt`, section 3, part a.

Stock peaks at 2 to 5 threads on the patch tutorial (2), the GCPW line (4), the SMA launch and the notch tutorial (5) and at 10 on the two large models, and it slows down beyond: the patch tutorial runs 36 MC/s at 15 threads against 270 at 2, the GCPW line 82 against 305 at 4. The table of 2026-11-12 took stock's best from its runs at 5 and 10 threads only. That missed stock's best count on the patch tutorial (2) and the GCPW line (4), and its stock rates differ from those of this night (the notch tutorial at 5 threads 270 then and 343 now; the patch tutorial 152 at 5 threads then and 166 now), so its factors were too large.

The two boards end to end

Table 26 has the two boards end to end, as in Table 11, on this machine with the toolkit's defaults (the engine picks the threads, the probe end at −60 dB with a hold of 0.5, on the SMA launch the 0.05 mm mesh floor; `emskit fdtd e2e`, one run each, the other processes on 0.2 to 0.4 cores). Stock has no probe-end criterion, so its runs go to `max_time`, and the toolkit gave it its fallback thread counts, 4 for the GCPW line and 8 for the SMA launch (the GCPW count is stock's best of Table 25; the SMA launch's best of the counts measured there is 5). The factors of 25 to 32 therefore have three parts: the probe end stops these runs after 2.7× (GCPW line) and 2.4× (SMA launch) fewer steps, the faster engine supplies the rest (10.9× and 11.3× in MC/s), and the 32 is the opt-in Metal engine on one board, where the CPU engine reaches 27. In these runs the engine's own thread choice ran the GCPW line at 3 281 MC/s, where 15 fixed threads ran 4 123 in Table 24, and the SMA launch at 4 147 against 4 307: on a run of a few seconds the search costs about 20 %, on the 11 s SMA run about 4 %.

board	stock v0.37.0	cg31, CPU engine	cg31, Metal engine
GCPW line	94 217 steps (`max_time`) 75.8 s	34 300 steps (probe end) 2.8 s, 27×	34 300 steps (probe end) 2.4 s, 32×
SMA launch	118 475 steps (`max_time`) 281.2 s	50 320 steps (probe end) 11.0 s, 25.6×	50 320 steps (probe end) 11.2 s, 25.1×

Table 26: M5 Pro: the two boards end to end through the toolkit with its defaults, wall time of `FDTD.Run`, one run each; factor: stock's time over cg31's. `results/m5pro_speed_set_2026-11-12.txt`, section 7.

The valley and the scheduling switches

The allocation-placement valley of Section 4.6 is still there on this chip under macOS 27.2.1, and the allocator's stagger still removes it: on the GCPW line (0.24 M cells) cg31 runs 563 MC/s on one thread and 191 with `OPENEMS_NO_STAGGER=1` (0.34×), 2 482 and 915 at 5 threads (0.37×); stock runs 155 and 289. Only the rates were measured; where macOS 27.2.1 places the two field arrays was not tested. Of the scheduling switches on notch x3, the engine runs 9 to 13 % slower without the neighbour flags of cg29 (Section 4.14): with the spin barriers in their place (`OPENEMS_FLAGS=0`) it runs 0.87× at 15 threads (4 308 against 4 953 MC/s) and 0.91× at 10 (3 714 against 4 074). A spin of 50 µs on the flag waits at every thread count (`OPENEMS_FLAGS_SPIN_US=50`; by default the waits do not spin once the threads outnumber the `hw.perflevel0` cores) and the default QoS

class in place of user-initiated (`OPENEMS_QOS=0`) change nothing measurable on a quiet machine: 0.99× each at 15 threads, and 1.01× for the second at 10 (`results/m5pro_speed_set_2026-11-12.txt`, sections 5 and 6; 4 rounds each).

The tiled sweep

Table 27 repeats the timing of Table 20 on this machine, at 5, 10 and 15 threads. The fused sweep scales to all 15 cores, 1.4–2.1× from 5 to 15 threads, except on the 16.8 M-cell PEC box, where it is nearly flat (2 986 to 3 326 MC/s): that grid is memory-bound, and the tiled sweep lifts it to 4 752 (1.43×). This is the one model where the tiled sweep pays, and the one where it also gained most in Table 20. On the other five it loses (0.41–0.97×): the fused sweep keeps gaining from the ten Performance cores, while the tiled sweep peaks at 10 threads and falls at 15. Equal shares (`OPENEMS_BALANCE=0`) came within 3 % of the measured ones except on the 2.95 M-cell box at 15 threads (3 530 against 3 258, a wide range); the hook stages on one thread (`OPENEMS_TBLOCK_HOOKS=0`) were slower wherever they ran (patch x5 2 613 against 3 071, notch x3 1 647 against 2 141, the PML cube 674 against 1 531, all at 5 threads); and set depths beat `auto` at the same thread count only within the noise (3 % or less), except on the SMA launch with tiles of 65 x-lines at 5 threads (1 660 against 1 564, +6 %; still 0.40× of the fused sweep at 15 threads).

		fused sweep (cg31)			tiled sweep, `auto`			best tiled /
model	cells	5 thr	10 thr	15 thr	5 thr	10 thr	15 thr	best fused
patch x5 (Mur)	7.86 M	2 827	3 764	4 078	3 071	3 945	3 896	0.97×
notch x3 (PML at the x ends)	5.25 M	2 577	4 078	4 946	2 141	2 477	2 155	0.50×
cube with 10-cell PML	8.0 M	1 508	2 117	2 355	1 531	1 533	1 237	0.65×
PEC box	16.8 M	2 986	3 103	3 326	3 049	4 357	4 752	1.43×
SMA launch	0.87 M	1 912	3 154	4 106	1 564	1 692	1 543	0.41×
PEC box	2.95 M	3 197	4 850	5 602	3 353	3 789	3 258	0.68×

Table 27: M5 Pro: the fused sweep (cg31) and the tiled sweep (cg32d with `OPENEMS_TBLOCK=auto`) of the multithreaded engine, in MC/s, medians of three interleaved rounds, 2026-11-12 03:36 to 04:16. Best tiled: the fastest tiled setting of a model, `auto` at 10 threads on every model but the 16.8 M-cell box (`auto` at 15 threads); best fused: 15 threads on every model. The other processes used a median of 0.1 to 0.6 cores per model and 2.3 during the PML cube, whose fused rows ran beside 2.3 to 3.1 cores and its tiled rows beside 0.15 to 0.74, so its 0.65× sets a loaded fused best against a quiet tiled best; notch x3's tiled row at 5 threads ran beside 1.14. All 162 runs gave probes identical to the fused sweep's at 5 threads. The models are those of Table 20. `results/m5pro_tiled_sweep_2026-11-12.txt`.

On a grid of the patch x5 size the standalone kernel gains more from the shared tile than the engine does on this machine, and it still gains at 15 threads, where the engine does not. In `kb_tile.cpp` one shared tile runs 1.30×, 1.24× and 1.21× the kernel's fused sweep at 5, 10 and 15 threads (Table 22). In the engine, on patch x5, the tiled sweep runs 1.086× the fused sweep at 5 threads (paired), 3 945 against 3 764 MC/s at 10 and 3 896 against 4 078 at 15, so 0.97× best against best. What the engine loses against the kernel was not measured; the engine adds a barrier per task, the hook stages of the extensions and the operator tables, and one barrier per task across all threads may cost more as threads are added (a reading, not a measurement). The kernel ran at 07:14, three hours after the engine's runs, in the other block of the night. The verdict is the one of Section 9.6: the tiled sweep stays opt-in, never a default, and it gives the two boards nothing. A rule that switched it on by itself would need the grid to be far past the cache and the fused sweep to be memory-bound, which only a timed comparison at run time can tell; that is not built.

10.1 Patches 0041 to 0043: the thread search, packed dispersive hooks and the guard (cg33)

Three commits of 2026-11-13, written on 2026-11-12 and 2026-11-13, follow the tiled sweep, so the fork's head is now 13350ca (patches 0001 to 0043); the installed build is still cg31. Snapshot cg33a is 0ef186b with patch 0041, cg33c adds 0042 and cg33d adds 0043; they were built before the commits were made, so their banners read v0.37.0-58-g0ef186b, and cg33, rebuilt from 13350ca, reads v0.37.0-61-g13350ca (the same sources as cg33d). The timings are those of the two nights of 2026-11-12 and 2026-11-13 on the M5 Pro (`results/cpu_items_cg33_2026-11-13.txt`); none of these numbers is a number of the M5.

What changed. *Patch 0041, the start of the thread search.* With `numThreads=0` the engine times itself to choose a thread count. Up to patch 0040 it started at the cores of the first performance level (`hw.perflevel0`, 5 here) and went up in steps of 2: on notch x3 it timed 5, 7, 9, 11, 13, 15, 3 and 14 threads, eight windows, and on the M5 Pro, where the fused sweep scales to all 15 cores, it cost 4 to 19 % of a short run (cg31 columns of Table 28). It now starts at every core whose performance level is not named "Efficiency" (15 here; 4 on the M5, whose second level is named Efficiency, so nothing changes there), steps down by 2 while each step is the fastest so far, and then tries one thread either side of the best; on notch x3 that is 15, 13 and 14, three windows. `OPENEMS_TUNE_START=p` restores the old rule and reproduces cg31's sequence. *Patch 0042, packed dispersive hooks.* The positions of a Debye, Lorentz or Drude extension are put in groups of up to four positions of one x-line, so that no group is shared by two threads, and the state and coefficients of a group sit in one block of memory instead of about fifteen separate arrays. Four positions are updated at a time, with the same floating-point operations per position in the same order as the one-position loops. Only the multithreaded engine packs; the `sse-compressed` and basic engines and the Metal engine's CPU fallback keep the old paths on Cartesian meshes, which makes them independent references for the check. Packing is skipped when the blocks would need more than twice the memory of the field arrays, and `OPENEMS_ADE_PACKED=0` turns it off. The reason: on one thread a Debye cell cost about 39 ns and a Lorentz cell about 14 ns (rough figures of a loaded machine, not results). *Patch 0043, the guard.* The packed code sits under the same GCC/clang guard as the other vector code, with stubs for other compilers, since its vector types are a compiler extension. The limit of twice the field memory now counts all dispersive extensions of one engine together, and a refused packing prints one line at verbose level 1 or above with the bytes needed and the limit. No arithmetic changed.

Identity. The new build equals stock v0.37.0 in every check made; the machine was loaded during them, so only the verdicts count. `scripts/ade_check/ade_check2.py` runs six cases (dispersive, antenna, mixed, four-pole Debye, refused packing and the closed cylinder) through `FDTD.Run` against stock and cg31 and requires the packed hooks to be in use (with a guard against all-zero probes, Section 8.4): at 37 mesh lines and 3 000 steps 66 of 66 rows are identical, and with a source on the mesh (101 lines, 600 steps) the dispersive, mixed, antenna, four-pole Debye and refused-packing cases give 9 rows each; the closed cylinder with a dispersive ring is identical at 1 500 steps (it needs at least that many before its far probe is non-zero). The configurations were 1, 4, 7 and 15 threads, the unfused plan, the tiled sweep, packing off, `sse-compressed` and the basic engine. The reduced check of the command line against stock is identical on the six models (18 of 18 rows), as are the Debye check, the diamond check (72 of 72), the validation suite (15 of 15 twice), six models in eight configurations (48 of 48), the tiled-sweep check and the cylindrical multigrid example (43 datasets, no difference).

The thread search. Table 28 sets the automatic count against cg31 at a fixed 15 threads on four models, in four paired rounds on the night of 2026-11-12 (the other processes used 20 to 37 % of one core). cg31's search ran at 0.81 to 0.96 of the fixed rate, cg33a's at 0.97 to 1.00, and cg33a with `OPENEMS_TUNE_START=p` at 0.81 to 0.98, the values of cg31 within the spread. Patch 0041 therefore takes back most of the cost that Table 23 shows for cg31.

	cg31,	automatic count relative to cg31 at 15 threads		
model (steps)	15 thr	cg31	cg33a	cg33a, `TUNE_START=p`
GCPW line (34 300)	4 053	0.81×	0.97×	0.81×
SMA launch (50 320)	3 999	0.96×	1.00×	0.98×
notch x3 (2 000)	4 804	0.82×	0.97×	0.81×
patch x5 (1 200)	3 944	0.87×	0.99×	0.86×

Table 28: M5 Pro: the automatic thread count of cg31, of cg33a (patch 0041) and of cg33a with `OPENEMS_TUNE_START=p`, as a ratio to cg31 at a fixed 15 threads (the rate of that column in MC/s), medians of 4 paired rounds, 2026-11-12. Others' CPU 20 to 37 % of one core. The probes of every run are identical to the reference's. `results/cpu_items_cg33_2026-11-13.txt`, section 2, T1.

The packed hooks. Table 29 compares cg31 with cg33c on upstream's `bench.py`, interleaved, three repeats, on the night of 2026-11-12. The dispersive case runs 1.9× as fast at 1 M cells at both thread counts and 2.11× at 8 M cells and 5 threads, but only 1.14× at 15 threads (the fork's dispersive rate at 8 M cells peaks at 10 threads, see below); the antenna case, whose dispersive part is small, gains 3 to 10 %, the last two inside the spread. Upstream's dispersive case ran on zero fields in these runs (the finding of Section 8.4); the rates stand as rates, the probe comparison of those rows does not count. On the author's GCPW board, whose conducting

sheets run packed (1 793 groups), 34 300 steps at 15 threads in four rounds on the night of 2026-11-13 (the others 17 to 29 % of a core) gave 4 201.5 MC/s for cg31, 4 190.4 for cg33d (0.998× [0.995–1.002]) and 4 194.7 for cg33d with packing off (0.998× [0.987–1.013]): no change on the board, inside the spread.

case	size	threads	cg31	cg33c	ratio	spread
dispersive	1 M	5	695.7	1 347.2	1.94×	12.9 %
		15	1 276.1	2 424.1	1.90×	6.0 %
	8 M	5	621.2	1 310.5	2.11×	7.8 %
		15	1 182.8	1 344.7	1.14×	4.5 %
antenna	1 M	5	1 164.7	1 280.6	1.10×	4.5 %
		15	2 500.1	2 708.8	1.08×	8.7 %
	8 M	5	1 436.5	1 524.0	1.06×	3.5 %
		15	2 383.6	2 452.2	1.03×	3.2 %

Table 29: M5 Pro: cg31 and cg33c (packed dispersive hooks) on upstream's `benchmark/bench.py`, interleaved, median of 3 repeats, in MC/s, 2026-11-12 21:40; the spread is the tool's. The dispersive rows ran on zero fields (Section 8.4); the antenna case has its own source. Others' CPU: a median of 22 % of one core. The probes of every row are identical. `results/cpu_items_cg33_2026-11-13.txt`, section 2, T2b.

Against master's cpu engine, with a source on the mesh. Upstream's own benchmark has the flaw described in Section 8.4: at its default sizes its empty, PML and dispersive cases have no source. On the night of 2026-11-13 `bench.py` was therefore run on meshes of 101 and 201 lines, so that the dipole sits on a mesh line, with master's `cpu` engine (1d710c2; A) against the multithreaded engine of cg33d (B), run for run, three repeats, the other processes a median of 20 % of one core (Table 30). The fork is ahead at 1 M cells, 1.4 to 2.5× on the empty and PML boxes, and level at 5 threads and 1.3 to 1.45× ahead at 10 and 15 on the dispersive case. At 8 M cells it is level to 14 % ahead on the empty and PML boxes at 10 and 15 threads (0.93× on the PML box at 5), and it is behind on the dispersive case, 0.73, 0.70 and 0.54× master's rate; there the fork peaks at 10 threads (1 296, 1 540 and 1 338 MC/s), and its operator setup takes 2.5 to 5.5 times master's. The spread of the rows is 0.8 to 6.5 %, except the dispersive 1 M-cell row at 10 threads (12.4 %). Master's `cpu` engine is a different algorithm (one cache-sized tile that all threads share, several time steps per pass), and its probes differ from the fork's by 5×10^{-6} to 3×10^{-5} relative on these cases, while the fork equals stock v0.37.0 bit for bit.

	5 threads			10 threads			15 threads		
case, size	master	fork	B/A	master	fork	B/A	master	fork	B/A
ideal, 1 M	2 293	3 258	1.42	2 338	5 138	2.20	2 524	6 346	2.52
ideal, 8 M	2 869	2 932	1.02	3 568	3 690	1.03	3 671	3 985	1.09
pml, 1 M	1 023	1 651	1.61	1 135	2 454	2.16	1 297	3 133	2.42
pml, 8 M	1 763	1 630	0.93	2 139	2 283	1.07	2 238	2 561	1.14
dispersive, 1 M	1 400	1 343	0.96	1 601	2 080	1.30	1 679	2 436	1.45
dispersive, 8 M	1 771	1 296	0.73	2 201	1 540	0.70	2 460	1 338	0.54

Table 30: M5 Pro: master's `cpu` engine (openEMS 1d710c2, CSXCAD fdff4b5; A) against the fork's multithreaded engine (cg33d; B), in MC/s, `bench.py` interleaved, median of 3 repeats, with a source on the mesh (101 and 201 mesh lines), 2026-11-13 21:34; B/A as the tool printed it. Others' CPU: a median of 20 % of one core. The probes differ between A and B by 5×10^{-6} to 3×10^{-5} relative. `results/cpu_items_cg33_2026-11-13.txt`, section 3, part c.

The control of that night kept upstream's own meshes (200 lines, no source, all probes zero): at 8 M cells and 15 threads the empty case ran 3 870 MC/s on master and 4 071 on the fork, the dispersive case 2 437 and 1 344, against 3 671 and 3 985 and 2 460 and 1 338 with a source. The rates with and without a source agree within the spread of the rows, so the rates of the common benchmarks of Section 8.4 stand, on this machine. The three cases that have their own sources (inhomogeneous, antenna, cylindrical) were run against master on the night of 2026-11-12 with cg31 (a different build from cg33d; 5, 10 and 15 threads, B/A): inhomogeneous 1.56, 2.18 and 2.43 at 1 M cells and 0.98, 1.11 and 1.19 at 8 M; antenna 1.41, 1.90 and 2.18, and 0.92, 1.04 and 1.14; cylindrical 1.53, 2.15 and 2.57, and 1.04, 1.17 and 1.30; master's probes differ from the fork's there by 3×10^{-7} to 10^{-5}.

The tiled sweep at 8 M cells. Table 31 repeats the comparison of the fused and the tiled sweep of Section 9.6 on the 8 M-cell cases with a source, on cg33d, in the same night block. On the empty box the tiled sweep with `OPENEMS_TBLOCK=auto` is ahead of the fused sweep, by 8, 22 and 15 % at 5, 10 and 15 threads (4 450 and 4 569 MC/s at 10 and 15, against 3 568 and 3 671 for master's `cpu` engine in the run of the table above; the two were not run side by side). A fixed depth of 4 and a tile width of 31 lines are worse or equal. On the dispersive case every tiled variant is slower than the fused sweep, by 19 to 59 %: the tiled sweep does not close the gap to master's engine there. It stays opt-in.

	ideal, 8 M cells			dispersive, 8 M cells		
	5 thr	10 thr	15 thr	5 thr	10 thr	15 thr
fused sweep	2 954	3 647	3 969	1 294	1 518	1 324
tiled, `auto`	3 183 (+8 %)	4 450 (+22 %)	4 569 (+15 %)	1 053 (−19 %)	1 106 (−27 %)	737 (−44 %)
tiled, $k = 4$	2 846 (−4 %)	4 059 (+11 %)	3 918 (−1 %)	988 (−24 %)	1 066 (−30 %)	543 (−59 %)
tiled, `auto`, $W = 31$	3 020 (+2 %)	4 009 (+10 %)	4 048 (+2 %)	914 (−29 %)	962 (−37 %)	1 024 (−23 %)

Table 31: M5 Pro: the fused sweep and three tiled settings of the multithreaded engine (cg33d, `OPENEMS_TBLOCK=auto`, `=4`, and `auto` with `OPENEMS_TBLOCK_W=31`) on upstream's ideal and dispersive cases at 8 M cells with a source on the mesh, in MC/s, `bench.py` interleaved, median of 3 repeats, 2026-11-13 21:42; in brackets the change against the fused sweep of the same thread count. Others' CPU: a median of 20 % of one core. The probes are identical to the fused sweep's in every row. `results/cpu_items_cg33_2026-11-13.txt`, section 3, part d.

What is open. The 8 M-cell dispersive case is the open CPU item: the fork is at 0.54 to 0.73 of master's `cpu` engine there, and the packed hooks (Table 29) and the tiled sweep do not close the gap. The fork's fused sweep on that case peaks at 10 threads and loses at 15, and its dispersive setup costs 2.5 to 5.5 times master's; why the rate falls at 15 threads was not examined.

What this set shows

On this chip the fork's CPU engine is fastest at 15 threads on every model measured (on the 16.8 M-cell box and the patch tutorial it gains only about 1.1× from 5 to 15 threads), while stock is slower at 10 threads than at 5 on four of six and peaks at 2 to 5 threads on four models, so the CPU engine runs 2.7 to 14.1× stock's best; the automatic thread count of cg31 costs 0.3 to 0.5 s a run, which matters on runs of a few seconds, and patch 0041 removes most of that cost (Section 10.1); against upstream master's new `cpu` engine the fork is ahead at 1 M cells, level to 14 % ahead at 8 M cells on the empty and PML boxes (0.93× on the PML box at 5 threads) and behind on the 8 M-cell dispersive case (Section 10.1); the Metal engine leads on the large grids and the tutorials and is level or behind on the two boards; the allocation-placement valley persists under macOS 27.2.1 and the stagger still removes it; the neighbour flags still pay; and the tiled sweep pays only on a memory-bound grid with little extension work. Bit identity held for every CPU run, as on the M5, and for the three patches of Section 10.1.

11 Positioning: fdtdKit and other electromagnetic solvers

fdtdKit makes one open-source solver faster on one class of machine; it does not widen what openEMS can model. This section places it against the tools an RF engineer would otherwise use, so that the speed-ups of Section 8 are not read as a claim they do not make. No commercial solver was run for this report; the comparisons with them below are qualitative or, where a number appears, a roofline bound [6] computed from published hardware figures, not a measurement.

Commercial solvers. Industrial RF design runs mostly on commercial packages. Ansys HFSS solves Maxwell's equations in the frequency domain with the finite-element method on an automatically and adaptively refined tetrahedral mesh [9]; this suits high-Q resonators, curved geometry and fine features that a Cartesian staircase mesh resolves poorly. CST Studio Suite offers, among several solvers, a time-domain finite-integration solver closely related to FDTD, with GPU acceleration [10]; Remcom XFdtd is an FDTD solver with GPU acceleration as well [11]. Beyond the solvers, these packages bring CAD import, automatic meshing with convergence control, parameter optimisation, a graphical front end, vendor validation and support. fdtdKit touches none of this: model setup, meshing and post-processing are openEMS's and the author's scripts.

Raw speed. Section 5.4 shows that on the M5 the CPU engine runs close to what the cores can draw from memory on large grids (on the M5 Pro only the largest grid measured, 16.8 M cells, was held back by it, Section 10). A commercial FDTD/FIT solver on a data-centre GPU is therefore bounded far higher: an NVIDIA H100 SXM has 3.35 TB/s of memory bandwidth [12], which at the 74 bytes per cell of an unfused update is a roofline bound of about 45 000 MC/s, some 20 times the M5's bound for the same traffic (Section 5.4). Real codes reach a fraction of their roofline, and we have no measurement of any commercial code, but on large models fdtdKit should not be expected to match such a system. Its comparison class is a laptop or desktop without a licence.

Open-source solvers. Among free tools, MEEP [13] is the reference FDTD code for photonics and gprMax [14] for ground-penetrating radar; openEMS is the usual choice for PCB-scale RF and antenna work, largely for its Matlab/Octave and Python interfaces. fdtdKit keeps openEMS's interfaces unchanged and its results bit for bit identical to stock for the same number of time steps (Section 3), so existing scripts gain the speed-up without modification (for SAR the two Python modules must be rebuilt against the new libraries, `BUILD.md` section 6). That holds for the CPU engines, which stay the default. The opt-in Metal engine (`--engine=metal-diamond`, Section 6) is bit-identical to the multithreaded CPU engine run with `OPENEMS_FTZ=1`, not to stock, because Apple's GPU flushes subnormal floats to zero; against stock its port probes differ by 1.7×10^{-7} to 9.4×10^{-7} relative (Section 3.3). A script that ends on the energy end criterion can stop at a different step than stock v0.37.0, because the fork's base checks that criterion on a different schedule (upstream commit eb1c5ee, Section 3.3); for the same number of steps the records are identical.

Prior work on FDTD and openEMS performance. That low-order FDTD is bound by memory bandwidth on multicore CPUs, and that the remedy is to reuse data in cache, is well established: Cessenat reports a more than twofold reduction from cache-aware restructuring of the Sophie code [15], and temporal (time-space) tiling of FDTD and of stencils in general has been studied extensively [16, 17]. For openEMS itself, an openEMS contributor (GitHub user `biergaizi`) has documented the engine's synchronisation and memory-traffic bottlenecks and developed an experimental tiling engine, reported bit-identical to the multithreaded engine, with speed-ups of about 2–6× reported in the project's discussion forum [18, 19]; at the time of writing that work is not merged. fdtdKit's CPU engine reaches a comparable range by a different route (fewer barriers, a leaner inner loop, a fused single-pass sweep, the allocation stagger and the dispersive-material hooks) without temporal tiling. Two time steps per sweep gained 10–14 % in our first standalone kernel, which ran the stock arithmetic (invariants hoisted, the 12 coefficient tables) as a fused sweep on one independent sub-grid per thread (project log; Section 12); with the current engine's loops in the same set-up, trapezoidal blocking of two to four steps per sweep (after MrHighVoltage's fork, next paragraph) gained at most 6 % as an upper bound, measured without halo lines or extension hooks (`results/cpu_items_cg29_2026-11-05.txt`, section 2), so the CPU engine's default sweep stays unblocked. Upstream master's new `cpu` engine blocks with one tile for all threads (Section 9.4 has what that is worth on this machine), and since 2026-11-11 the fork has that form as an opt-in tiled sweep, which at 4 threads is ahead of the fused sweep at its best on two large grids and behind on four other models (Section 9.6). Wavefront or diamond blocking beyond that sweep's trapezoids along x was not tried on the CPU. The Metal engine of Section 6 does block over time steps, in diamond tiles on the schedule of another contributor's Metal engine (next paragraph). Both fdtdKit's CPU factors and the 2–6× above are measured against stock openEMS, whose sweep is unfused, and the two approaches are not exclusive. The FAQ [19] also advises macOS users to build openEMS with GCC rather than clang. The stock baseline of this report is the default clang build, and openEMS was not built with GCC; in the standalone kernel at one thread GCC 16 runs the stock inner loop 1.16–1.39× faster than Apple clang and the rewritten one at about the same speed (the inner-loop item of Section 4). fdtdKit was also built on upstream's GPU work: the pre-merge `gpu-engine` branch at commit 7bb294f (2026-10-12), whose GPU engine Sean Mollet added (commit bf30fa4, 2026-10-08) with follow-up work by Thorsten Liebig, and whose gather and asynchronous dumps the fork uses (Section 5.3). The branch at 7bb294f alone runs the patch tutorial 1.13× faster than stock (both at their best), through its asynchronous dump writer, so fdtdKit's 6.9× there (cg14, Table 9) includes about 1.13× from upstream; on the notch tutorial, the GCPW line and patch x5 it runs at 0.98–1.03× stock (Section 8). Upstream has since merged a reworked GPU engine, with a region-based gather, into master (c3a956b) and deleted the branch; the merged engine was measured here on 2026-11-04 (below).

Concurrent work on openEMS speed. In the weeks before and during this work, other contributors posted speed work on openEMS, much of it, like this one, written with AI assistants. The list gives what their authors posted, as of 2026-11-04: the numbers in it are their own, from other machines and models, and do not compare directly with Table 9. When the list was written none of this work had been built or

run here; later that day two of the GPU engines were, on the six benchmark models (the paragraph after the list). The CPU pull requests, the fork of Discussion #492 and the SYCL backend were not built or run, but cg29 builds on three of them, credited in Section 4.14: it takes the UPML part of #244 by Noriaki Takashima (nori-proto), and it follows the strict-order check of #245 by Roel Budé (unsettledroell404) and the lumped-RLC state slots indexed by the time step that MrHighVoltage proposed in Discussion #492.

- **A Metal engine with diamond blocking** (luikore, openEMS pull request #228, opened 2026-09-29 [20]), not merged. It advances a space-time diamond of four time steps per block; fdtdKit's Metal engine (Section 6) follows its tile schedule, without using any of its code. On a uniform grid with one dielectric box and PEC walls it reports 5 752 and 4 042 MC/s on an Apple M4 Pro (3.0 and 17.0 million cells) and 14 734 and 11 532 MC/s on an M5 Max, 1.9–6.3× the best multithreaded CPU run on the same machine. Its own unfused path ran at 2 216 MC/s against 2 455 for the CPU at 3.0 million cells on the M4 Pro, in line with Section 5.4: on these machines the blocking, not the GPU as such, carries the gain. Its author calls the uniform grid Metal's best case and expects real PCB geometry to lose much of the speed-up. The engine is Cartesian only, a model with ADE materials (which include openEMS's conducting sheets, Section 4) aborts on its default path, and its fields are checked against the SSE engine within relative-L2 limits, not bit for bit.
- **Upstream's GPU engine** (Sean Mollet [21]): the Metal and HIP engine that this work measured on the pre-merge branch (Section 5.4) went into master as c3a956b on 2026-10-17. The merged version was measured here on 2026-11-04 (below).
- **CPU engine changes with identical results.** Pull request #231 (ismailakdag, opened 2026-10-05 [22]) lets the multithreaded engine skip the barriers of extension hooks that have nothing to do, the idea behind this work's hook plan, and walks the SSE UPML loops along z; its outputs are bitwise identical on 24 setups, and it reports 1.2× (Mur) to 5× (PML-heavy) at one thread on a Ryzen 9 7900X and 1.30–1.67× on an M5 Pro on an earlier base. A fork by MrHighVoltage (Discussion #492, 2026-10-26 [23]) adds an AVX2 engine (x86 only), trapezoidal temporal blocking on the CPU and the GPU, barrier elision (552 to 189 barriers per time step), a thread tuner that scores real time steps, an asynchronous dump writer and a Vulkan GPU engine; it reports 1.7–4.45× multithreaded against upstream's SSE engine on x86 machines, with results bit-identical or differing by float32 rounding. Smaller pull requests multithread the Lorentz and Debye updates (#245 [24]) and the UPML and Debye coefficient setup (#244 [25]), both near items of Section 4, and shorten the mesh-delta lookup (#230 [26]). A SYCL backend for Intel GPUs (#243 [27]) fuses the voltage and current steps where the extensions allow.
- **Upstream's plan** (Thorsten Liebig, Discussion #495, 2026-10-31 [28]): first a refactoring that separates the mesh geometry from the coefficient storage, with the speed pull requests on hold until it is done; then GPU kernels written once and shared by all backends. Fused voltage and current steps, asynchronous dump writing and temporal blocking are to go into that shared code once, temporal blocking last. Speed claims are to be measured with the `benchmark` tool of master against the existing engine on the same machine.

Two of these engines on the M5. On 2026-11-04 upstream master at 5fecbb5 (2026-11-02), with its CPU engine and `--engine=gpu` (the Metal backend of c3a956b), and luikore's pull request at 55ce11b, with its diamond path and its legacy two-dispatch path (`OPENEMS_METAL_FUSED_PIPELINE=0`), were built and run on the six benchmark models next to stock and cg18 (`results/others_2026-11-04/README.md`). The runs used this report's `bench.py` (fixed step counts, the faster of two runs, port probes and S-parameters compared with stock), not master's `benchmark` tool that Discussion #495 asks for. Other work shared the Mac (a KiCad render loop from another session at about 950 % CPU in bursts, Spotlight indexing), the other processes' GPU use was not measured, and from about 06:00 another session was developing fdtdKit's Metal engine and may have used the GPU in the later runs, so the numbers are rough. The SMA launch there has the 0.871 M-cell mesh of the toolkit's current default, not the 1.071 M cells of Table 9. luikore's diamond path refused all six models before the first step, with the message that the Mur extension (on the GCPW line the conducting-sheet extension) "has not migrated to the diamond wavefront"; its legacy path, which that message reserves for diagnostic runs, ran them at 172–884 MC/s, below cg18 on every model. With every Mur face replaced by PEC (speed-only test cases, whose S-parameters are not those of the real models; the GCPW line is left out, its sheets being refused too), the diamond path ran patch x5 and notch x3 at 3 128 and 3 734 MC/s, 2.00 and 1.86 times cg18's CPU engine, the notch tutorial at 1.47, the SMA launch at 1.05 and the patch tutorial at 0.39 times, and at 2.1 to 6.4 times its own legacy path: on the M5, too, the blocking carries the gain. Master's GPU engine ran the six models as they are at 0.42 to 1.06 times cg18: about level on patch x5 (1 529 against 1 510 MC/s) and on the SMA launch (1 084 against 1 019), 0.82 times on the GCPW line, and slowest relative to cg18 on the patch tutorial (782 against 1 856). On the PEC variant of the notch tutorial (0.270 M cells) it ran at 2 374 MC/s against cg18's 2 053, the faster of its two runs (1.82 s, other processes at 43 % of a core in the second before it; the other run at 1 452 MC/s; `notch_s1_n16000_master-nomur.txt` in that

folder), above the roughly 2 000 MC/s that 74 B per cell allow from the M5's memory (Section 2); whether its kernels fuse the two half steps or this small model stays in the GPU's caches was not examined. cg18 was identical to stock in every run, and so were master's and luikore's CPU engines, except master's on the SMA launch (probes 1.5×10^{-4} relative, S_{11} 3.4×10^{-3} dB), a change in master since v0.37.0 that its GPU engine shows too. Apart from that, the GPU engines differed from stock by 1.7×10^{-7} to 1.3×10^{-6} in the probes and by at most 2.6×10^{-4} dB in the S-parameters (notch tutorial; within 3×10^{-5} dB elsewhere): master's on five models by the same printed figures as fdtdKit's Metal engine (Section 3.3), and luikore's two paths, on the PEC variants, by the same figures as each other. None of them was compared with a CPU run with `OPENEMS_FTZ=1`. fdtdKit's Metal engine was not run on the PEC variants, and luikore's diamond path refused the models as they are (its legacy path ran them), so on these models the two diamond engines have not been compared head to head. They were on 2026-11-11, on luikore's own benchmark model and with master's engines beside them (Section 9.4, Table 19): fdtdKit's Metal engine (cg31) ran 3 981 and 3 309 MC/s at 3.0 and 17.0 million cells, luikore's 3 952 and 2 829 and master's `gpu` engine 1 638 and 1 566.

Against this background the parts of fdtdKit that we have not found elsewhere are narrower than its speed-ups: the acceptance rule applied throughout (bit identity on six models, with the closed-form validation suite and its negative controls; Sections 3 and 14), the allocation-placement valley of the macOS allocator, which slows stock openEMS too (Section 4.6), the measured ceiling of one base-M5 memory bus for the CPU and for the unfused GPU engine of the fork's base (Section 5.4), with a fused and a diamond-blocked Metal sweep measured against it, and a diamond-blocked Metal engine that runs all six models on the GPU with every extension they use (Mur and UPML boundaries, lumped ports, conducting sheets; Drude/Lorentz and Debye materials as well), bit-identical to the CPU engine run with `OPENEMS_FTZ=1` (Section 6).

Where fdtdKit stands. For printed-circuit-board RF models on the CPU of Apple silicon, fdtdKit is to our knowledge the fastest configuration of openEMS reported that keeps stock's results bit for bit: 3.2–6.9× stock on the four models of up to about a million cells with snapshot cg14 (Table 9, the SMA launch on its 1.071 M-cell mesh; Table 10 has cg29 and Table 18 the installed build, cg31), and, with the modelling levers of Section 7, 15× end to end on the SMA launch with the toolkit's defaults since 2026-11-04 (the 0.05 mm mesh floor and a hold of 0.5, against stock's full run on the 0.02 mm mesh; cg18, Table 11), or 5.8× with the toolkit's defaults until 2026-11-04 (0.02 mm and a hold of 1). On the GPU its opt-in Metal engine keeps the CPU engine's results bit for bit when the CPU run also flushes subnormals, runs all six models on the GPU with every extension they use (Section 6), and runs notch x3 and patch x5 at 3 226 and 2 917 MC/s (cg28, medians of five paired rounds on a quiet Mac: other processes on 0.5–1.3 cores and 2–4 % of the GPU; `results/diamond_engine_cg28_tuner_2026-11-04.txt`), where cg18's CPU engine reached 2 046 and 1 716 at best on 2026-11-03 (`results/*_cg18.txt`); with cg31 it runs them at 3 702 and 3 100 (Table 18). Of the other GPU engines measured on the M5, luikore's diamond path refused all six models and master's ran them at 0.42 to 1.06 times cg18's CPU engine; on luikore's own benchmark model cg31's Metal engine was level with luikore's at 3.0 million cells, 1.17× as fast at 17 million, and 2.1–2.4× as fast as master's `gpu` engine (Table 19). Whether diamond blocking on the GPU keeps its gain on a base M5 and on PCB models is answered for fdtdKit's engine: on the M5 the fused, unblocked Metal sweep, with its best tile, ran notch x3 and patch x5 at 0.58 and 0.65 times the blocked one (paired runs, other processes on 2.5–4 cores) and the SMA launch (0.871 M cells) and the GCPW line at 0.74 and 0.77 times (the faster of two unpaired runs, other processes on 3.3–5.6 cores; `results/fused_sweep_k1_2026-11-04.txt`).

What is not shown: luikore's reported 4 042–14 734 MC/s on the M4 Pro and the M5 Max have no like-for-like fdtdKit figure, because fdtdKit was not run on luikore's machines (on luikore's model it was, on this M5: Table 19), and on the six models its diamond path ran here only on the PEC variants, next to cg18's CPU engine and not next to fdtdKit's Metal engine; and master's GPU engine, about level with cg18's CPU engine on patch x5 and the SMA launch, was not run on the six models next to cg29, cg31 or fdtdKit's Metal engine (the same table has it on luikore's model). fdtdKit closes none of the functional gaps listed above; closing some of them (Cartesian sub-gridding, conformal conductors, automatic mesh convergence, PCB import) is the natural continuation and would need a different acceptance rule than bit identity, since stock openEMS has no reference to be identical to.

11.1 Outlook: clusters of Apple-silicon machines

This subsection is an outlook: no cluster was built or run. Apart from the memory figures, which were measured for this report on small boxes, and one STREAM figure of the M5 Pro (Section 10), its numbers are published hardware figures or arithmetic on them. The result of Section 5.4 (on the M5 the CPU engine on large grids sits near the bandwidth of one memory bus; on the M5 Pro only the largest grid measured, 16.8 M cells, did, Section 10) points to the natural way to scale for grids that do not fit: add buses. Every Mac mini or Mac Studio brings its own unified memory and its own bus, and since macOS 26.2 machines

connected by Thunderbolt 5 can exchange data by remote direct memory access (RDMA) [29], over links with a nominal rate of 80 Gb/s (real throughput is lower). Published figures for current machines: a Mac mini with M5 Pro has up to 64 GB at 307 GB/s [30]; a Mac Studio with M5 Ultra up to 512 GB at 1.2 TB/s [31]. On a 24 GB Mac mini with M5 Pro, STREAM's Triad measured 259 GB/s at 15 threads, and the five Super cores alone reached that rate (Section 10).

Parameter studies. Two simulations side by side on one machine gain nothing, because they share the bus (Section 12). On separate machines they do not, so a sweep or an optimisation run spread over n machines should scale close to n with no change to the engine, only a job dispatcher. This is the cheapest use of a cluster and the one most RF design work needs.

One large model. openEMS had an MPI engine for domain decomposition up to v0.0.36, but it had not compiled for years (it used the C++ MPI bindings that MPI-3 removed), and upstream removed it before v0.37.0 (commit 5261463, 2026-09-16). A model split across machines therefore needs a new domain-decomposed engine, which could carry the fork's engine changes from the start. One fork has tried an x-slab split of one model across two GPUs with host-staged halo exchange (td-krzysiek, described in Discussion #495 [28]); upstream's maintainer replied there that multi-device engines are not a priority and that optimising the mesh usually pays more. The arithmetic is favourable. During the time loop a plain cell holds about 28 bytes (six float field components and a 4-byte operator index), but the operator setup peaks near 100 bytes per cell, about 124 with a field-dump or nf2ff box, and on a Mac the two share the same memory (measured for this report with snapshot cg18 at 2 threads on boxes of up to 64^3 cells; `results/memory_per_cell_2026-11-03.txt`). cg29 allocates the operator index once per z-vector of four cells instead of once per cell, about 3 bytes per cell less (patch 0017; not measured on these boxes). Memory alone would therefore allow about four to five billion cells on one 512 GB Mac Studio, before the operating system's share; but the arrays use 32-bit sizes, and the code refuses meshes above about 4.29 billion nodes, or 1.43 billion with field dumps, an nf2ff box, conducting sheets or the basic engine or `--engine=gpu`; `--engine=metal-diamond` keeps a model on the GPU only below 2^{32} floats per field array (about 1.43 billion nodes) and up to 65 535 lines in x or y, and runs the one-thread CPU loops above that (Section 6). A setup that does not hold the whole mesh at once, and 64-bit sizes, would move the limit towards the run state. At equal bytes per cell, 512 GB is 6.4 times the 80 GB of an H100. With the domain split into slabs, each half step exchanges one plane of tangential fields per neighbour: for a 1000^3-cell model on four machines that is about 8 MB per interface and half step, 16 MB per step, about 1.6 ms at the nominal link rate (more at the real throughput), against roughly ten milliseconds of update per step if each machine ran at the bandwidth roofline. With the exchange overlapped with the interior update, four M5 Ultra machines would offer an aggregate bus of about 4.8 TB/s, above the 3.35 TB/s of one H100 [12]. Two caveats: whether the CPU cores of an Ultra-class chip can draw on the whole bus, and how far the Metal engine gets there, have to be measured (on the M5 a fused, unblocked Metal sweep stayed near the fused CPU sweep, and only the diamond-blocked one ran the large models clearly faster, Section 6); and the bit-identity rule carries over naturally, since a domain decomposition need not change any cell's arithmetic, so a distributed run can be checked against a single-machine run to the bit.

12 What did not work

Recorded so that the same ground is not covered twice on this class of machine. The numbers in this list are measurements from the project log (not in `results/`) unless a result file is named. The SMA launch is on its 1.071 M-cell mesh in the project-log numbers and on its 0.871 M-cell mesh in the result files of 2026-11-04 and 2026-11-05 (Table 1).

- **Two simulations side by side** for parameter studies. GCPW alone at 4 threads 1 785 MC/s; two concurrent 4-thread runs 967 + 957; two 5-thread runs 864 + 857; SMA one 8-thread run 1 397 versus two 4-thread runs 691 + 692. The bus is shared; pairs give −4 to +8 %, inside the noise. Studies stay sequential.
- **Coarser fused blocks** (`OPENEMS_FUSED_BLOCK=k` x-lines per block): GCPW +4 % (noise), notch tutorial 0 at 4 threads and −7 % at 8, SMA −14 % (block 4) and −26 % (block 16) at 4 threads. The L2 reuse of the line behind is lost with wider blocks. The knob stays at 1.
- **CPU field snapshots** so dumps could be evaluated in the background: −3 %; the 12 small nf2ff dumps evaluated concurrently: 0 %.
- **Two or more time steps per sweep, with a sub-grid per thread** (k-step temporal blocking on the CPU; the form with one tile for all threads came later, end of this item): +10–14 % for $k = 2$ in the standalone kernel of 2026-10-28 (`scripts/kernel/kb_tb.cpp`: the stock arithmetic with its invariants hoisted and the 12 coefficient tables, a fused sweep, $k = 1$, against $k = 2$, on one independent sub-grid per

thread), while every extension hook would need rewriting; not pursued then. Measured again for cg29 as an upper bound (standalone kernel `kb_tbk`, not in `scripts/`: the fork's fused IJKN loops, one independent sub-grid per thread, so no halo exchange and no extension hooks, bit-identical for every k; 6 rounds, ratio to $k = 1$): on a 7.9 M-cell grid 1.038, 1.032 and 0.958× for $k = 2$, 3 and 4 at 4 threads and 0.990–1.020× at 8 (0.5 cores of other load); on a 5.25 M-cell grid 0.971–0.976× at 4 threads and 1.055–1.059× at 8 (1.2 cores). At most +6 %, against the +10–14 % above; why the fork's loops gain less was not examined. A real version (halo lines per thread, every extension's line hooks per sub-step) would gain less. Not built (`results/cpu_items_cg29_2026-11-05.txt`, section 2). On the GPU, diamond blocking is the fork's Metal engine (Section 6, with the tile schedule of luikore's engine, PR #228 [20]). Both tests gave every thread a sub-grid of its own, and they stand as the record of that design. Upstream master's `cpu` engine blocks with all threads in one cache-sized tile, which is worth 1.15–1.49× to it on this machine (Section 9.4). That form was built on 2026-11-11 as an opt-in tiled sweep (patch 0040, Section 9.6): at 4 threads it ran patch x5 and a 16.8 M-cell box 1.08 and 1.23× as fast as the fused sweep at its best, and four other models at 0.71–0.92×, so it is a switch and the default sweep stays unblocked. Wavefront or diamond blocking [17, 16] beyond that sweep's trapezoids along x was not tried on the CPU.

- **Compiler flags**: `-mcpu=apple-m4` gives an identical instruction stream; link-time optimisation (LTO) was not tried; `-ffast-math` is ruled out (not bit-identical); GCC 16, measured at one thread in the standalone kernel only, runs the stock inner loop 1.16–1.39× faster than Apple clang but the rewritten one at about the same speed (and not bit-identical; `results/kernel_gcc_vs_clang_2026-11-03.txt`, the inner-loop item of Section 4). Flush-to-zero: no speed effect on Apple cores (Section 5.4).
- **fp16 field storage**: needs field scaling (late-run currents are 10^{-6} A against a half-precision normal range ending at 6×10^{-5}), touches the 33 Metal kernels and the host mirror of the base's GPU engine, or, in the Metal diamond engine, its one kernel and the CPU code that reads the same field arrays (probes, dumps, the CPU fallback), and cannot be bit-identical. Not started.
- **A Swift rewrite of the engine**: the prototype runs at the cell rate of upstream's Metal engine and of the CPU engine of the time (1 670 versus 1 638 versus 1 562 MC/s on 201^3, the CPU on snapshot cg9). Its diamond mode (`--diamond K`, 2026-11-04) reached 2 445.7 MC/s at $K = 4$ on 201^3 against 1 540.0 unblocked (one run each on a loaded machine, `results/diamond_swift_2026-11-04.txt`) and became the prototype of the Metal diamond engine (Section 6). The rewrite itself stays a side project.
- **The default thread count by wall-clock ramp**: replaced, not tuned.
- **The second review's CPU items that gained nothing** (measured for cg29; paired A/B in one build, median ratio; `results/cpu_items_cg29_2026-11-05.txt`, section 2; item numbers of Table 32): a cache of the automatic thread count per model (item 3; at real run lengths GCPW 1.021× [0.893–1.044], SMA 1.002×, 5 rounds, 0.5–0.8 cores: the search costs about 1.5 s of a 10–80 s run and finds the same count); a per-thread QoS split between P and E cores, a 200 μs spin and 5 ms slab balancing (item 4; at 8 threads on five models 0.949–1.357×, 0.907–1.122× and 0.991–1.008×, 5 rounds, 0.3–0.5 cores; with the flags the QoS split lost on three of six models at the automatic count, 0.886–0.948×: gains on two or three models, losses of 3–11 % on the others); Mur z-face vectors (item 6; 0.982–1.012×, 6 rounds, 0.4–2 cores); per-line hook masks (item 7; 0.960–1.020×, GCPW 1.005× and 1.009× at 4 and 8 threads, 5–8 rounds); the split form of item 9, the distinct vectors in one table and four 16-bit ids per operator (SMA 0.952–0.999×, GCPW 0.933–0.954×, patch x5 and notch x3 0.967–1.023×, 5 rounds: the extra loads cost more than the smaller tables save; the geometry tables of Section 4.14 replace it); and a 50 ms tuning window (short runs GCPW 1.053×, notch tutorial 1.100×, but at real run lengths GCPW 1.017× [0.861–1.120] and SMA 0.993×; the default stays 150 ms). Not built: item 10, spin barriers for the start and stop of each processing interval (the boards' calls last 50–300 ms, so the two wake-ups of 20–40 μs per call are under 0.1 %; the patch tutorial could gain 1–2 % at most, an estimate). Skipped: item 8, since the Metal diamond engine replaces the base's GPU engine. Of other contributors' ideas, two were already in the fork, MrHighVoltage's thread tuner that scores real time steps (since cg8) and PR #231 [22] (the hook plan and the UPML rows along z), and two parts were not needed: the Debye half of PR #244 (the Debye slab's setup takes 0.02 s) and the threaded unfused dispersive hooks of PR #245.
- **On the GPU** (the Metal diamond engine, Section 6): a fused but unblocked sweep ($K = 1$) runs the two large models at 0.57–0.65× of the default $K = 4$ (0.54× on patch x5 at the widest tile; paired, 4 rounds, 2.5–4 cores of other load; `results/fused_sweep_k1_2026-11-04.txt`); the UPML folded into the cell update gained nothing (paired against cg24, 4 rounds: SMA 1.023× [0.934–1.106], notch tutorial 0.948×, notch x3 0.988×; `results/diamond_engine_cg24_cg27_2026-11-04.txt`); the I-J-K-N field order of the CPU engines is 7–9 % slower on the GPU than N-I-J-K (`results/diamond_engine_cg19_2026-11-04.txt`); 1 024 threads per tile, the pipeline maximum and the first default, cost 44 % on the notch tutorial and 21 % on the SMA launch (`results/diamond_engine_cg21_2026-11-04.txt`). A one-dispatch fused kernel was not built: it needs a second copy of both fields, and $K = 4$ already runs above the bound such a kernel has. On 2026-11-11 (cg31, Section 9), moving the Pre stages of Mur and of the conducting sheets into the half step

before, beside the cell update, saved a barrier per half step with identical fields but ran 2–4 % slower, and was dropped.

13 Discussion and future work

Three lessons generalise. First, measure the stock code on the real models before believing a roofline: the stock engine was not bandwidth-bound on the models that mattered, it was synchronisation- and instruction-bound, and the largest single gains (barriers, inner loop, the allocation-placement valley, the dispersive hooks) were not about bytes at all. Second, bit identity is a practical acceptance test for FDTD engine work: it is cheap to check, it forbids the reorderings that would otherwise creep in, it caught one real bug (the hook plan, wrong in two snapshots), and it confirmed the Debye bug that the review found by reading the code. Third, on a unified-memory laptop the GPU is another client of the same bus, and it pays only once it moves fewer bytes per cell update than the CPU can. The unfused GPU engines measured here (the base's and the Swift/Metal prototype) were at best about as fast as the fused CPU sweep. Fusing alone is not enough: the fused, unblocked Metal sweep ($K = 1$ of the diamond engine) runs the two large models at 1 730–1 840 MC/s, 1.25–1.30× the unfused GPU engine of the fork's base but 0.58–0.65× of $K = 4$ and 67–75 % of the 2 470–2 590 MC/s bound of Section 5.4, 85–90 GB/s at 49 B per cell, because its per-step structure (four dispatches, the tile edges, a threadgroup barrier between the two half steps) costs more than the bytes it saves (paired A/B, 4 rounds, 2.5–4 cores of other load; `results/fused_sweep_k1_2026-11-04.txt`). Blocked over time steps the GPU runs past that bound, 2 820–2 990 MC/s at $K = 4$ ($K = 2$, at 2 120–2 410, stays below it). Paired with the CPU engine of the same build at its best, 8 threads (Table 10: cg29, 4 rounds, other processes on a median of 0.5–0.8 cores), the Metal diamond engine ran patch x5 and notch x3 1.47× and 1.46× as fast and the GCPW line 1.17×; it was level on the SMA launch on its 1.071 M-cell mesh (1.00×), and on the two tutorials the CPU engine was ahead (the GPU at 0.97× on the notch tutorial and 0.93× on the patch tutorial). At cg19 the CPU engine was ahead on a 61^3 Mur box too (0.23 M cells, 2 281 against 2 099 MC/s at 4 threads, rough numbers under a load of 2–7; `results/diamond_engine_cg19_2026-11-04.txt`): the GPU runs below its own rate on a model that does not fill it. In a size sweep of three models (Figure 3) it led in every round from 1.7–2.9 M cells on, by 1.25–1.52× above 3 M; on smaller grids it waits for the CPU between short calls (18–27 % of the loop on the patch tutorial at cg29) or runs at a lower rate of its own (the notch tutorial), and the CPU engine is at its fastest between 0.6 and 1.9 M cells; below 1.7–2.9 M cells the two are within 0.89–1.23× of each other (Section 6). The build of 2026-11-11, cg31, raised the Metal engine's rates on the six models by 1.07–1.53× (three dispatches per block, UPML and dispersive stages on z-vectors or on the three components of a cell, seven tilings); in its paired rounds the Metal engine is ahead of the CPU engine at its best on five of them, by 1.08–1.59×, and level on the patch tutorial (Section 9, Table 18). Once both engines are near what they can draw from memory, the cheapest remaining wins are in the model. Still open: fp16 field storage (Section 12); larger Apple GPUs, on which luikore's diamond engine (PR #228 [20]) reports several times the CPU (Section 11; on this M5 it refused the six models, which all have Mur faces, and ran the PEC variants of five of them, speed-only cases, at 0.39–2.00× cg18, `results/others_2026-11-04/README.md`; on its own benchmark model cg31's Metal engine was level with it at 3.0 M cells and 1.17× as fast at 17 M, Table 19); and spreading one model over several machines (Section 11.1).

On the engine side, a second read-only optimisation review, made on 2026-11-02 after the work up to then, ranked what was left (Table 32); its gains were estimates. Its CPU items were built and measured for cg29 on 2026-11-05 (Section 4.14): items 1, 2 and 5 are kept, item 9 in another form and item 4 in part; items 3, 6 and 7 gained nothing outside the noise and were removed (Section 12); item 10 was not built and item 8 was skipped. Against the estimates, item 1 gained 12.5–18.3 % on the SMA launch (estimate 15–25 %), item 2 3.1–4.1 % on the GCPW line (8–12 %), and item 5 more than expected on the two boards at 8 threads (28.6 and 10.9 % against 0–3 and 3–8 %). The cost of the nf2ff dumps on the smallest model remains (14 % at snapshot cg8, project log; on cg18 at 4 threads 1 845.9 MC/s with them against 2 198.1 without, `results/patch_s1_n10500_base.txt`). The one item that review missed is done in cg29: the operator index, allocated per cell up to cg28 although only one entry per z-vector is used, is now allocated per z-vector, which saves up to 3 B per cell of DRAM traffic on meshes of up to 42 z lines (bit-identical, since only the layout changes): patch x5 1.119× at 4 threads and 1.101× at 8, notch x3 1.042× and 1.052×, the other four models 1.005–1.021× (paired A/B in one build against `OPENEMS_OP_INDEX_PER_CELL=1`, other processes on 0.3–1.4 cores; `results/cpu_items_cg29_2026-11-05.txt`, section 1).

The review also located why threads still wait at the barriers: the fused sweep runs static slabs that are rebalanced only between processing intervals, while macOS migrates threads between the clusters on millisecond timescales (per-x-line stealing is used only by the unfused plan). cg29 measured the wait (at 8 threads 24–27 % of the time at the two barriers, 10–22 % on the neighbour flags that replace them; Section 4.14). The review also separates the two boards, by its own estimates rather than by measurement: the GCPW

idea	expected gain GCPW / SMA / big models	measured on 2026-11-05 (cg29)	effort (h)	bit-identical
1. UPML y-vector path for PML boxes that are not full-z slabs (four consecutive y rows as one vector, lane gathers)	0 / +15–25 % / 0	kept (0019): SMA 1.183× (1 thr), 1.125× (4), 1.129× (8); 6 rounds, 0.4–0.6 cores	8–12	yes
2. NEON 4-position ADE update for conducting sheets (Lorentz line hooks)	+8–12 % / 0 / 0	kept (0018): GCPW 1.041× (4 thr), 1.031× (8); 8 rounds, 0.3–0.5 cores	5–8	yes
3. Thread-count cache per model and a 4-candidate search	about 1.2 s per run / +1 % / 0	dropped: at real run lengths GCPW 1.021× [0.893–1.044], SMA 1.002×; 5 rounds, 0.5–0.8 cores	3	yes
4. Stable P/E assignment by per-thread QoS, longer spin, faster slab balancing	0 / 0–8 % at 8 thr / 0–8 % at 8 thr	dropped: at 8 thr QoS split 0.949–1.357×, spin 200 μs 0.907–1.122×, 5 ms balancing 0.991–1.008×; 5 rounds, 0.3–0.5 cores; kept in part: no flag spin above the P-core count (0027)	2–3	yes
5. Neighbour flags instead of the two global barriers	0–3 % / +3–8 % / +4–10 % at 8 thr	kept (0025): 1.015–1.055× (4 thr), 1.073–1.286× (8; GCPW 1.286×, SMA 1.109×, big models 1.077–1.102×); 5 rounds, 0.3–0.6 cores	10–16	yes
6. Mur z-face vectors (lane gathers over y)	+3–8 % / 0 / 0	dropped: 0.982–1.012× (GCPW, patch tutorial, patch x5); 6 rounds, 0.4–2 cores	4	yes
7. Per-line active masks for the line hooks	+2–4 % / +1–2 % / 0	dropped: 0.960–1.020× (GCPW 1.005×, 1.009×); 5–8 rounds	2	yes
8. The base's Metal engine: one dispatch per hook for all Mur faces, skip empty hooks	GPU path +30–50 % on GCPW-class models (estimate) / +5–10 % / 0	skipped: the Metal diamond engine (Section 6) replaces the base's GPU engine	8	GPU only
9. Split packed coefficient tables (geometry part 48 B + a small material id)	0 / +3–6 % / +1–3 %	split form dropped (SMA 0.952–0.999×, GCPW 0.933–0.954×); kept as geometry tables (0023): SMA 1.067× (8 thr), 1.033× (auto), 1.025× (4), 0.992× (1); 8 rounds, 0.3–0.4 cores	3	yes
10. Spin barriers for the start/stop of each processing interval	tutorials +3–8 % / 0 / 0	not built: two wake-ups of 20–40 μs per call of 50–300 ms, under 0.1 %; patch tutorial 1–2 % at most (estimate)	1–2	yes

Table 32: Second optimisation review (read-only, 2026-11-02): the gain it expected on the GCPW line, the SMA launch and the two large models (estimates, made before any item was built), effort in hours, whether the result stays bit-identical, and the outcome measured on 2026-11-05 for snapshot cg29 (Section 4.14; paired interleaved A/B in one build, median of the per-round ratios, rounds and the other processes' load where recorded, the SMA launch on its 0.871 M-cell mesh; `results/cpu_items_cg29_2026-11-05.txt`, sections 1 and 2).

board looks instruction-bound in the P-cluster cache (its conducting-sheet update about 20–25 % of a step), while the SMA launch looks bus-bound, with its scalar PML path costing about 5.7 plain cells per PML cell. One part of that is now measured: with every z-vector on one operator (`OPENEMS_DBG_OPMASK=1` in a development build, wrong fields, timing only) the SMA launch ran 1.140×, 1.128× and 1.076× faster at 4, 8 and 1 threads, the GCPW line 1.086× and 1.027× at 4 and 8, and the two large models within the noise, so only the SMA launch streams its coefficient tables from memory (`results/cpu_items_cg29_2026-11-05.txt`, section 2).

cg31 (Section 9) worked on the Metal engine's per-step structure and on its extension stages: three dispatches per block instead of four, UPML and dispersive stages that take a z-vector or the three components of a cell at a time instead of single floats, Mur faces of one axis behind one barrier, seven tilings instead of four. Three items are open after it. On the CPU, temporal blocking with one cache-sized tile for all threads, as upstream master's `cpu` engine does it: the blocking is worth 1.15–1.49× to that engine on this machine, and on a 17 M-cell grid it is ahead of the fork's fused sweep (Section 9.4), while the tests of Section 12 gave every thread a sub-grid of its own. The fork has had this form since 2026-11-11, as an opt-in tiled sweep (patch 0040, Section 9.6): at 4 threads it is ahead of the fused sweep at its best on two large grids (1.08 and 1.23×) and behind on four other models (0.71–0.92×); on the M5 Pro, timed as a set of its own, it pays only on the 16.8 M-cell box (1.43×; Section 10). What stays open is a choice between the two sweeps at run time and, on the M5, a paired run of the tiled sweep against master's `cpu` engine. The fused sweep was run against that engine on the M5 Pro (Section 10.1); the largest open CPU item is the 8 M-cell dispersive case, where the fork is at 0.54 to 0.73 of master's rate and neither the packed hooks of patch 0042 nor the tiled sweep close the gap. In the Metal kernel, the cost of an extension that is compiled in: in timing-only builds of two snapshots the GCPW line ran 1.26–1.35× as fast without its Mur faces and 1.22–1.32× without its sheets, but skipping one of their stages at run time (on the later of the two) gave 1.02–1.07×, and what in the GPU's execution makes it so was not found. The third is a shorter excitation pulse (next paragraph).

Open on the modelling side: the same mesh-floor rule for the other builders; a convergence pair (coarse/fine) as part of every study; the SMA launch's top PML, eight cells over the whole x-y face although only the coax wave has to be absorbed there (a thinner one was not validated on the full-size board); a baseband excitation pulse for boards on which every conductor has a DC path (on the two boards it cut the steps 1.52× and 1.94× with S_{21} within 0.005 dB of the full run; a board with a conductor without a DC path, or with a series capacitor, where the signals may not settle, was not tested; Section 9.7); and the time step. openEMS's default step (`CalcTimestep_Var3`) is a per-node bound that is exact on a uniform mesh and conservative at grading steps: on the GCPW line the largest eigenvalue of the lossless update operator puts the leapfrog limit of the bulk update 1.26 times above it (computed for this report from the operator exported by cg18, `results/timestep_bound_gcpw_2026-11-03.txt`). A larger step was not adopted: the results would not be bit-identical, every mesh needs an eigenvalue solve, and stability with the Mur, PML, lumped and dispersive extensions, which that bound leaves out, is not proven.

Upstreaming: the CPU engine changes are platform-independent apart from a few guarded details: thread QoS and the core counts that start the thread search (since patch 0041 every core outside the efficiency class, before it the performance-core count) and, since cg29, end the spin of the neighbour flags (the performance-core count; macOS; other systems start at half the available hardware threads, and their flag waits spin like the barriers), the opt-in flush-to-zero (arm64), and the cache size that `OPENEMS_TBLOCK=auto` reads for the tiled sweep (the performance cores' L2 cache on macOS, elsewhere the last-level cache where the C library reports one; without a figure `auto` keeps the fused sweep); the separable nf2ff path needs a CBLAS, and only Apple's Accelerate is wired in. The Metal diamond engine is Apple-only: it is built with the base's Metal backend (`ENABLE_METAL`, on by default on Apple) and runs only when `--engine=metal-diamond` asks for it. The allocator stagger is not guarded: it applies on every platform, although the valley it removes was seen only with the macOS allocator, and on the M5 it costs nothing measurable outside the valley (Section 4.6). The fork is based on commit 7bb294f (2026-10-12) of upstream's `gpu-engine` branch. Upstream has since merged a reworked GPU engine into master (c3a956b, 2026-10-17), deleted the branch, and replaced the operator and engine classes the patches change (c0816a2, 2026-11-02), so a submission means porting each item and verifying it again against a stock build of master. Upstream has also put speed pull requests on hold until a refactoring of the operator classes is done [28], and several items here overlap pull requests already in that queue (#228, whose tile schedule the Metal diamond engine follows; #231; #244, whose UPML part cg29 takes; #245, whose strict-order check cg29 follows; Section 11), so a submission would start from what is still missing after them. Upstream's `AGENTS.md` also asks that a pull request claiming a speed-up include the `benchmark/bench.py compare` table of master against the branch, run interleaved on one machine, and that no dependency be added. Every case of that benchmark uses a sinusoidal source, which turns on the steady-state detection; up to cg28 that extension had no line hooks, so the fork ran those cases on the unfused plan. Since cg29 its line hooks and `TS_LOCAL` put them on the fused sweep and the neighbour flags (fused against unfused: pml 1.36× at 4 threads, inhomogeneous 1.29× and 1.34× at 4 and 8, the dispersive case, with Debye line hooks

too, 6.5× and 10.0×; Section 4.14). The tool was run on master and on the fork on 2026-11-10 (Section 8.4, Table 13), without its probes column, which needs master's Python module: on the `multithreaded` engine the fork is 1.23–10.0× master, and master's new `cpu` engine is level with or ahead of the fork's CPU engine at 8 M cells (on luikore's benchmark model, run interleaved on 2026-11-11, cg31's CPU engine was 1.27× master's `cpu` engine at 3.0 M cells and 0.86× at 17 M, the fork's row the better of 8 and 10 threads and master's at 8; Table 19; on the M5 Pro, paired and with a source on the mesh, the fork was ahead at 1 M cells, level to 14 % ahead at 8 M on the empty and PML cases (0.93× on PML at 5 threads), and behind on the dispersive case at 8 M, Section 10.1), so a submission of the CPU items would have to show a gain over that engine, not over `multithreaded`. The Metal diamond engine does not run the steady-state detection on the GPU: with it, every step runs the single-threaded CPU loops of the compressed engine (Section 6). A submission would also follow upstream's AI policy [8] (a `Generated-by:` trailer, a sign-off, and a human who has read every line).

14 Reproducibility and artefacts

fdtdKit, the subject of this report, is the FDTD kit of EMSKit, a family of electromagnetic solvers for Apple silicon; the family's other kits (femKit for finite elements, momKit for the method of moments and sbrKit for shooting and bouncing rays) are developed separately and are not covered here. Until 2026-11-05 fdtdKit was called emsKit, and the repository and its folder keep that name, as do the result files, the patches and the dated entries of `CHANGELOG.md`. The repository that accompanies this report, `https://github.com/anmartinez1256/emsKit` (archived at Zenodo, DOI 10.5281/zenodo.24118865), contains:

- `patches/openems/`: the fork as `git am` patches 0001–0043 on commit 7bb294f of upstream's `gpu-engine` branch (`BASE_origin_gpu-engine.txt`; upstream has since deleted the branch, so `BUILD.md` fetches the commit by its hash), and one cumulative diff that gives the same tree (211a3de). 0001–0006 are cg18, the build installed on the author's Mac from 2026-11-04 to 2026-11-06; 0007–0016 add the Metal diamond engine, one patch per snapshot cg19–cg28; 0017–0027 are the CPU items of cg29 (Section 4.14), so 0001–0027 give cg29, the build installed there from 2026-11-06 to 2026-11-11; 0028–0039 are the first twelve commits of 2026-11-11 (72a6492 to ff2538a, Section 9), so 0001–0039 give cg31, the build installed there since (tree dc4a352); 0040 (0ef186b, snapshot cg32d) adds the opt-in tiled sweep of Section 9.6; 0041 (8bcec90, snapshot cg33a), 0042 (c6ef5c5, cg33c) and 0043 (13350ca, the fork's head, cg33d) are the thread-search start, the packed dispersive hooks and their guard of Section 10.1; 0040 to 0043 are not installed. The result files carry these snapshot names. All 43 patch files were made again on 2026-11-13, when 0041 to 0043 were added (before that, 40 on 2026-11-11, when 0040 was added); in 0001–0040 only the numbering in the subject line changed. The fork's commits up to cg29 were rewritten for publication on 2026-11-07, with the author's public identity, the trailers of upstream's AI policy and the same trees, and pushed that day; the result files made before then and the entries of `CHANGELOG.md` dated before then give their earlier hashes, which `CHANGELOG.md` maps to the published ones. The version strings that the libraries of those snapshots print keep the hashes at which their build folders were configured, from before the rewrite; the fresh build of 2026-11-07 (pub29) prints the hash of its own `git am` commit, v0.37.0-45-g1d091c7, and the installed cg31 that of its commit, v0.37.0-57-gff2538a. Snapshot cg32d was built in the folder configured for cg31 and prints cg31's string, and cg33a to cg33d print v0.37.0-58-g0ef186b; cg33, rebuilt from the fork's head after configuring again, prints v0.37.0-61-g13350ca. `patches/csxcad/`: one patch on v0.7.0 (+9 lines), which the openEMS patches need. The changes made in this project to the author's simulation toolkit (rfsim) are not published, since the toolkit itself is private; Section 7 and `CHANGELOG.md` describe them.
- `bench/`: `bench.py`, `runner.py`, `models.py`, `check.sh`, `step1.sh`, and since 2026-11-04 `usage.py` (the other processes' CPU and GPU use during a run, which every benchmark line now carries) and `abtest.py` (the paired interleaved A/B of Section 4.14). The four tutorial models run on any openEMS install; the two boards import the author's rfsim toolkit, which is private: neither it nor the changes made to it in this project are published, so they run only where the toolkit is installed (`RFSIM_PATH`). `bench/validation/`: 15 checks, 12 against closed-form or exact references (dispersive slabs, a rectangular and a cylindrical cavity, a WR-90 guide, a microstrip line) and three that compare the engine under test with the basic engine of the same build on the dispersive slabs. Its runs are in `results/validation_<tag>.md` (and `.json`). On the M5 cg18 passes 15 of 15 with the multithreaded engine at 4 threads (`cg18_mt4`) and with the default engine and thread count (`cg18_fastest`), and so do cg29 (`cg29_mt4`, `cg29_fastest`; `cg29pre_*` are an earlier build of it), the fresh build of 2026-11-07 of the patches published then, 0001–0027 (`pub29_mt4`, `pub29_fastest`), and stock v0.37.0 with its defaults (`stock-v0.37.0_macos`); the Metal diamond engine passes 15 of 15 at cg23 and cg27 (`cg23_metal`, `cg27_metal`; `VALIDATION_ENGINE` selects the engine, the cylindrical cavity runs the cylinder engine whatever it says, and the dispersive slabs on the GPU equal the basic engine of the same build);

snapshot cg17 (patches 0001–0005) passes 15 of 15 with its defaults and with the multithreaded engine at 8 threads (`cg17`, `cg17_mt8`, two runs not recorded in this project's log). Earlier runs: stock v0.37.0 on a Linux x86_64 virtual machine passed 12 of 12 (`stock-v0.37.0_linux-sandbox`, the suite's earlier version without the three engine comparisons). Two negative controls fail as they should: a stock build with the Debye engine hooks disabled, on that machine, passes 2 of 3 (`negative-control_debye-hooks-disabled`); on the M5 a build of patches 0001–0004 (snapshot cg16, which has the Debye bug) with the multithreaded engine at 2 threads passes 4 of 6, and both Debye checks fail (`negative-control_patches-0001-0004_mt2`).

- `results/`: the benchmark tables, one file per model, step count and snapshot, with the load figure of every run, among them the full checks of cg18 (`*_cg18.txt`) and of a fresh build of the patches published on 2026-11-07, 0001–0027 (`*_pub29.txt`), and the comparison of stock, the fork's base and cg18 (`*_base.txt`); `summary.json` (best speed per model and snapshot; the cg17 tables and every table after cg18, those of cg19, cg21, cg22, cg29, cg29pre and pub29 and the `*gpu.txt` tables of cg20 to cg29 and pub29, are correctness checks with unpaired runs, many of them on a loaded machine, so their speeds do not compare; for a `*_cg2Ngpu.txt` or `*_pub29gpu.txt` table the entry is its multithreaded row with `OPENEMS_FTZ=1`); the mesh and cache-valley sweeps and the valley's placement test; the end-to-end runs of the two boards (`rfsim_e2e_2026-11-03.txt`, and on the GPU `rfsim_e2e_gpu_2026-11-04.txt`); the kernel comparisons (GCC against clang, instructions per loop iteration); the correctness checks of cg18; the validation runs; the memory and time-step tables; the initial review. From 2026-11-04 and 2026-11-05: the Metal diamond engine's snapshots (`diamond_engine_cg19_2026-11-04.txt` to `diamond_engine_cg28_tuner_2026-11-04.txt`, the Swift diamond mode `diamond_swift_2026-11-04.txt`, the thread sweep `diamond_sweep_cg21_2026-11-04.txt`, the `diamond_check` logs of cg21–cg23, the reduced CPU checks `*_cg19.txt`, `*_cg21.txt` and `*_cg22.txt`, and the GPU-against-CPU tables `*_cg`N`gpu.txt` of cg20–cg23 and cg27–cg29); the $K = 1$ measurement (`fused_sweep_k1_2026-11-04.txt`); the runs of upstream master and of PR #228 (`others_2026-11-04/`, with its `README.md`); the CPU items of cg29 (`cpu_items_cg29_2026-11-05.txt`, with the identity check tables `*_cg29.txt`, whose speeds are the faster of two unpaired runs under the check's load, and `*_cg29pre.txt` of an earlier build); the GPU against the CPU by grid size on cg29 (`gpu_crossover_cg29_2026-11-05.txt`, Figure 3); and the tables of cg29: stock, cg29 on the CPU and the Metal diamond engine side by side (`speed_cg29_2026-11-05.txt`, Table 10) and the end-to-end runs of cg29 on the CPU and the GPU (`rfsim_e2e_cg29_2026-11-05.txt`, Table 12), from which `paper/figures/tables.py` writes those two tables and the speed table of `README.md`. From 2026-11-10 and 2026-11-11: the common benchmarks (`common_benchmarks_2026-11-10.txt`, Section 8.4) and the records of the installed build, cg31 (Section 9): its Metal engine against cg29's (`metal_engine_cg31_2026-11-11.txt`), the three builds on luikore's model (`engines_on_luikore_model_2026-11-11.txt`; with cg29 the night before, `luikore_compare_cg29_2026-11-11.txt`) and the pulse test (`pulse_test_2026-11-11.txt`); and the record of the tiled sweep of patch 0040 (`tiled_sweep_cg32_2026-11-11.txt`, Section 9.6). From 2026-11-12, the second machine, an M5 Pro Mac mini (Section 10): bandwidth, kernels, thread sweep, speed table, valley, switches and the two boards end to end (`m5pro_speed_set_2026-11-12.txt`), and the tiled sweep (`m5pro_tiled_sweep_2026-11-12.txt`). From 2026-11-12 and 2026-11-13, on the M5 Pro, the build of patches 0041 to 0043: its identity checks, the thread search, the packed dispersive hooks, master's `cpu` engine against the fork with a source on the mesh, the tiled sweep at 8 M cells, and stock at 1 to 15 threads, which corrects the speed table of 2026-11-12 (`cpu_items_cg33_2026-11-13.txt`, Section 10.1); the earlier records carry dated notes where this paper's later sections correct them.
- `scripts/` (one line per script in `scripts/README.md`): the standalone kernels (`kb_idx.cpp`, `kb_tb.cpp` and, for the shared tile of Section 9.6, `kb_tile.cpp`) with the GCC-against-clang comparison (`gcc_vs_clang.sh`) and the instruction count per loop iteration (`loop_insns.py` with the program-counter sampler `pc_sampler.cpp`), the cache-valley box, sweep and placement test (`place.c`, `placement.sh`), the cylindrical-multigrid check, the setup/nf2ff phase timers, the probe-end and z-padding checks, the mesh sweep, the end-to-end runs (`scripts/e2e/`; `E2E_ENGINE=metal-diamond` in `run_one.py` runs them on the GPU), the memory and time-step checks, the HDF5 compare, the Debye regression check of patch 0005 (`scripts/checks/debye_check.sh`), the Metal diamond engine's bit-identity check against the multithreaded engine with `OPENEMS_FTZ=1` (`scripts/checks/diamond_check.sh` with `diamond_box.py` and `diamond_compare.py`), the GPU-against-CPU size sweep (`scripts/gpu_crossover/`: `sweep.sh`, `tiles.py`, `cells.py`, `analyze.py`, and `mkresult.py` with `header.txt`), the builds and runs of the other engines (`scripts/others/`: `run_others.sh`, `bench_ext.py`, `xcompare.py`), and from 2026-11-10 and 2026-11-11 the common benchmarks (`scripts/common_bench/`), the three builds on luikore's model (`scripts/luikore_compare/`), the closed cylinder (`scripts/cylinder_closed/`), the pulse test (`scripts/pulse/`) and the identity check and the timing of the tiled sweep (`scripts/tiled_sweep/`), and from 2026-11-12 the stages of the M5 Pro set (`scripts/m5pro/`: `common.zsh` with the gate, `stageA.zsh` to `stageE.zsh`, `wait_quiet.zsh`, `summarize.py`); from 2026-11-13 the check of the dispersive hooks (`scripts/ade_check/ade_check2.py`), the night queues of the thread search, the packed hooks, stock at 1

to 15 threads and master's `cpu` engine (`slotF.zsh`, `tune33.zsh`, `master33.zsh`) and the scripts that built master and ran it beside the fork (`scripts/m5pro/master/`).

- `swift_engine/`: the Metal kernel, with its diamond mode (`--diamond K`), its model builder and field compare.
- `Package.swift`, `Sources/` and `Tests/`: fdtdKit's command line `emskit fdtd` (version 0.1.0), a subcommand of the `emskit` tool of the EMSKit Swift package. It runs the tools above unchanged (`bench.py`, `abtest.py`, `check.sh`, the validation suite, the Debye and diamond checks, the end-to-end runs, or any openEMS Python script) and prints one JSON document per call; every speed and run time it prints carries the other processes' CPU and GPU use during that run, it installs nothing, and it refuses any location it would write inside the installed openEMS prefix or the snapshot folder. Its plan, with the log of its builds, tests and acceptance runs, is `Docs/fdtdKit/PLAN.md`. Written on 2026-11-05 and 2026-11-06 and corrected on 2026-11-09 and 2026-11-10 after two code reviews (Section 15), it was built and tested on 2026-11-10 with Swift 6.4.2 on macOS 27.0.1: its 244 unit tests passed (258 tests in 28 suites with the 14 smoke runs against the real tools). Since 2026-11-11 its list of the fork's switches had 41 names (the eight of cg31 and the three of the tiled sweep were added) and the suite one test more: 245 unit tests, 259 tests in 28 suites with the smoke runs; since 2026-11-13 it has 43 names (`OPENEMS_TUNE_START` and `OPENEMS_ADE_PACKED` were added), and the suite then had 260 tests (the 246 unit tests and the three smoke runs that start no simulation passed that day). The full smoke runs passed last on 2026-11-10, before that day's later corrections, as did the runs on cg29: `check --lite` gave results identical to stock on the six models, `diamond-check` 72 of 72, and the GCPW end-to-end run finished (34 300 steps). The checks of the installed cg29 (Section 3) were run with it.
- `paper/`: this report, its figures and the scripts that draw them, and `code_notes.md`, the description of every code change of patches 0001–0006 and of patches 0040 to 0043, written from the diffs (Section 4; patches 0007–0039 are described by their commit messages, `CHANGELOG.md` and Sections 6, 4.14 and 9). Six figure scripts read `results/`: the cache-valley figure reads its two sweep files, the crossover figure `gpu_crossover_cg29_2026-11-05.txt`, and the speed, thread, progress and bandwidth figures read `summary.json` (written by `figures/parse_results.py`); the bandwidth figure adds four bars from the project log. The mesh-sweep and run-time figures hold their values in the script, copied from `results/sma_mesh_sweep_2026-11-01.txt` and `results/rfsim_e2e_2026-11-03.txt`.

`BUILD.md` gives the build, install, verification and revert steps. Licences: openEMS is under the GPL and CSXCAD under the LGPL, both version 3 or later. The openEMS patches are under the GPL like the files they change, the CSXCAD patch under the LGPL like its files (the licence text is in `patches/csxcad/`); files copied from or based on openEMS keep their upstream copyright and licence notices; the harness, the scripts (the figure scripts in `paper/figures/` included), `paper/code_notes.md`, `swift_engine/` and the result files of this repository are under GPL-3.0-or-later (`LICENSE`); the text and figures of this report are under CC BY 4.0 (`paper/LICENSE-CC-BY-4.0.txt`). The Swift package of the command line (`Package.swift`, `Sources/`, `Tests/`) is under GPL-3.0-or-later: its Swift files and its Python launcher (`fdtdkit_run.py`) each carry `SPDX-License-Identifier: GPL-3.0-or-later`, and `LICENSE` holds the licence text.

15 Disclosure

The code changes, the benchmark harness, the command line, the measurements and most of this text were produced by AI assistants (Claude, Anthropic, operated through Claude Code) over seventeen days (2026-10-28 to 2026-11-13), directed by the author through a chat interface; the models are those recorded in the sessions' transcripts. In the main session: Claude Opus 5.5 on 2026-10-28 until the evening (the benchmark harness, the stock baseline and the engine work of patch 0001, snapshots A to cg8); Claude Fable 5.1 from then to the early morning of 2026-11-03 (it made the commits of patches 0001–0005, so their trailers name it, and the trailer of 0001 also names Claude Opus 5.5, which wrote that patch's code; it made the engine work of snapshots cg9 to cg17, the CSXCAD patch and the toolkit changes up to 2026-11-02, and most of the first version of this report); and Claude Opus 5.5 from then on: the final review of 2026-11-03 with patch 0006 (below), on 2026-11-04 the Metal diamond engine (patches 0007–0016) with its checks and the $K = 1$ measurement, and on 2026-11-05 the CPU items of cg29 (patches 0017–0027), the measurements of Tables 10 and 12 and the documentation pass that brought this report up to date with them, whose fact sheets and edits were made by Claude Opus 5.5 agents in workflows of the main session. The main session also ran the GPU-against-CPU size sweep of Figure 3 on 2026-11-05; five checkers and a skeptic, Claude Opus 5.5 agents in a read-only workflow, then recomputed its rows from the raw runs and checked its text. At the author's request, Claude Opus 5.5 agents in workflows of the main session renamed the documents to fdtdKit on 2026-11-05 and made the command line `emskit fdtd` on 2026-11-05 and 2026-11-06: its specification, code and tests, its build, its reviews and its acceptance runs. On 2026-11-06 the main session and Claude Opus 5.5 agents installed cg29 on the author's Mac, checked it from the install and brought this report up to date with

the command line and the install. Later that day the main session and Claude Opus 5.5 agents in one of its workflows applied the author's publishing decisions to the repository and to this report (the author line, the repository's address, the licence of the command line's Swift package, and leaving the toolkit changes out). On 2026-11-07, at the author's request, the main session rewrote the commits of the two fork branches for publication (the author's public identity and the trailers of upstream's AI policy, with the same trees), compared every rewritten commit with its old one and pushed the two branches to the author's GitHub forks; it then made the patches again from the published commits and replaced the hashes that point to the fork's code in this report and in the repository's files (Section 14). Claude Opus 5.5 agents in two read-only workflows of the main session checked this work: before the push the first rewrite and a first version of the changed files, after it the pushed branches and the final files. The same day the main session built the published patches from a fresh clone and ran the full check on that build (Section 3); Claude Opus 5.5 agents in a third read-only workflow checked its record against the raw logs, and the changed files. Later that day Claude Opus 5.5 agents in a fourth read-only workflow (six auditors, a skeptic and a critic) audited the copy of the repository made for publication; on 2026-11-08 Claude Opus 5.5 agents in another workflow of the main session applied the fixes that the audit proposed and the author's decisions of that day (among them the licence wording of Section 14) to the repository and to this report. The same day the author ran Claude Code's cloud code review (`/code-review ultra`; the model of its reviewers is not shown) on 18 of the command line's 31 source files (7 867 of the 11 634 lines of the reviewed version: the files that start programs, choose the libraries, write or delete files and decide pass or fail). It reported one bug (a binding without package metadata was taken for one that failed to import) and five minor issues. On 2026-11-09 a separate Claude Opus 5.5 session, opened by the author in the review's copy, wrote the six fixes and four tests. Claude Opus 5.5 agents in a read-only workflow of the main session checked them; the main session corrected what they found (one fix had dropped a fallback; the handling of an interruption in another needed three small changes), added five tests and carried all of it into the repository. From the evening of 2026-11-09 the main session ran on Claude Fable 5.2: its agents reviewed all 31 source files of the command line in a read-only workflow (four reviewers, a refuting verifier for each, a critic), and on 2026-11-10 agents of a second workflow fixed the confirmed findings and reviewed the fixes, three times; the main session ran the checks. After that period, on 2026-11-11, the main session (Claude Fable 5.2) made the build cg31 (patches 0028–0039), its measurements and Section 9; the search for the bottlenecks before it was the work of four read-only reviewers, each followed by a checker that tried to refute its findings (agents of the same model in a workflow of the main session), and the day before one read-only checker had recomputed the numbers of the common benchmarks of Section 8.4. The tiled sweep of Section 9.6 (patch 0040, whose trailer names Claude Fable 5.2), its checks and its measurements are of the same day; three read-only reviewers, each finding handed to a second reader (agents again), reviewed it before the commit (`results/tiled_sweep_cg32_2026-11-11.txt`). On 2026-11-12 the main session, on Claude Opus 5.5 from the night before, ran the set of Section 10 on the M5 Pro in a night queue on the machine (the tiled sweep's timing and stages A to E of `scripts/m5pro/`, which it had written that evening on Claude Fable 5.2) and wrote its two records; agents in a workflow of the main session (Claude Opus 5.5) wrote that section and brought the other documents up to date, and two read-only checkers and a fixer (agents again) checked that text against the two records and corrected it. On 2026-11-12 and 2026-11-13 the code of patches 0041 to 0043 (Section 10.1) was written by Claude Opus 5.5 (the hardening of 0043 by an Opus 5.5 subagent) and planned and reviewed by Claude Fable 5.2, in the main session; the trailers of the three commits name Claude Opus 5.5. The check script `ade_check2.py` was written by a Claude Sonnet subagent. The night runs of those two days were made by the main session on the M5 Pro, one job at a time. The flaw in upstream's benchmark cases (Section 8.4) was found on 2026-11-13, when that script's guard against all-zero probes stopped rows at the S size; the corrections to the stock baseline of the M5 Pro speed table (Section 10) and the other corrections of the same day came from a review of 2026-11-12 and from the new thread table. The documentation pass of 2026-11-13, this report included, was done by Claude Sonnet subagents directed and checked by Claude Fable 5.2. A separate Claude Opus 5.5 session in the repository's folder wrote the paragraph on concurrent work and measured two of the engines it lists (below); another Claude Opus 5.5 session wrote the work plan of 2026-11-04 (a working note that is not part of the published repository), which the author pasted into the main session. The multi-agent reviews and the research of 2026-11-04 and 2026-11-05 (a performance review of the diamond kernel, a review of its tiling tuner, the reading of the other contributors' pull requests and fork) ran as read-only workflows of the main session with Claude Opus 5.5 agents. Since 2026-11-07 the fork's commits carry the author's public git identity and name the model in a `Generated-by:` trailer above a `Signed-off-by:` line. The goal, the acceptance rule (bit identity, measured on their models), the plans and the choice of what to pursue next and what to reject were the author's, and so were the decisions recorded with the work, among them the toolkit defaults of 2026-11-04 and the rule of the same day that every benchmark line carries the other processes' CPU and GPU use. The mesh-floor recommendation of Section 7.3 (0.05 mm with a hold of 0.5) came from the assistants; it stands on the numbers re-measured on 2026-11-03, and the author adopted it as the toolkit's default on 2026-11-04 (toolkit commit 4a7e20e; on 2026-11-03 the

toolkit still had the earlier defaults, 0.02 mm and a hold of 1). Following arXiv's policy the assistants are not listed as authors. Two read-only reviews of the branch, besides the initial review of Section 3.4, by fresh instances of Claude Fable 5.1 (one for correctness, one for further optimisation) found no confirmed bug and located the conducting-sheet problem of Section 4. A second round of four such reviews (fork correctness, further optimisation, the rfsim changes, and this report's numbers against the result files) was run before publication; the correctness review found a real wrong-result bug (a Debye material losing its dispersion in the multithreaded engine, fixed in patch 0005, Section 4.13); the findings of all four are folded into Sections 4, 7 and 13, and the corrections they forced are listed in the repository's `CHANGELOG.md`. Section 11 (with the abstract's sentence on clusters), `ROADMAP.md` and the validation suite `bench/validation` with its two runs on the Linux virtual machine were added on 2026-11-03 by an AI assistant session that the project log does not record; its model is not recorded. The two runs of snapshot cg17 on the M5 (`results/validation_cg17*`) came later that day from a Claude Opus 5.5 session in the Claude desktop app, also outside the project log. The final review of 2026-11-03 was run by Claude Opus 5.5 (Anthropic): 30 reviewers, every finding checked by two verifiers, the nits by one. Its fixes are in patch 0006 and in these documents, and a large part of the present text was rewritten or added in that round. In the same round the session that coordinated the review (Claude Opus 5.5) re-measured the end-to-end runs, on stock v0.37.0 and on cg18 (patches 0001–0006), and the SMA mesh sweep on cg18 (`results/rfsim_e2e_2026-11-03.txt`): Tables 11, 6 and 7 and Figures 8 and 4 show those runs (Table 7 and Figure 4 also keep one run of 2026-11-01 as history). The same session also made the full check of cg18 against stock (`results/*_cg18.txt`), the comparison of stock, the fork's base and cg18 (`*_base.txt`), the valley's placement test on cg18 and its sweep on stock, and the GCC/clang comparison in the standalone kernel with the instruction counts of the inner loops. It also made the correctness checks of cg18 and three validation runs on the M5 (cg18 twice, stock once), and re-ran the memory and time-step checks on cg18. The validation suite's negative control on the M5 came from the final review itself. The author has not yet read every line of the fork; this draft is marked accordingly. On 2026-11-04, at the author's request, Claude Opus 5.5 (Claude Code) added the paragraph on concurrent work of Section 11 with its references and narrowed the sentences it affects (the abstract, the introduction, Sections 5.4, 11, 12, 13 and this one's upstreaming paragraph); it read the public pull requests, discussions and benchmark notes it cites, and when it wrote the paragraph it had built or run none of that work. Later that day the same session built upstream master (5fecbb5, with the Metal engine merged as c3a956b) and luikore's PR #228 (55ce11b) and ran them on the six models next to stock and cg18 (`results/others_2026-11-04/`, Section 11).

Where the numbers come from: the benchmark tables (with the full checks of cg18, cg29 and the fresh build of 2026-11-07 and the comparison of stock, the fork's base and cg18), the end-to-end and probe-end runs (Tables 11, 12 and 6), the mesh sweep (Table 7), the valley sweeps, the kernel comparisons, the correctness checks of cg18, the validation runs, and the measurements of 2026-11-04 and 2026-11-05 (the Metal diamond engine, the $K = 1$ sweep, the other engines, the CPU items of cg29, the GPU-against-CPU size sweep of Figure 3 and the table of cg29 against stock, Table 10) and the records of 2026-11-10, 2026-11-11, 2026-11-12 and 2026-11-13 that Sections 8.4, 9, 10 and 10.1 name are files in `results/`. So are the smaller checks made for this report during the final review, which are marked where they appear: the memory figures of Section 11.1, the time-step bound of Section 13 and the placement test of Section 4.6 (all three on cg18), and the flush-to-zero check of Section 3.3 (snapshot cg17); only the rerun of the Debye check with 2 threads (Section 4.13) is not. Two sets of checks of 2026-11-06 are not in `results/` either: the command line's acceptance runs and the checks of the installed cg29 (Section 3), both logged in `Docs/fdtdKit/PLAN.md`, section 16. A few figures of Section 4.14 come from the commit messages of the published patches and are marked so. Numbers that exist only in the project log, which is not published, are marked as project-log numbers where they appear; among them, from before the final review, the setup and far-field timings, the 201^3-box figures and the standalone-kernel figures, and part of the list of Section 12. Numbers that were computed or estimated rather than measured say so: the bytes per cell of the traffic model, the bounds of Sections 5.4 and 11 (the fused GPU bound now has the $K = 1$ measurement beside it), and the expected gains of Table 32, which now stand beside their measured outcomes.

References

[1] T. Liebig, A. Rennings, S. Held, D. Erni, "openEMS – a free and open source equivalent-circuit (EC) FDTD simulation platform supporting cylindrical coordinates suitable for the analysis of traveling wave MRI applications," *Int. J. Numer. Model.* 26(6), 680–696, 2013. Software: `https://github.com/thliebig/openEMS`.

[2] K. S. Yee, "Numerical solution of initial boundary value problems involving Maxwell's equations in isotropic media," *IEEE Trans. Antennas Propag.* 14(3), 302–307, 1966.

[3] A. Taflove, S. C. Hagness, *Computational Electrodynamics: The Finite-Difference Time-Domain Method*, 3rd ed., Artech House, 2005.

[4] G. Mur, "Absorbing boundary conditions for the finite-difference approximation of the time-domain electromagnetic-field equations," *IEEE Trans. Electromagn. Compat.* 23(4), 377–382, 1981.

[5] J.-P. Berenger, "A perfectly matched layer for the absorption of electromagnetic waves," *J. Comput. Phys.* 114(2), 185–200, 1994.

[6] S. Williams, A. Waterman, D. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Commun. ACM* 52(4), 65–76, 2009.

[7] Apple Inc., "Apple unleashes M5, the next big leap in AI performance for Apple silicon," press release, October 2025 (unified memory bandwidth 153 GB/s). `https://www.apple.com/newsroom/2025/10/apple-unleashes-m5-the-next-big-leap-in-ai-performance-for-apple-silicon/`

[8] openEMS project, `AI_POLICY.md` (disclosure trailers, sign-off, licence), in the openEMS repository, 2026. `https://github.com/thliebig/openEMS/blob/master/AI_POLICY.md`

[9] Ansys Inc., "An introduction to multi-frequency adaptive meshing in HFSS," Ansys Electronics documentation, 2025. `https://ansyshelp.ansys.com/public/Views/Secured/Electronics/v252/en/PDFs/An%20Introduction%20to%20Multi-Frequency%20Adaptive%20Meshing%20in%20HFSS.pdf`

[10] Dassault Systèmes, *CST Studio Suite GPU Computing Guide*. `https://updates.cst.com/downloads/GPU_Computing_Guide.pdf`

[11] Remcom Inc., "XStream GPU acceleration for XFdtd." `https://remcom.com/xf-xstream`

[12] NVIDIA Corp., *NVIDIA H100 Tensor Core GPU*, product page (H100 SXM: 3.35 TB/s memory bandwidth), fetched 2026-11-03. `https://www.nvidia.com/en-us/data-center/h100/`

[13] A. F. Oskooi, D. Roundy, M. Ibanescu, P. Bermel, J. D. Joannopoulos, S. G. Johnson, "MEEP: A flexible free-software package for electromagnetic simulations by the FDTD method," *Comput. Phys. Commun.* 181(3), 687–702, 2010.

[14] C. Warren, A. Giannopoulos, I. Giannakis, "gprMax: Open source software to simulate electromagnetic wave propagation for Ground Penetrating Radar," *Comput. Phys. Commun.* 209, 163–170, 2016.

[15] O. Cessenat, "Sophie, an FDTD code on the way to multicore, getting rid of the memory bandwidth bottleneck better using cache," arXiv:1301.4539, 2013.

[16] T. Fukaya, T. Iwashita, "Time-space tiling with tile-level parallelism for the 3D FDTD method," in *Proc. HPC Asia 2018*, ACM, 2018. doi:10.1145/3149457.3149478.

[17] T. Malas, G. Hager, H. Ltaief, H. Stengel, G. Wellein, D. Keyes, "Multicore-optimized wavefront diamond blocking for optimizing stencil updates," *SIAM J. Sci. Comput.* 37(4), C439–C464, 2015; arXiv:1410.3060.

[18] biergaizi, "Notes on HPC programming and openEMS optimizations," openEMS-Project GitHub Discussion #154, 2023–2024. `https://github.com/thliebig/openEMS-Project/discussions/154`

[19] biergaizi, "[FAQ] Why are openEMS simulations slow, and how to make them faster," openEMS-Project GitHub Discussion #29, 2023. `https://github.com/thliebig/openEMS-Project/discussions/29`

[20] luikore, "Metal engine and diamond wavefront," openEMS pull request #228, opened 2026-09-29; benchmark and design notes `macos/doc/metal-benchmark.rst` (M5 Max results recorded 2026-09-17, M4 Pro 2026-09-28) and `macos/doc/metal-engine.rst` in luikore's fork of openEMS, fetched 2026-11-04. `https://github.com/thliebig/openEMS/pull/228`, `https://github.com/luikore/openEMS/blob/master/macos/doc/metal-benchmark.rst`

[21] S. Mollet, "FDTD: add a GPU engine with Metal and HIP backends," openEMS pull request #235, opened 2026-10-08; in master as commit c3a956b, 2026-10-17. `https://github.com/thliebig/openEMS/pull/235`

[22] ismailakdag, "FDTD: faster multithreaded engine with PML and other extensions," openEMS pull request #231, opened 2026-10-05. `https://github.com/thliebig/openEMS/pull/231`

[23] MrHighVoltage, "Many optimizations: Vulkan GPU, AVX2, temporal blocking, better output-handling," openEMS-Project GitHub Discussion #492, 2026-10-26; fork at `https://github.com/MrHighVoltage/openEMS`. `https://github.com/thliebig/openEMS-Project/discussions/492`

[24] unsettledroell404, "Multithreaded Debye and Lorentz material updates," openEMS pull request #245, opened 2026-10-26. `https://github.com/thliebig/openEMS/pull/245`

[25] nori-proto, "extensions: set up the UPML and Debye coefficients on several threads," openEMS pull request #244, opened 2026-10-26. `https://github.com/thliebig/openEMS/pull/244`

[26] whahnsr, "Optimize mesh delta calculation," openEMS pull request #230, opened 2026-10-03. https://github.com/thliebig/openEMS/pull/230

[27] A. Iercosan-Lucaci (lucaci32u4), "GPU: New SYCL (Intel) backend," openEMS pull request #243, opened 2026-10-26. https://github.com/thliebig/openEMS/pull/243

[28] T. Liebig, "GPU and speed contributions: where this is going," openEMS-Project GitHub Discussion #495, 2026-10-31, with replies to 2026-11-03. https://github.com/thliebig/openEMS-Project/discussions/495

[29] Apple Inc., *macOS Tahoe 26.2 Release Notes*, section "RDMA over Thunderbolt" (low-latency communication between Thunderbolt 5 hosts), and "TN3205: Low-latency communication with RDMA over Thunderbolt," technical note, first published 2026-03-19 (available starting with macOS 26.2 on Macs with Apple silicon with Thunderbolt 5); Apple Developer Documentation, fetched 2026-11-08. https://developer.apple.com/documentation/macos-release-notes/macos-26_2-release-notes, https://developer.apple.com/documentation/technotes/tn3205-low-latency-communication-with-rdma-over-thunderbolt

[30] Apple Inc., *Mac mini technical specifications*, fetched 2026-11-03 (M5 Pro: up to 64 GB, 307 GB/s). https://www.apple.com/mac-mini/specs/

[31] "Apple announces the M5 Ultra Mac Studio with up to 512GB of RAM," Macworld, August 2026 (M5 Ultra: 1.2 TB/s). https://www.macworld.com/article/3220024/apple-announces-the-m5-ultra-mac-studio-with-up-to-512gb-of-ram.html

[32] J. D. McCalpin, "Memory bandwidth and machine balance in current high performance computers," *IEEE Computer Society Technical Committee on Computer Architecture (TCCA) Newsletter*, pp. 19–25, December 1995; STREAM version 5.10, https://www.cs.virginia.edu/stream/

[33] C. Warren, A. Giannopoulos, A. Gray, I. Giannakis, A. Patterson, L. Wetter, and A. Hamrah, "A CUDA-based GPU engine for gprMax: Open source FDTD electromagnetic simulation software," *Computer Physics Communications*, vol. 237, pp. 208–218, 2019.

[34] G. Mie, "Beiträge zur Optik trüber Medien, speziell kolloidaler Metallösungen," *Annalen der Physik*, vol. 330, no. 3, pp. 377–445, 1908.